

UNIVERSIDAD DE ALMERIA

ESCUELA SUPERIOR DE INGENIERÍA

Desarrollo de un videojuego roguelike 2D con generación procedural en Unity

Curso: 2025/2026

Alumno/a:

Naoufal Saidi Hassani

Director/es:

Nicolás Padilla Soriano
Javier Criado Rodríguez

Dedicado a mi familia, por su apoyo incondicional y su amor constante, que han sido mi fuerza motriz en cada paso de este viaje académico.

ÍNDICE GENERAL

	Página
ÍNDICE DE FIGURAS	IX
ÍNDICE DE TABLAS	XII
ABREVIATURAS	XIV
Resumen y Abstract	xv
1. Introducción	1
1.1. Contexto	1
1.2. Motivación y justificación	2
1.3. Objetivos	3
1.3.1. Objetivo general	3
1.3.2. Objetivos específicos	3
1.4. Descripción del proyecto	4
1.5. Metodología	5
1.5.1. Tipo de investigación	5
1.5.2. Metodología de desarrollo de software	5
1.5.3. Proceso de implementación y validación	6
1.5.4. Ejemplos representativos de iteraciones y sus resultados	8
1.6. Planificación temporal	8
1.6.1. Estimación de la dedicación	8
1.6.2. Fases del proyecto	9
1.6.3. Representación gráfica de la planificación	10
1.7. Estructura de la memoria	10
2. Estado del arte	11
2.1. El subgénero <i>roguelike</i> : definición y características	11
2.1.1. Origen histórico y evolución	11
2.1.2. Características técnicas fundamentales	12
2.1.3. Importancia del género en la industria del videojuego	13
2.1.4. Distinción entre <i>roguelike</i> y <i>roguelite</i>	14

2.1.5. Generación procedural de contenido	14
2.1.6. Inteligencia artificial en videojuegos 2D	16
2.1.7. Mecánicas de combate en <i>roguelikes</i>	17
2.2. Análisis de mercado: referencias comerciales del subgénero	18
2.2.1. The Binding of Isaac (2011, Edmund McMillen)	18
2.2.2. Enter the Gungeon (2016, Dodge Roll)	19
2.2.3. Hades (2020, Supergiant Games)	20
2.2.4. Dead Cells (2018, Motion Twin)	21
2.2.5. Tabla comparativa de referencias comerciales	22
2.2.6. Elementos diferenciadores de <i>Ascension</i>	23
2.3. Análisis de herramientas y tecnologías	24
2.3.1. Motores de desarrollo de videojuegos	24
2.3.2. Tabla comparativa de motores de desarrollo	26
2.3.3. Justificación de la elección de Unity y C#	26
2.3.4. Correspondencia entre funcionalidades de Unity/C# y subsistemas del proyecto	27
2.3.5. Otras herramientas utilizadas	27
2.4. Patrones de diseño en el desarrollo de videojuegos	29
2.4.1. Patrón <i>Singleton</i>	29
2.4.2. Patrón <i>Observer</i>	29
2.4.3. Patrón <i>Strategy</i>	30
2.4.4. Patrón <i>Template Method</i>	30
2.4.5. Arquitectura basada en componentes (Entidad-Componente)	30
2.4.6. Máquina de estados (<i>State Machine</i>)	30
2.5. Síntesis del estado del arte	30
3. Diseño del sistema	33
3.1. Terminología utilizada en este capítulo	33
3.2. Requisitos funcionales: historias de usuario	35
3.2.1. Sistema de combate	35
3.2.2. Sistema de movilidad	36
3.2.3. Sistema de progresión de salas	36
3.2.4. Sistema de enemigos	36
3.2.5. Sistema de progresión y puntuación	37
3.2.6. Sistema de generación procedural	38
3.2.7. Sistema de interfaz de usuario	38
3.2.8. Historias técnicas	38
3.3. Propuesta arquitectónica	39
3.3.1. Visión general de la arquitectura	39
3.3.2. Flujo de escenas	42

3.3.3. Gestores centrales (<i>managers</i>)	42
3.3.4. Diagrama de dependencias	42
3.3.5. Flujo de producción: del activo gráfico al juego	45
3.4. Patrones y mecanismos de diseño aplicados	51
3.4.1. Patrón <i>Singleton</i>	51
3.4.2. Patrón <i>Observer</i>	51
3.4.3. Patrón <i>Strategy</i>	52
3.4.4. Patrón <i>Template Method</i>	52
3.4.5. Arquitectura basada en componentes (Entidad-Componente)	53
3.4.6. Máquina de estados (<i>State Machine</i>)	53
3.4.7. Interacción entre patrones: análisis de flujos transversales	54
4. Desarrollo de Ascension	57
4.1. Sistema de gestión central	57
4.1.1. GameManager: controlador principal del juego	57
4.1.2. RoomFlowController: control de progresión de salas	63
4.1.3. ScoreManager: sistema de puntuación y persistencia	64
4.2. Sistema del jugador	64
4.2.1. PlayerController: control de movimiento	64
4.2.2. PlayerHealth: sistema de vida	67
4.2.3. PlayerSpawner: instanciación del jugador	70
4.2.4. PlayerClass: datos de configuración mediante <i>ScriptableObjects</i>	70
4.3. Sistema de enemigos	71
4.3.1. Enemy: clase base y patrón <i>Template Method</i>	71
4.3.2. SlimeGreen: enemigo francotirador	74
4.3.3. SlimeRed: enemigo perseguidor	78
4.3.4. SlimeBlue: enemigo saltador	78
4.3.5. EnemyManager: gestión de aparición de enemigos	81
4.3.6. Configuración de enemigos: EnemyData	83
4.3.7. Síntesis comparativa de los tipos de enemigo	84
4.3.8. Diagrama de flujo de decisión de la IA por tipo de enemigo	85
4.4. Sistema de armas y combate	88
4.4.1. Weapon: clase base del sistema de armas y patrón <i>Strategy</i>	88
4.4.2. MeleeWeapon: armas cuerpo a cuerpo	89
4.4.3. RangedWeapon: armas a distancia	90
4.4.4. Projectile: comportamiento de proyectiles	91
4.4.5. WeaponData: configuración de armas mediante <i>ScriptableObjects</i>	91
4.4.6. Flujo completo de daño: del ataque a la puntuación	92
4.5. Sistema de generación procedural	94

4.5.1. RoomGenerator: generación de salas	95
4.5.2. VariantTile: tiles con variantes visuales y selección determinista	99
4.5.3. TileEffect: efectos de tiles	100
4.5.4. FloorTileManager: aplicación de efectos	100
4.5.5. DamageBoostManager: bonificación de daño	102
4.6. Sistema de gestión de salas	102
4.6.1. RoomController: control de sala individual	102
4.6.2. Stairs: lógica de escaleras	104
4.7. Sistema de interfaz de usuario	104
4.7.1. HeartDisplay: representación de vida	104
4.7.2. PauseMenu: menú de pausa	105
4.7.3. Pantallas de fin de partida	105
4.7.4. ScoreboardMenuUI: historial de puntuaciones	106
4.8. Sistemas preparados para extensión	107
4.8.1. Sistema de jefes con fases múltiples	107
4.8.2. Generación procedural de armas con rareza	107
4.9. Compilación y ejecución	108
4.9.1. Proceso de compilación	108
4.9.2. Estructura del ejecutable	108
4.9.3. Ejecución del juego	109
5. Conclusiones y trabajo futuro	111
5.1. Cumplimiento de objetivos	111
5.2. Valoración del trabajo realizado	112
5.3. Limitaciones del proyecto	113
5.4. Líneas de trabajo futuro	113
5.4.1. Integración del sistema de jefes con fases múltiples	113
5.4.2. Integración del sistema de generación procedural de armas	114
5.4.3. Ampliación del contenido	114
5.4.4. Mejoras técnicas	114
5.5. Reflexión final	115
Bibliografía	115
A. Artefactos software	119
A.1. Estructura del proyecto	119
A.2. Arquitectura del sistema	119
A.3. Diagramas del sistema	120
A.3.1. Diagrama de clases por secciones	122

ÍNDICE DE FIGURAS

1.1. Diagrama de Gantt con la distribución temporal y la dedicación en horas de las ocho fases del proyecto (elaboración propia).	10
2.1. Captura de pantalla del videojuego original <i>Rogue</i> (1980), mostrando la interfaz basada en caracteres ASCII y la disposición procedural de las salas. <i>Fuente: captura histórica del juego.</i>	11
2.2. Gráfico que muestra la evolución del número de títulos <i>roguelike</i> publicados en Steam por año (2006–2026). <i>Fuente: SteamDB</i>	13
2.3. Captura de pantalla de <i>The Binding of Isaac: Rebirth</i> en la que se observa la perspectiva cenital, la estructura de sala cerrada con puertas y la interfaz de vida mediante corazones en la esquina superior izquierda. <i>Fuente: captura del juego en Steam.</i>	19
2.4. Captura de pantalla de <i>Enter the Gungeon</i> en la que se muestra la mecánica de esquiva (<i>dodge roll</i>) durante el combate y la variedad de proyectiles en pantalla. <i>Fuente: captura del juego en Steam.</i>	20
2.5. Captura de pantalla de <i>Hades</i> en la que se aprecia el sistema de combate en tiempo real con arma cuerpo a cuerpo, la variedad de enemigos simultáneos y la interfaz de estado del personaje. <i>Fuente: captura del juego en Steam.</i>	21
2.6. Captura de pantalla de <i>Dead Cells</i> en la que se observa el combate lateral con armas cuerpo a cuerpo. <i>Fuente: captura del juego en Steam.</i>	22
2.7. Captura de pantalla de <i>Ascension</i> durante una partida, mostrando la perspectiva cenital, la sala generada proceduralmente con tiles especiales visibles (hielo, lodo), enemigos activos y la interfaz de corazones. <i>Elaboración propia.</i>	23
2.8. Captura de pantalla de <i>Ascension</i> mostrando la pantalla de selección de clase. <i>Elaboración propia.</i>	24
2.9. Captura de la interfaz de LibreSprite durante la edición de un <i>sprite</i> del juego <i>Ascension</i> , mostrando la paleta de colores y el lienzo de edición. <i>Elaboración propia.</i>	29
3.1. Diagrama de componentes de la arquitectura general del sistema. <i>Elaboración propia.</i>	41
3.2. Diagrama del flujo de escenas. <i>Elaboración propia.</i>	42
3.3. Diagrama de dependencias entre los módulos del sistema. <i>Elaboración propia.</i>	44

3.4. Configuración de importación de un <i>sprite</i> en el inspector de Unity, mostrando los parámetros PPU, filtrado y compresión. <i>Elaboración propia.</i>	46
3.5. Vista del inspector de Unity mostrando los componentes configurados en el <i>GameObject</i> del jugador: <i>SpriteRenderer</i> , <i>Rigidbody2D</i> , <i>CircleCollider2D</i> , scripts y <i>Animator</i> . <i>Elaboración propia.</i>	47
3.6. Vista del inspector de Unity mostrando los componentes de un enemigo <i>Slime Verde</i> , con detalle del <i>CircleCollider2D</i> y sus dimensiones. <i>Elaboración propia.</i>	48
3.7. Editor de <i>tilemaps</i> de Unity mostrando las dos capas (muros y suelo). <i>Elaboración propia.</i>	49
3.8. Controlador de animación (<i>Animator Controller</i>) del jugador en Unity, mostrando los estados y las transiciones entre animaciones de reposo, movimiento y esquiva. <i>Elaboración propia.</i>	50
3.9. Diagrama de la composición del jugador. <i>Elaboración propia.</i>	53
3.10. Diagrama de los estados del juego. <i>Elaboración propia.</i>	54
4.1. Diagrama de estados UML del <i>GameManager</i> . <i>Elaboración propia.</i>	59
4.2. Diagrama de la coordinación entre gestores al iniciar una nueva partida. <i>Elaboración propia.</i>	61
4.3. Diagrama del flujo de completar una sala. <i>Elaboración propia.</i>	62
4.4. Captura de pantalla del menú principal de <i>Ascension</i> , mostrando las opciones de nueva partida, tabla de puntuaciones y salir. <i>Elaboración propia.</i>	62
4.5. Diagrama del ciclo de funcionamiento de generación de salas. <i>Elaboración propia.</i>	63
4.6. Diagrama del funcionamiento de la esquiva. <i>Elaboración propia.</i>	66
4.7. Ciclo de vida del jugador. <i>Elaboración propia.</i>	69
4.8. Diagrama de la FSM del <i>SlimeGreen</i> . <i>Elaboración propia.</i>	74
4.9. Diagrama de la FSM del <i>SlimeBlue</i> . <i>Elaboración propia.</i>	79
4.10. Captura de pantalla de <i>Ascension</i> durante un encuentro con un enemigo jefe, mostrando el incremento visual de tamaño respecto a los enemigos estándar y los proyectiles de mayor escala. <i>Elaboración propia.</i>	81
4.11. Diagrama de flujo de decisión de la IA por tipo de enemigo. <i>Elaboración propia.</i>	85
4.12. Diagrama de clases UML de la jerarquía de enemigos. <i>Elaboración propia.</i>	86
4.13. Diagrama de secuencia UML del flujo de combate. <i>Elaboración propia.</i>	87
4.14. Diagrama de flujo del sistema de daño. <i>Elaboración propia.</i>	93
4.15. Diagrama de clases UML del sistema de armas. <i>Elaboración propia.</i>	94
4.16. Diagrama de secuencia UML de la generación procedural de una sala. <i>Elaboración propia.</i>	98
4.17. Diagrama de secuencia UML del ciclo de progresión entre salas. <i>Elaboración propia.</i>	103

4.18. Captura de pantalla del menú de pausa de <i>Ascension</i> , mostrando las opciones de reanudar y menú principal. <i>Elaboración propia.</i>	106
A.1. Diagrama de clases del sistema	121
A.2. Diagrama de clases del sistema (sección 1: interfaz de usuario, jugador y armas)	122
A.3. Diagrama de clases del sistema (sección 2: gestión de juego, eventos y enemigos)	123
A.4. Diagrama de clases del sistema (sección 3: generación procedural y efectos de tiles)	124
A.5. Diagrama de casos de uso del sistema	126

ÍNDICE DE TABLAS

2.1. Comparativa entre <i>roguelike</i> clásico y <i>roguelite</i>	14
2.2. Comparativa de técnicas de generación procedural y su adecuación al proyecto.	16
2.3. Comparativa de técnicas de IA para enemigos en videojuegos 2D.	17
2.4. Comparativa entre referencias comerciales y el proyecto <i>Ascension</i>	22
2.5. Comparativa de motores de desarrollo de videojuegos.	26
2.6. Correspondencia entre funcionalidades de Unity/C# utilizadas en el proyecto y sus equivalentes en motores alternativos.	27
2.7. Herramientas complementarias utilizadas en el desarrollo.	28
2.8. Comparativa de herramientas para producción de gráficos 2D.	28
3.1. Glosario de términos técnicos utilizados en este capítulo.	33
3.2. Gestores centrales del sistema.	42
4.1. Eventos del GameManager.	60
4.2. Estados, transiciones y acciones de la FSM del SlimeGreen.	74
4.3. Estados y transiciones de la FSM del SlimeBlue.	79
4.4. Comparativa de comportamiento, parámetros y rol táctico de los tres tipos de enemigo.	84
4.5. Probabilidades y formato de distribución de tiles especiales.	97

ABREVIATURAS

API	Application Programming Interface
AI	Artificial Intelligence
CSV	Comma-Separated Values
ECS	Entity-Component System
ESI	Escuela Superior de Ingeniería
FSM	Finite State Machine
GPC	Generación Procedural de Contenido
IA	Inteligencia Artificial
INSO	Ingeniería del Software
JSON	JavaScript Object Notation
PNJ	Personaje No Jugador
PPU	Pixels Per Unit
SBSE	Search Based Software Engineering
TFG	Trabajo Fin de Grado
UI	User Interface
UAL	Universidad de Almería
UML	Unified Modeling Language

RESUMEN Y ABSTRACT

Este Trabajo de Fin de Grado presenta Ascension, un videojuego 2D roguelike desarrollado en Unity. El sistema implementa generación procedural, IA enemiga, gestión de estados y un sistema de puntuación. La arquitectura utiliza ScriptableObjects y patrones de diseño (Singleton, Observer, Strategy, Template Method) para asegurar el desacoplamiento y la mantenibilidad del código. Los resultados confirman una solución robusta y extensible, con mejoras futuras previstas en el diseño de jefes finales y el sistema de audio.

Abstract. This Final Degree Project presents Ascension, a 2D roguelike game developed in Unity. The system features procedural generation, enemy AI, state management, and score tracking. The architecture employs ScriptableObjects and design patterns (Singleton, Observer, Strategy, Template Method) to ensure modularity and maintainability. Results demonstrate a robust and extensible architecture, with future work focusing on boss integration and the audio systems.

1 INTRODUCCIÓN

1.1 CONTEXTO

La industria del videojuego se ha consolidado durante las últimas décadas como uno de los sectores tecnológicos de mayor crecimiento a nivel mundial. En 2020, el mercado global de videojuegos generó ingresos estimados de 159.300 millones de dólares [1]. Esta magnitud económica, unida a la constante evolución tecnológica del sector, convierte el desarrollo de videojuegos en un campo de estudio relevante para la ingeniería informática.

Desde la perspectiva de la ingeniería del software, el desarrollo de videojuegos plantea desafíos técnicos de considerable complejidad. Entre ellos destacan la arquitectura de sistemas en tiempo real, la implementación de algoritmos de inteligencia artificial, la generación procedural de contenido (creación automática de elementos del juego mediante algoritmos) y el diseño de interfaces interactivas. Este contexto convierte el desarrollo de un videojuego en un ámbito adecuado para la realización de un Trabajo de Fin de Grado (TFG) en Ingeniería Informática, al permitir la aplicación práctica e integrada de conocimientos adquiridos durante la formación académica en áreas como la programación orientada a objetos, las estructuras de datos, la arquitectura de software y la ingeniería del software.

Los videojuegos pertenecientes al subgénero *roguelike* presentan una serie de características técnicas que los convierten en un objeto de estudio de especial interés desde el punto de vista de la ingeniería. Entre dichas características destacan la generación procedural de niveles, la ausencia de persistencia del progreso entre partidas (lo que se conoce como *permadeath* o muerte permanente), la progresión limitada a cada ejecución del juego y una elevada rejugabilidad derivada de la variabilidad del contenido generado. Estas propiedades obligan a diseñar sistemas software capaces de producir escenarios jugables, coherentes y equilibrados de forma automática, sin intervención manual directa por parte de un diseñador de niveles.

En este contexto se plantea el siguiente problema técnico: cómo diseñar e implementar una arquitectura de software modular que permita la generación automática de niveles funcionables, la gestión concurrente de múltiples subsistemas (como la inteligencia artificial de los enemigos, el motor de físicas, la interfaz de usuario (UI) o el control de estados del juego) y la parametrización del contenido de manera escalable y mantenible. Este problema resulta representativo de los retos que afronta la ingeniería del software aplicada al desarrollo de sistemas interactivos en tiempo real.

En el marco de este trabajo, la inteligencia artificial de los enemigos se implementa como un sistema propio basado en máquinas de estados finitos (FSM) y eventos en C#, sin depender de un módulo de IA predefinido del motor Unity.

1.2 MOTIVACIÓN Y JUSTIFICACIÓN

La elección de un videojuego del subgénero *roguelike* como objeto del presente Trabajo de Fin de Grado responde a una doble motivación: académica y técnica.

Desde el punto de vista académico, el desarrollo de un videojuego constituye un dominio de aplicación que permite integrar de forma práctica las competencias adquiridas a lo largo del Grado en Ingeniería Informática. En particular, el proyecto ofrece un marco adecuado para aplicar los contenidos estudiados en asignaturas clave de la titulación:

- **Ingeniería del Software (INSO).** Los patrones de diseño de software (*Singleton*, *Observer*, *Strategy* y *Template Method*), la captura de requisitos mediante historias de usuario, la arquitectura modular basada en la separación entre datos configurables y lógica de ejecución, y la gestión del ciclo de vida del software mediante una metodología iterativa e incremental constituyen contenidos propios de esta asignatura que se aplican de forma directa en el proyecto.
- **Programación Orientada a Objetos.** La jerarquía de clases del sistema de enemigos (Enemy -> SlimeGreen / SlimeRed / SlimeBlue) y del sistema de armas (Weapon -> MeleeWeapon / RangedWeapon) aplica los principios de herencia, polimorfismo y encapsulamiento. El patrón *Template Method*, que define comportamiento base en la clase padre y delega la especialización en las subclases, constituye un ejemplo representativo de la aplicación de estos conceptos.
- **Algoritmia y Estructuras de Datos.** La generación procedural de salas, el algoritmo de expansión por *clusters* (agrupaciones contiguas de tiles) para la distribución de tiles especiales, la generación ponderada de enemigos mediante presupuesto de coste y el control del flujo del juego mediante máquinas de estados constituyen problemas algorítmicos no triviales que permiten demostrar competencias en esta área. En este contexto, un tile es una unidad gráfica elemental del escenario, normalmente cuadrada, que representa partes del entorno como el suelo o los muros. Su agrupación en *tilemaps* permite construir salas mediante una rejilla de celdas individuales y asignar efectos concretos a determinados elementos del nivel.
- **Diseño de Sistemas Interactivos.** La implementación de la interfaz de usuario (sistema de corazones, menú de pausa, pantallas de fin de partida) y la atención a la experiencia de juego mediante retroalimentación visual (parpadeo de invulnerabilidad, parpadeo rojo de muerte, efectos de impacto) se vinculan con los contenidos de diseño de interacción y usabilidad.

Esta vinculación con la Ingeniería del Software resulta especialmente significativa. El desarrollo de un videojuego *roguelike* no se limita a la escritura de código: requiere definir una

1.3. OBJETIVOS

arquitectura de software modular, aplicar patrones de diseño reconocidos, gestionar la separación entre datos configurables y lógica de ejecución mediante mecanismos como los *ScriptableObjects* (activos de datos serializables de Unity que permiten configurar parámetros desde el inspector del motor sin modificar el código fuente), y seguir una metodología de desarrollo que permita validar el producto de forma progresiva. Estos principios coinciden con los que articulan la disciplina de la Ingeniería del Software, lo que convierte el proyecto en un ejercicio representativo de las competencias que esta asignatura desarrolla.

Desde el punto de vista técnico, el subgénero *roguelike* presenta una complejidad que lo convierte en un objeto de estudio adecuado para un Trabajo de Fin de Grado. La generación procedural de contenido obliga a diseñar algoritmos capaces de producir entornos jugables, coherentes y equilibrados sin intervención manual. La coordinación de múltiples subsistemas en tiempo real (combate, inteligencia artificial, generación de niveles, interfaz de usuario) exige una arquitectura con separación de responsabilidades y bajo acoplamiento entre componentes. Asimismo, la naturaleza iterativa del desarrollo de videojuegos, donde las mecánicas se ajustan mediante ciclos de prueba y refinamiento, se alinea con las metodologías de desarrollo incremental estudiadas durante la formación académica.

La relevancia cultural e industrial del subgénero refuerza la pertinencia de esta elección. Títulos como *Hades* (Supergiant Games, 2020), *Dead Cells* (Motion Twin, 2018), *The Binding of Isaac* (Edmund McMillen, 2011) o *Enter the Gungeon* (Dodge Roll, 2016) se han consolidado como referencias contemporáneas del subgénero. Por ejemplo, *Hades* ha recibido reconocimiento en premios de la industria; en *The Game Awards 2020* obtuvo el galardón a Mejor Juego Independiente [2]. El análisis de estos sistemas permite abordar conceptos avanzados de ingeniería del software en un contexto práctico y representativo.

1.3 OBJETIVOS

1.3.1 Objetivo general

El objetivo general del proyecto es diseñar, implementar y documentar un videojuego *roguelike* funcional que permita evidenciar la aplicación de principios de ingeniería del software en el ámbito del desarrollo de videojuegos, con especial énfasis en la arquitectura modular y la generación procedural de contenido.

1.3.2 Objetivos específicos

De este objetivo general se derivan los siguientes objetivos específicos (OE):

1. **OE-1.** Diseñar e implementar una arquitectura modular basada en patrones de diseño (*Singleton*, *Observer*, *Strategy*, *Template Method*) que facilite la separación de responsabilidades entre subsistemas y promueva la mantenibilidad y extensibilidad del código.
2. **OE-2.** Desarrollar un algoritmo de generación procedural de salas que garantice la jugabilidad y la coherencia espacial del entorno, incluyendo la distribución de tiles especiales mediante clusters, la colocación de muros con colisionadores y el posicionamiento de escaleras de transición.

3. **OE-3.** Implementar tres comportamientos diferenciados de inteligencia artificial para los enemigos (persecución, ataque a distancia y aproximación por saltos) y un sistema de enemigos especiales con atributos escalados que actúen como jefes periódicos.
4. **OE-4.** Crear un sistema de armas con mecánicas de combate cuerpo a cuerpo y a distancia, incluyendo detección de colisiones, proyectiles físicos y un sistema de órbita del arma alrededor del jugador orientado hacia la posición del cursor.
5. **OE-5.** Implementar un sistema de tiles con efectos que modifiquen la jugabilidad, incluyendo su configuración mediante *ScriptableObjects* y su distribución controlada en salas generadas proceduralmente.
6. **OE-6.** Implementar un sistema de persistencia de puntuaciones mediante *PlayerPrefs* con visualización de un ranking en el menú principal, registrando estadísticas completas de cada partida.
7. **OE-7.** Documentar el proceso de desarrollo, las decisiones técnicas adoptadas y las limitaciones encontradas, exponiendo el trabajo realizado de forma rigurosa y reproducible.

1.4 DESCRIPCIÓN DEL PROYECTO

El videojuego desarrollado, denominado *Ascension*, es un *roguelike* bidimensional con vista cenital (*top-down*, perspectiva en la que la cámara se sitúa sobre el escenario mirando hacia abajo), en el que el jugador avanza a través de una secuencia de salas generadas de forma procedural, enfrentándose a enemigos hasta alcanzar la condición de victoria establecida. Dicha condición consiste en la derrota de tres enemigos especiales con atributos incrementados (denominados jefes) que aparecen de forma periódica cada cinco salas completadas.

El sistema implementado incluye un flujo de juego completo que abarca las siguientes funcionalidades:

- **Menú principal** con acceso a la partida, tabla de puntuaciones y opción de salida.
- **Selección de clase** entre tres personajes con estadísticas diferenciadas (vida máxima, velocidad de movimiento y arma inicial), lo que permite al jugador adaptar la experiencia a distintos estilos de juego.
- **Gestión de estados globales** mediante un *GameManager* (componente gestor central del juego) que coordina las transiciones entre menú, selección de clase, juego activo, pausa, victoria y derrota.
- **Generación procedural de salas** mediante un *RoomGenerator* que produce la estructura espacial de cada nivel basándose en *tilemaps* con muros perimetrales, tiles de suelo con variantes visuales, tiles especiales con efectos (hielo, lodo, curación y potenciación de daño) y escaleras de transición entre salas.



- **Sistema de combate** que contempla dos tipos de armas: cuerpo a cuerpo, cuyo ataque realiza un barrido angular con detección de colisión, y a distancia, que genera proyectiles físicos dirigidos hacia la posición del cursor. Ambos tipos de arma se configuran mediante *ScriptableObjects* para su parametrización independiente del código.
- **Inteligencia artificial de enemigos** implementada mediante máquinas de estados finitos (FSM) y eventos, con tres comportamientos diferenciados: persecución directa, ataque a distancia con mantenimiento de distancia óptima y aproximación por saltos (*dash*, desplazamientos cortos y rápidos del enemigo hacia el jugador). Cada tipo de enemigo se configura mediante datos almacenados en *ScriptableObjects*, lo que permite parametrizar atributos como velocidad, daño, vida y radio de detección.
- **Sistema de gestión de enemigos** que utiliza un algoritmo de generación ponderada por coste, posicionamiento seguro respecto al jugador, limpieza automática de salas y control de interacción con las escaleras en función del estado del combate.
- **Interfaz de usuario completa** con visualización dinámica de la salud del jugador mediante corazones, menú de pausa funcional y pantallas de victoria y derrota.
- **Sistema de persistencia de puntuaciones** mediante *PlayerPrefs* (almacenamiento local clave-valor proporcionado por Unity) y serialización JSON (*JavaScript Object Notation*, conversión de estructuras de datos a texto estructurado), con un historial de hasta treinta partidas que registra puntuación, clase utilizada, salas completadas, enemigos eliminados, resultado y fecha.

El proyecto no incluye un sistema integral de audio ni un controlador genérico de objetos recolectables, al considerarse estos aspectos de menor prioridad para los objetivos académicos del trabajo. Asimismo, el sistema de jefes con fases múltiples y el sistema de generación de armas con rarezas constituyen funcionalidades preparadas a nivel de arquitectura para su extensión futura, pero no se encuentran completamente integradas en el flujo de juego durante la versión actual.

1.5 METODOLOGÍA

1.5.1 Tipo de investigación

El trabajo se enmarca dentro de una investigación de tipo aplicado, centrada en el diseño e implementación de un producto software funcional. El objetivo principal es demostrar la aplicación práctica de conceptos propios de la ingeniería del software en el contexto del desarrollo de videojuegos, concretamente en la creación de un videojuego bidimensional del subgénero *roguelike*.

1.5.2 Metodología de desarrollo de software

Se ha adoptado una metodología de desarrollo **iterativa e incremental**, basada en la implementación progresiva de funcionalidades completas y su validación iterativa. Esta decisión se justifica por dos razones fundamentales:

1. **Naturaleza exploratoria del proyecto.** El desarrollo de un videojuego requiere experimentar con mecánicas de juego, ajustar parámetros de balance y validar la jugabilidad de forma temprana. Una metodología secuencial (cascada) no permitiría esta flexibilidad.
2. **Necesidad de adaptación continua.** Durante el desarrollo se identificaron limitaciones técnicas y oportunidades de mejora que exigieron redefinir determinados aspectos del diseño. El enfoque iterativo permitió incorporar estos cambios sin comprometer la estabilidad del sistema.

El proceso de trabajo se estructuró en **iteraciones de corta duración**, cada una de las cuales generó un incremento funcional del sistema. En cada iteración se llevaron a cabo las siguientes actividades:

1. **Diseño** de un subsistema concreto, definiendo su responsabilidad, sus interfaces y sus dependencias con otros módulos.
2. **Implementación** de la funcionalidad asociada, siguiendo los principios de separación de responsabilidades y bajo acoplamiento.
3. **Validación** mediante pruebas funcionales manuales en el editor de Unity, comprobando el funcionamiento del subsistema de forma aislada y su integración con los subsistemas existentes.
4. **Corrección** de errores detectados durante la validación.
5. **Ajuste del alcance** de iteraciones posteriores en función de los resultados obtenidos y de las prioridades identificadas.

No se ha aplicado una metodología ágil formal como *Scrum* o *Kanban*, dado que el proyecto se ha desarrollado de manera individual. Sin embargo, el enfoque iterativo adoptado comparte principios fundamentales con las metodologías ágiles: entrega frecuente de incrementos funcionales, adaptación al cambio y validación iterativa del producto.

La elección del diseño iterativo se justifica por su adecuación al desarrollo de videojuegos, un ámbito en el que la experimentación, el ajuste de mecánicas y la validación temprana de funcionalidades son factores relevantes para favorecer la viabilidad del producto final y reducir el riesgo técnico. En la práctica, este enfoque permitió, por ejemplo, detectar la necesidad de implementar un sistema de gracia temporal (*grace period*, un breve intervalo tras la aparición en el que el enemigo no puede dañar al jugador), evitando daño instantáneo antes de que el jugador pudiera reaccionar. Este tipo de ajustes, derivados de la validación funcional durante el desarrollo, habría sido difícil de anticipar en una fase de diseño inicial.

1.5.3 Proceso de implementación y validación

La implementación del sistema se llevó a cabo siguiendo un proceso de definición de subsistemas, codificación, pruebas funcionales y refinamiento. Cada componente del videojuego se desarrolló de forma modular e independiente y posteriormente se integró en el sistema global, comprobando su funcionamiento conjunto con los subsistemas ya existentes.

Las iteraciones de desarrollo se organizaron en torno a los siguientes subsistemas, implementados de forma secuencial con solapamientos parciales entre fases:

1. Gestión central del juego (*GameManager*, máquina de estados, transiciones de escena).
2. Sistema de jugador (movimiento, esquiva, sistema de vida).
3. Generación procedural de salas (*RoomGenerator*, *tilemaps*, colisionadores).
4. Sistema de combate y armas (cuerpo a cuerpo, a distancia, proyectiles).
5. Inteligencia artificial de enemigos (tres comportamientos diferenciados).
6. Sistema de gestión de salas (flujo de progresión, enemigos especiales).
7. Sistema de tiles con efectos (hielo, lodo, curación, potenciación).
8. Interfaz de usuario (corazones, puntuación, menú de pausa, pantallas finales).
9. Sistema de persistencia (puntuaciones, historial de partidas).

El proceso de validación se basó en pruebas funcionales manuales ejecutadas directamente en el editor de Unity. En cada iteración se comprobaron aspectos clave del sistema, entre los que se incluyen:

- La generación procedural de salas con muros de contención íntegros y área jugable accesible
- El comportamiento de la inteligencia artificial de enemigos con distintos patrones de actuación
- La aplicación correcta de efectos de tiles sobre el jugador y los enemigos
- El flujo de avance entre salas y la aparición periódica de enemigos especiales
- La activación de la condición de victoria tras la superación de los tres encuentros con jefes
- La coherencia de la interfaz de usuario con el estado real del juego
- La persistencia de puntuaciones entre sesiones de juego

No se realizó un proceso de pruebas de usuario con participantes externos. La validación durante el desarrollo se centró en comprobar la estabilidad funcional del sistema y detectar errores críticos durante las sesiones de juego realizadas.

1.5.4 Ejemplos representativos de iteraciones y sus resultados

Para ilustrar cómo la metodología iterativa e incremental influyó en las decisiones técnicas del proyecto, a continuación se describen tres situaciones concretas en las que la validación funcional de un incremento reveló problemas que motivaron cambios de diseño:

1. **Descubrimiento del sistema de gracia temporal.** Durante la iteración correspondiente al sistema de enemigos, las partidas de validación revelaron que los enemigos infligían daño al jugador en el mismo fotograma en que aparecían, antes de que el jugador pudiera reaccionar. Este problema no era visible en el diseño inicial (que no contemplaba un retardo de activación) pero resultaba evidente al jugar. Como resultado, se diseñó e implementó un sistema de periodos de gracia (`contactDamageGraceSeconds` y `aiGraceSeconds` de 0,75 segundos), que se integró en la clase base `Enemy` para que todos los tipos de enemigo lo heredasen automáticamente.
2. **Transición de generación de salas como escenas separadas a regeneración en la misma escena.** El diseño inicial contemplaba cargar una nueva escena de Unity para cada sala, lo que resultaba en tiempos de carga perceptibles (entre 0,5 y 1,5 segundos por transición, dependiendo de la complejidad de la escena) que rompían el ritmo de juego. La validación durante la iteración 3 evidenció este problema, y se rediseñó el sistema para regenerar el contenido del *tilemap* dentro de la misma escena (`Regenerate FloorKeepWalls`), eliminando los tiempos de carga y manteniendo los colisionadores de muros intactos. Este cambio tuvo un impacto arquitectónico significativo: el `Room FlowController` dejó de gestionar cargas de escena y pasó a coordinar directamente el `RoomGenerator` y el `EnemyManager` dentro de la escena activa.
3. **Limitador de velocidad de los enemigos.** Durante las pruebas del slime verde (francotirador), se detectó que en su estado de huida podía moverse más rápido que el jugador, lo que hacía imposible alcanzarlo. Este problema no se había previsto en el diseño porque la velocidad del enemigo y del jugador se configuraban de forma independiente en sus respectivos *ScriptableObjects*. La solución fue implementar el método `CapToPlayerSpeed()`, que limita la velocidad efectiva de cualquier enemigo a un máximo igual a la velocidad actual del jugador, garantizando que la evasión nunca sea imposible.

Estos tres casos ilustran el valor del enfoque iterativo: ninguno de los problemas habría sido detectado en una fase de diseño inicial, ya que solo se manifestaron durante la ejecución del juego con sistemas integrados.

1.6 PLANIFICACIÓN TEMPORAL

1.6.1 Estimación de la dedicación

La dedicación total estimada del Trabajo de Fin de Grado asciende a 300 horas, distribuidas de forma desigual entre las distintas fases del proyecto según la naturaleza de las tareas.

1.6.2 Fases del proyecto

El proyecto se ha estructurado en las siguientes fases, con una estimación del esfuerzo dedicado a cada una:

Fase 1. Investigación y definición de alcance (30 horas)

Durante esta fase se realizó una exploración de posibles enfoques para el TFG, evaluando distintas ideas de proyecto y tecnologías. Se analizaron los requisitos generales del trabajo académico, se estudiaron las características del subgénero *roguelike* y se llevó a cabo una primera toma de contacto con el motor Unity y el lenguaje C#. Como resultado de esta fase se decidió orientar el TFG hacia el desarrollo de un videojuego del subgénero *roguelike*, al considerarse un dominio adecuado para aplicar conocimientos de ingeniería del software. Se definieron los objetivos generales y específicos del proyecto.

Fase 2. Diseño de arquitectura y prototipos iniciales (35 horas)

En esta fase se diseñó la arquitectura inicial del sistema, definiendo los principales subsistemas del juego (gestión de estados, flujo de salas, generación procedural y combate) y estableciendo las bases del proyecto en Unity. Se tomaron decisiones técnicas clave relativas al uso de *ScriptableObjects* para la parametrización de datos, la aplicación de patrones de diseño (*Singleton*, *Observer*) y la separación entre lógica de juego y datos configurables. Se implementaron prototipos funcionales del movimiento del jugador y de la generación básica de salas.

Fase 3. Implementación del núcleo del juego (90 horas)

Esta fase constituyó el núcleo del trabajo práctico. Se implementaron de forma incremental los sistemas fundamentales del videojuego:

- Sistema de generación procedural de salas con tiles, muros y colisionadores
- Gestión del flujo de juego mediante *RoomFlowController*
- Sistema de combate con armas cuerpo a cuerpo y a distancia
- Inteligencia artificial de los tres tipos de enemigos
- Sistema de enemigos especiales con atributos escalados
- Condición de victoria y derrota

La implementación se realizó de manera iterativa, validando la funcionalidad del juego en cada incremento y ajustando decisiones de diseño cuando fue necesario.

Fase 4. Sistema de tiles, efectos e interfaz de usuario (40 horas)

Se implementó el sistema de tiles especiales con efectos diferenciados (hielo, lodo, curación, potenciación de daño), incluyendo el algoritmo de distribución por clusters y la persistencia temporal de efectos. De forma paralela, se desarrolló la interfaz de usuario completa: sistema dinámico de corazones, indicador de puntuación, menú de pausa, pantallas de victoria y derrota, y tabla de puntuaciones con historial persistente.

Fase 5. Ajuste de configuración y validación del contenido (30 horas)

Se llevaron a cabo tareas de verificación y ajuste de configuraciones globales del proyecto (parámetros de importación de sprites, físicas, colisionadores y consistencia de activos), con el objetivo de reducir errores de configuración y facilitar la iteración sobre los contenidos del juego.

Fase 6. Refinamiento, pruebas y depuración (20 horas)

Se realizó un proceso de pruebas funcionales del sistema completo, comprobando la estabilidad del flujo de juego y corrigiendo errores detectados. Se llevaron a cabo ajustes de balance, mejoras en la interfaz de usuario y optimizaciones de rendimiento.

Fase 7. Creación de assets gráficos (15 horas)

Se diseñaron y crearon los sprites del juego, incluyendo los personajes jugables, los tres tipos de enemigos, las armas, los proyectiles, los tiles de suelo y muros, y los elementos de interfaz de usuario. Se utilizó la herramienta LibreSprite para la creación de *pixel art* (técnica de creación de gráficos digitales en la que las imágenes se editan a nivel de píxel individual) a una resolución de 16 píxeles por unidad.

Fase 8. Documentación y memoria (40 horas)

Se redactó la memoria académica del Trabajo de Fin de Grado, documentando cada fase del proyecto, las decisiones técnicas adoptadas, la arquitectura del sistema, los módulos implementados, los patrones de diseño aplicados, los resultados obtenidos y las conclusiones alcanzadas. Se elaboraron los diagramas de clases, de secuencia y de Gantt incluidos en el documento.

1.6.3 Representación gráfica de la planificación

La Figura 1.1 muestra la planificación temporal del proyecto, con la distribución de las ocho fases, su dedicación en horas y la superposición parcial entre algunas de ellas.



Figura 1.1: Diagrama de Gantt con la distribución temporal y la dedicación en horas de las ocho fases del proyecto (elaboración propia).

1.7 ESTRUCTURA DE LA MEMORIA

Esta memoria se organiza en seis capítulos. Tras la introducción, se analiza el estado del arte, referentes y tecnologías empleadas. Posteriormente, se detalla el diseño del sistema (requisitos y arquitectura) y su implementación técnica. Finalmente, se exponen las conclusiones y líneas de trabajo futuro.

2 ESTADO DEL ARTE

Este capítulo contextualiza el desarrollo de *Ascension* mediante cuatro bloques: definición y relevancia del género *roguelike*, análisis de referentes del mercado, justificación de las herramientas tecnológicas 2D seleccionadas y exposición de los patrones de diseño implementados.

2.1 EL SUBGÉNERO *roguelike*: DEFINICIÓN Y CARACTERÍSTICAS

2.1.1 Origen histórico y evolución

El término *roguelike* tiene su origen en el videojuego *Rogue*, desarrollado originalmente para sistemas Unix por Michael C. Toy, Glenn Wichman y Ken Arnold [3]. Este juego introducía dos mecánicas que resultarían fundacionales para todo un subgénero: la generación procedural de niveles (de modo que cada partida presentaba una disposición espacial diferente) y la muerte permanente (*permadeath*), que obligaba al jugador a reiniciar la partida desde el principio al perder todos los puntos de vida.

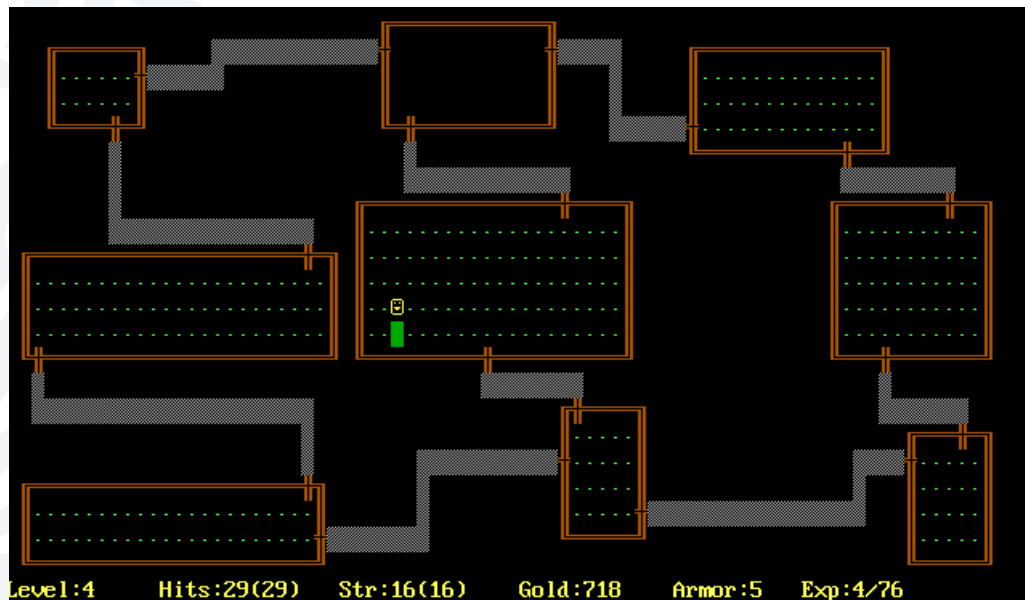


Figura 2.1: Captura de pantalla del videojuego original *Rogue* (1980), mostrando la interfaz basada en caracteres ASCII y la disposición procedural de las salas. Fuente: captura histórica del juego.

A lo largo de las décadas siguientes, numerosos títulos adoptaron y expandieron estas mecánicas. Juegos como *NetHack* (1987), *Angband* (1990) y *Dungeon Crawl Stone Soup* (2006) mantuvieron la esencia del *roguelike* clásico, caracterizado por gráficos basados en caracteres ASCII, mecánicas por turnos y una dificultad elevada. Sin embargo, a partir de la década de 2010, el subgénero experimentó una transformación significativa con la aparición de los denominados *roguelites*, que incorporaban elementos del *roguelike* clásico (generación procedural y muerte permanente) en combinación con mecánicas de acción en tiempo real, progresión entre partidas y diseño visual contemporáneo.

2.1.2 Características técnicas fundamentales

La comunidad de desarrolladores y jugadores ha intentado establecer criterios formales para definir qué constituye un *roguelike*. La denominada *Interpretación de Berlín* (International Roguelike Development Conference, 2008) [4] identificó las siguientes características como definitorias del subgénero:

1. **Generación procedural de niveles.** La estructura espacial del juego se genera algorítmicamente en cada partida, de modo que no existen dos ejecuciones idénticas. Esta característica implica la necesidad de diseñar algoritmos capaces de producir entornos coherentes, equilibrados y accesibles sin intervención manual.
2. **Muerte permanente (*permadeath*).** Al perder todos los puntos de vida, la partida finaliza de forma definitiva y el jugador debe reiniciar desde el principio. No existe la posibilidad de guardar y cargar el progreso del juego, lo que confiere a cada decisión del jugador un peso significativo.
3. **Progresión limitada a cada ejecución (*run*, una partida completa desde el inicio hasta la muerte o victoria del jugador).** Las mejoras, armas y habilidades obtenidas durante una partida se pierden al morir. Cada ejecución comienza desde un estado inicial predefinido, aunque títulos más recientes (*roguelites*) introducen sistemas de progresión permanente entre partidas.
4. **Sistema basado en turnos.** En la definición clásica, el jugador y los enemigos actúan de forma alternada. Sin embargo, los *roguelikes* modernos y los *roguelites* frecuentemente adoptan mecánicas de acción en tiempo real, como ocurre en el caso del proyecto *Ascension*.
5. **Exploración de mazmorras (*dungeon crawling*).** El jugador explora una sucesión de niveles o salas, enfrentándose a enemigos y recogiendo recursos, con el objetivo de alcanzar una condición de victoria definida.
6. **Alta rejugabilidad.** La combinación de generación procedural y la variabilidad de los elementos de juego (enemigos, objetos, disposición espacial) contribuye a que cada partida ofrezca una experiencia diferente, incentivando al jugador a repetir la experiencia.
7. **Complejidad de sistemas interconectados.** Los *roguelikes* suelen presentar múltiples sistemas que interactúan entre sí de forma emergente, generando situaciones impredecibles que enriquecen la experiencia de juego.

2.1.3 Importancia del género en la industria del videojuego

El subgénero *roguelike* ha experimentado un crecimiento notable en la última década, consolidándose como uno de los géneros más populares en el segmento de los videojuegos independientes. La amplia presencia de títulos etiquetados como *roguelike* en catálogos de distribución digital refleja esta popularidad.

La relevancia del género se manifiesta en varios indicadores:

- **Éxito comercial.** Diversos títulos del subgénero han alcanzado un rendimiento comercial significativo, consolidándose como referencias dentro del mercado independiente.
- **Reconocimiento crítico.** *Hades* obtuvo reconocimiento en premios de la industria, incluyendo el premio al Mejor Juego Independiente en *The Game Awards* 2020 [2]. Estos reconocimientos evidencian que el subgénero ha trascendido el ámbito de los juegos independientes para competir con producciones de mayor presupuesto.
- **Influencia en el diseño de juegos.** Elementos propios del *roguelike* (generación procedural, *permadeath*, rejugabilidad) se han integrado en géneros distintos, dando lugar a híbridos como *Returnal* (Housemarque, 2021), un juego de acción en tercera persona con estructura *roguelike*, o *Slay the Spire* (Mega Crit, 2019), que combina la estructura *roguelike* con mecánicas de juegos de cartas [5].
- **Accesibilidad para desarrolladores independientes.** La generación procedural de contenido reduce la necesidad de diseño manual de niveles, lo que permite a equipos pequeños o desarrolladores individuales crear productos con una cantidad de contenido que, de otro modo, requeriría recursos significativamente mayores. Esta característica convierte al subgénero en un dominio adecuado para proyectos académicos como el presente Trabajo de Fin de Grado.

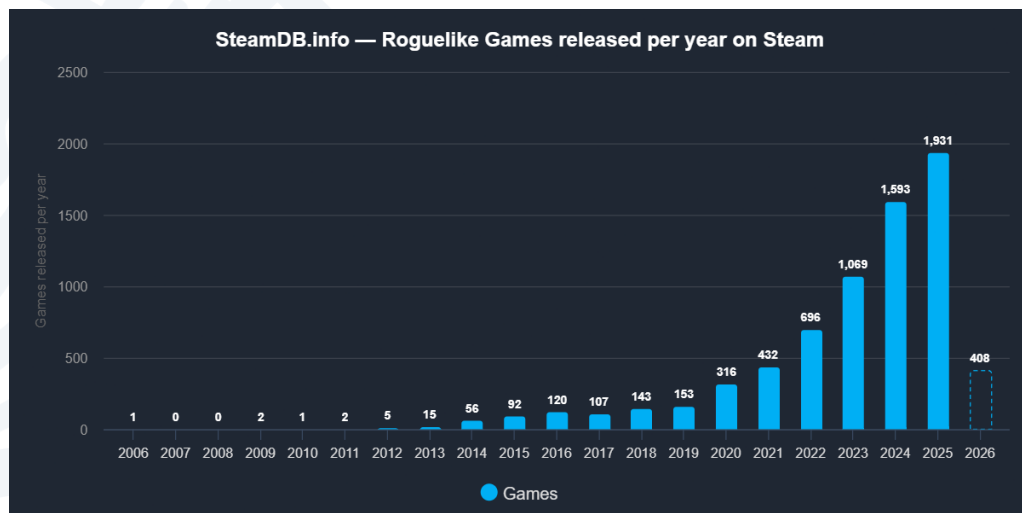


Figura 2.2: Gráfico que muestra la evolución del número de títulos *roguelike* publicados en Steam por año (2006–2026). Fuente: SteamDB

2.1.4 Distinción entre *roguelike* y *roguelite*

Resulta pertinente distinguir entre *roguelike* en sentido estricto y *roguelite*, dado que el proyecto *Ascension* se sitúa en la intersección de ambas categorías:

Característica	<i>Roguelike</i> clásico	<i>Roguelite</i>
Generación procedural	Sí	Sí
Muerte permanente	Total	Total o parcial
Mecánica de combate	Por turnos	Tiempo real
Progresión entre partidas	No	Frecuente
Representación gráfica	ASCII o 2D simple	2D/3D variada
Complejidad de sistemas	Muy alta	Moderada-alta

Tabla 2.1: Comparativa entre *roguelike* clásico y *roguelite*.

Ascension adopta las mecánicas fundamentales del *roguelike* (generación procedural, *permadeath*, exploración de mazmorras) con un sistema de combate en tiempo real propio del *roguelite*, sin incluir progresión permanente entre partidas (salvo la persistencia de puntuaciones) lo que lo acerca más a la definición clásica del subgénero.

2.1.5 Generación procedural de contenido

La Generación Procedural de Contenido (GPC) constituye un conjunto de técnicas orientadas a la creación automática de datos de juego mediante algoritmos, reduciendo o eliminando la necesidad de diseño manual de niveles, escenarios o elementos del entorno (Shaker, Togelius y Nelson, 2016) [6].

En el contexto de los *roguelikes*, la GPC resulta un componente central, ya que permite ofrecer variabilidad estructural y funcional en cada partida.

Entre los enfoques más utilizados para la generación procedural de niveles en videojuegos bidimensionales se encuentran:

- **Algoritmos basados en reglas.** Definen un conjunto de restricciones que deben cumplirse en el contenido generado (por ejemplo, que toda sala tenga al menos una salida accesible). El generador aplica estas reglas de forma iterativa hasta producir un resultado válido.
- **Autómatas celulares.** Utilizados frecuentemente para la generación de cuevas orgánicas. Cada celda de una rejilla se evalúa según las celdas vecinas, produciendo formaciones naturales mediante iteraciones sucesivas.
- **BSP (*Binary Space Partitioning*).** Divide recursivamente el espacio disponible en subespacios menores, creando en cada subdivisión una habitación. Las habitaciones se conectan mediante pasillos, lo que genera estructuras de mazmorra clásicas.
- **Sistemas deterministas con semillas pseudoaleatorias.** Utilizan un valor de semilla para inicializar el generador de números aleatorios, permitiendo reproducir exactamente la misma disposición espacial a partir de la misma semilla.

- **Sistemas basados en *Wave Function Collapse* (WFC).** Inspirados en la mecánica cuántica, definen restricciones de adyacencia entre tiles y resuelven iterativamente la disposición completa del nivel, produciendo resultados visualmente coherentes.
- **Gramáticas formales y L-systems.** Utilizan reglas de producción para generar estructuras complejas a partir de axiomas simples, aplicados habitualmente a la generación de vegetación y elementos orgánicos.

En el caso de *Ascension*, la generación procedural se basa en un enfoque híbrido: se utiliza una estructura rectangular fija de 26×12 tiles con bordes de muros perimetrales de un tile de grosor, sobre la cual se aplica un algoritmo de distribución aleatoria de tiles de suelo con variantes visuales y un algoritmo de expansión por *clusters* para la colocación de tiles especiales con efectos.

La elección de este enfoque frente a técnicas más complejas como BSP o WFC se justifica por tres razones concretas. En primer lugar, la estructura rectangular fija simplifica el cálculo de posiciones válidas para la aparición de enemigos, ya que cualquier coordenada dentro del perímetro de muros constituye una posición válida sin necesidad de verificaciones adicionales de accesibilidad. En segundo lugar, los bordes perimetrales de muros impiden que el jugador pueda abandonar el área de juego sin necesidad de lógica de contención adicional; el colisionador del *tilemap* de muros se genera automáticamente a partir de la posición de los tiles. En tercer lugar, el algoritmo de *clusters* permite controlar la distribución espacial de los tiles especiales (agrupándolos en manchas de entre cuatro y nueve tiles) de forma que el jugador encuentre zonas con efectos lo suficientemente grandes como para influir en las decisiones tácticas de combate, pero no tan extensas como para dominar la sala. Este enfoque combina la previsibilidad estructural con la variabilidad de contenido que caracteriza al subgénero.

Técnicas emergentes y trabajos recientes

Además de los enfoques clásicos enumerados anteriormente, en los últimos años han surgido técnicas que amplían las posibilidades de la generación procedural en videojuegos:

- **Generación basada en restricciones con *solvers* (Constraint Satisfaction Problems, CSP).** Formulan la generación de niveles como un problema de satisfacción de restricciones y emplean *solvers* (como SAT o programación por restricciones) para encontrar configuraciones válidas que cumplan todas las condiciones impuestas. Smith y Mateas (2011) aplicaron esta técnica a la generación de niveles de plataformas 2D [7].
- **Generación asistida por aprendizaje automático.** Se han explorado redes neuronales generativas (GANs (*Generative Adversarial Networks*) y autoencoders variacionales (VAEs) (*Variational AutoEncoders*) para aprender la distribución de niveles diseñados manualmente y generar nuevos niveles consistentes con ese estilo (Volz et al., 2018) [8].
- **Funciones de ruido para generación de terreno.** Las funciones de ruido Perlin y Simplex, originalmente propuestas para la generación de texturas (Perlin, 1985) [9], se

utilizan ampliamente en juegos como *Minecraft* y *Terraria* para generar terrenos con apariencia natural.

- **Wave Function Collapse (WFC) aplicado a tilemaps.** Gumin (2016) propuso el algoritmo WFC como un procedimiento genérico de generación a partir de ejemplos [10]. Karth y Smith (2017) analizaron formalmente el algoritmo y demostraron su equivalencia con ciertos modelos de propagación de restricciones [11].

La siguiente tabla sintetiza la comparación entre las técnicas de generación procedural evaluadas y el enfoque adoptado en *Ascension*:

Técnica	Fortaleza principal	Limitación principal	Adecuación para <i>Ascension</i>
BSP	Salas irregulares con pasillos	Requiere verificación de accesibilidad	Baja: complejidad innecesaria para sala única
Autómatas celulares	Formas orgánicas (cuevas)	Difícil control de proporciones	Baja: salas rectangulares requeridas
WFC	Coherencia visual automática	Coste variable, reglas de adyacencia complejas	Moderada: sobrecomplejo para 8 tipos de tile
CSP / solvers	Garantía de validez	Tiempo de cómputo elevado	Baja: no requiere verificación formal
GANs / VAEs	Captura de estilos implícitos	Requiere datos de entrenamiento, sin garantía	Baja: sin corpus de niveles previo
Ruido Perlin/Simplex	Terrenos naturales continuos	No discreto, sin control de salas	Baja: generación por tiles, no continua
Rejilla fija + clusters	Control total, posicionamiento simple	Geometría invariante	Alta: equilibrio complejidad/variabilidad

Tabla 2.2: Comparativa de técnicas de generación procedural y su adecuación al proyecto.

La tabla evidencia que, para un proyecto con sala rectangular fija, tres tipos de enemigos y un sistema de tiles con efectos como fuente principal de variabilidad, las técnicas más sofisticadas habrían incrementado la complejidad de implementación sin producir un beneficio proporcional en la experiencia de juego. La variabilidad táctica de *Ascension* reside en la composición de tiles con efectos y en la combinación de oleadas de enemigos, no en la forma geométrica de la sala.

2.1.6 Inteligencia artificial en videojuegos 2D

La Inteligencia Artificial (IA) en videojuegos bidimensionales se orienta al diseño de comportamientos creíbles y coherentes para los Personajes No Jugables (PNJ) (Millington y Funge, 2009) [12].

Las técnicas más empleadas para la implementación de IA en videojuegos incluyen:

- **Máquinas de estados finitos (FSM).** Definen un conjunto discreto de estados posibles para un PNJ y las condiciones que provocan la transición entre estados. Son fáciles de

implementar y de depurar, pero pueden resultar rígidas cuando el número de estados crece.

- **Árboles de comportamiento (*behavior trees*).** Estructuran la lógica de decisión en forma de árbol jerárquico, donde cada nodo representa una condición, una acción o una secuencia de acciones.
- **Sistemas de decisión basados en reglas.** Evalúan un conjunto de condiciones y ejecutan la acción asociada a la regla de mayor prioridad que se cumpla.
- **Navegación por mallas (*navmesh*) y *pathfinding*.** Algoritmos como A* o Dijkstra permiten calcular rutas óptimas entre dos puntos del nivel.
- **Sistemas de utilidad (*Utility AI*).** Evalúan numéricamente la utilidad de cada acción disponible en función del estado actual del mundo.
- **Goal-Oriented Action Planning (GOAP).** Los agentes definen objetivos y planifican secuencias de acciones para alcanzarlos. Orkin (2003) introdujo GOAP en *F.E.A.R.* [13].
- **Aprendizaje por refuerzo.** Los agentes aprenden comportamientos óptimos mediante la maximización de una función de recompensa. Unity ML-Agents (Juliani et al., 2018) [14] proporciona un *framework* (un entorno de desarrollo con herramientas y bibliotecas predefinidas) integrado para entrenar agentes mediante aprendizaje por refuerzo profundo (DRL, *Deep Reinforcement Learning*).

En *Ascension*, la inteligencia artificial de los enemigos se implementa mediante una combinación de máquinas de estados simples y sistemas de decisión basados en reglas, cuya justificación se detalla en la siguiente tabla comparativa:

Técnica	Complejidad de implementación	Expresividad	Depurabilidad	Adecuación para 3 tipos de enemigo
FSM	Baja	Moderada	Alta (estados discretos)	Alta
Árboles de comportamiento	Moderada	Alta	Moderada (nodos jerárquicos)	Moderada
Sistemas de utilidad	Moderada-alta	Alta	Baja (valores continuos)	Baja
GOAP	Alta	Muy alta	Baja (planes dinámicos)	Baja
Aprendizaje por refuerzo	Muy alta	Variable	Muy baja (caja negra)	Muy baja

Tabla 2.3: Comparativa de técnicas de IA para enemigos en videojuegos 2D.

2.1.7 Mecánicas de combate en *roguelikes*

El sistema de combate constituye una de las mecánicas centrales de los videojuegos *roguelike* y *roguelite*. Los enfoques más habituales incluyen:

- **Combate por turnos.** El jugador y los enemigos ejecutan acciones de forma alternada. Es el enfoque clásico del subgénero (*Rogue*, *NetHack*).
- **Combate en tiempo real con *hack and slash* (combate de acción basado en ataques cuerpo a cuerpo rápidos y encadenados).** El jugador controla el personaje en tiempo real, ejecutando ataques de forma continua. Ejemplos representativos incluyen *Hades* y *Dead Cells*.
- **Combate en tiempo real con componente de disparos (*top-down shooter*).** El jugador dispara proyectiles hacia los enemigos desde una perspectiva cenital. Títulos como *The Binding of Isaac* y *Enter the Gungeon* han popularizado este enfoque.
- **Combate híbrido.** Combina armas cuerpo a cuerpo y a distancia, permitiendo al jugador alternar entre estilos según la situación. *Ascension* adopta este enfoque, ofreciendo ambas modalidades de combate con mecánicas diferenciadas.

2.2 ANÁLISIS DE MERCADO: REFERENCIAS COMERCIALES DEL SUBGÉNERO

El diseño y desarrollo de *Ascension* se ha llevado a cabo tomando como referencia una selección de títulos comerciales representativos del subgénero *roguelike* y *roguelite*. El estudio de estos productos ha permitido identificar mecánicas de juego, decisiones de diseño y patrones de interacción que han influido en la definición de los sistemas del proyecto. A continuación se analizan los títulos más relevantes.

2.2.1 *The Binding of Isaac* (2011, Edmund McMillen)

The Binding of Isaac es un *roguelike* con perspectiva cenital (*top-down*) cuya mecánica de combate se basa en el disparo de proyectiles en las cuatro direcciones cardinales. El jugador explora una sucesión de salas generadas proceduralmente, enfrentándose a enemigos con patrones de movimiento y ataque variados.

Características relevantes para *Ascension*:

- **Estructura de salas.** Cada sala constituye un espacio cerrado con enemigos que deben ser derrotados antes de poder avanzar. En *The Binding of Isaac*, las puertas de la sala se cierran al entrar y se abren al completarla. En *Ascension*, la progresión se representa mediante escaleras que se bloquean durante el combate y se habilitan al completar la sala.
- **Vista cenital.** La perspectiva *top-down* permite al jugador una visión completa de la sala, facilitando la planificación táctica del movimiento y el combate. *Ascension* utiliza esta misma perspectiva.
- **Variedad de enemigos.** Cada tipo de enemigo presenta un comportamiento diferenciado (persecución, disparo, movimiento errático), lo que obliga al jugador a adaptar su estrategia continuamente.

- **Sistema de jefes periódicos.** Al final de cada piso, el jugador se enfrenta a un enemigo jefe con estadísticas superiores y mecánicas de ataque propias. *Ascension* implementa un sistema análogo con jefes cada cinco salas.



Figura 2.3: Captura de pantalla de *The Binding of Isaac: Rebirth* en la que se observa la perspectiva cenital, la estructura de sala cerrada con puertas y la interfaz de vida mediante corazones en la esquina superior izquierda. Fuente: captura del juego en Steam.

2.2.2 Enter the Gungeon (2016, Dodge Roll)

Enter the Gungeon es un *roguelite* cenital centrado en el combate con armas de fuego. Destaca por su sistema de esquiva (*dodge roll*) y por la amplia variedad de armas disponibles, cada una con mecánicas diferenciadas.

Características relevantes para *Ascension*:

- **Mecánica de esquiva (*roll*).** El jugador puede ejecutar un movimiento rápido con invulnerabilidad temporal para evitar el daño de los proyectiles enemigos. *Ascension* implementa una mecánica de esquiva equivalente, activada mediante la tecla Espacio, con un periodo de invulnerabilidad y un tiempo de espera entre usos.
- **Combinación de combate a distancia y cuerpo a cuerpo.** *Ascension* ofrece ambas modalidades de combate como mecánicas diferenciadas asociadas a tipos de arma distintos.
- **Generación procedural con estructura por pisos.** El juego genera cada piso de la mazmorra de forma procedural. *Ascension* adopta una estructura en la que se genera proceduralmente el contenido de la sala.



Figura 2.4: Captura de pantalla de *Enter the Gungeon* en la que se muestra la mecánica de esquiva (*dodge roll*) durante el combate y la variedad de proyectiles en pantalla. Fuente: captura del juego en Steam.

2.2.3 Hades (2020, Supergiant Games)

Hades es un *roguelite* de acción con perspectiva isométrica que combina combate rápido con una narrativa progresiva. Ha recibido reconocimiento en premios de la industria; por ejemplo, en *The Game Awards 2020* obtuvo el galardón a Mejor Juego Independiente [2].

Características relevantes para *Ascension*:

- **Variedad de armas con mecánicas únicas.** Cada arma presenta un estilo de combate diferenciado (espada, arco, escudo, lanza). *Ascension* implementa armas cuerpo a cuerpo y a distancia con mecánicas de ataque diferenciadas.
- **Sistema de clases de personaje.** Aunque en *Hades* no existen clases como tales, la selección de arma al inicio de cada ejecución determina el estilo de juego. *Ascension* formaliza este concepto mediante un sistema de selección de clase con estadísticas diferenciadas.
- **Enemigos con comportamientos variados.** Los enemigos presentan patrones de ataque diferenciados y coordinados. *Ascension* implementa esta variedad mediante tres tipos de enemigos con comportamientos específicos.
- **Retroalimentación visual del combate.** *Hades* destaca por la retroalimentación visual que proporciona al jugador. *Ascension* incorpora mecanismos análogos como el parpadeo en blanco del jugador al recibir daño y un parpadeo rojo rápido tanto en enemigos como en el jugador justo antes de morir.



Figura 2.5: Captura de pantalla de *Hades* en la que se aprecia el sistema de combate en tiempo real con arma cuerpo a cuerpo, la variedad de enemigos simultáneos y la interfaz de estado del personaje. Fuente: captura del juego en Steam.

2.2.4 Dead Cells (2018, Motion Twin)

Dead Cells es un roguelite de acción lateral (*side-scroller*) que combina exploración, combate fluido y un sistema de progresión permanente entre partidas.

Características relevantes para *Ascension*:

- **Combate ágil y fluido.** *Dead Cells* se caracteriza por un combate rápido y preciso. En *Ascension* se han implementado controles fluidos y tiempos de espera (*cooldown*) reducidos con el objetivo de aproximarse a este enfoque.
- **Armas con comportamientos diferenciados.** El juego ofrece una amplia variedad de armas cuerpo a cuerpo y a distancia. *Ascension* implementa esta distinción mediante las clases *MeleeWeapon* y *RangedWeapon*, aplicando el patrón *Strategy*.
- **Escalonamiento de dificultad.** La dificultad aumenta de forma progresiva. En *Ascension*, la dificultad se incrementa mediante el aumento del presupuesto de coste de enemigos por sala y la aparición periódica de enemigos especiales con atributos escalados.

En este contexto, el término *cooldown* se refiere al **tiempo mínimo de espera entre dos ejecuciones de una misma acción** (por ejemplo, entre dos ataques consecutivos), y se utiliza como mecanismo de equilibrado para evitar acciones excesivamente frecuentes.



Figura 2.6: Captura de pantalla de *Dead Cells* en la que se observa el combate lateral con armas cuerpo a cuerpo. Fuente: captura del juego en Steam.

2.2.5 Tabla comparativa de referencias comerciales

La siguiente tabla compara las características de diseño y tecnología de los referentes analizados frente a *Ascension*, justificando las decisiones adoptadas en cuanto a perspectiva, combate y mecánicas de movimiento.

Característica	The Binding of Isaac	Enter the Gun-geon	Hades	Dead Cells	Ascension
Perspectiva	Cenital	Cenital	Isométrica	Lateral	Cenital
Generación procedural	Salas conectadas	Pisos con salas	Salas conectadas	Niveles laterales	Salas secuenciales
Combate	Disparo 4 direcciones	Disparo 360° + esquiva	Melee + distancia	Melee + distancia	Melee + distancia
Esquiva	No	<i>Dodge roll</i>	<i>Dash</i>	<i>Roll</i>	<i>Roll</i> con invulnerabilidad
Selección de personaje	Múltiples personajes	Múltiples personajes	Arma inicial	No	3 clases con stats
<i>Permadeath</i>	Sí	Sí	Sí (con progresión)	Sí (con progresión)	Sí
Sistema de jefes	Final de piso	Final de piso	Final de zona	Por bioma	Cada 5 salas
Tiles con efectos	No	No	No	No	Sí (hielo, lodo, curación, daño)
Motor	Flash / C++ custom	Unity	Supergiant Engine	Custom	Unity

Tabla 2.4: Comparativa entre referencias comerciales y el proyecto *Ascension*.

2.2.6 Elementos diferenciadores de *Ascension*

A partir del análisis comparativo anterior, es posible identificar los elementos que diferencian a *Ascension* respecto a las referencias comerciales estudiadas:

1. **Sistema de tiles con efectos.** Ninguno de los títulos analizados incorpora un sistema de tiles especiales que modifiquen las condiciones de combate en tiempo real. Se incorporó este sistema en *Ascension* para resolver un problema de diseño concreto: en un *roguelike* con tres tipos de enemigos y un repertorio limitado de armas, la variabilidad táctica de cada sala depende exclusivamente de la composición de la oleada de enemigos, lo que puede generar repetitividad. Los tiles con efectos (hielo (incremento de velocidad), lodo (reducción de velocidad), curación y potenciación de daño) introducen una segunda dimensión de variabilidad: el entorno físico de la sala condiciona las decisiones tácticas del jugador. La alternativa descartada era añadir más tipos de enemigo, pero esto habría incrementado significativamente el volumen de arte y animación necesario.
2. **Arquitectura orientada a la extensibilidad.** Se adoptó la separación entre datos configurables (*ScriptableObjects*) y lógica de juego como principio arquitectónico deliberado. Esta decisión responde a una necesidad práctica del desarrollo: los parámetros de configuración se modificaron en numerosas ocasiones. Con *ScriptableObjects*, cada ajuste se realiza editando un campo numérico en el inspector, con efecto inmediato y sin recompilación. La implicación arquitectónica es que el sistema cumple el principio de apertura-cierre (*Open-Closed Principle*) de los principios SOLID.



Figura 2.7: Captura de pantalla de *Ascension* durante una partida, mostrando la perspectiva cenital, la sala generada proceduralmente con tiles especiales visibles (hielo, lodo), enemigos activos y la interfaz de corazones. *Elaboración propia.*



Figura 2.8: Captura de pantalla de *Ascension* mostrando la pantalla de selección de clase. *Elaboración propia.*

2.3 ANÁLISIS DE HERRAMIENTAS Y TECNOLOGÍAS

2.3.1 Motores de desarrollo de videojuegos

La selección del motor de desarrollo constituye una decisión técnica de gran relevancia en cualquier proyecto de videojuegos. A continuación se analizan los motores más relevantes para el desarrollo de videojuegos bidimensionales.

Unity

Unity es un motor de desarrollo multiplataforma creado por Unity Technologies en 2005. Constituye una de las herramientas más utilizadas en la industria; Unity indica que más del 70 % de los 1.000 juegos móviles más populares se desarrollaron con Unity (dato referido a Q4 2022 con fuente Apptopia) [15]. Sus características principales incluyen:

- **Arquitectura basada en componentes (*Entity-Component System*).** Las entidades del juego se definen como agregaciones de componentes reutilizables.
- **Sistema de *Tilemaps* integrado.** Proporciona herramientas nativas para la creación y gestión de niveles basados en tiles, incluyendo colisionadores automáticos.
- **Lenguaje de programación C#.** Ofrece tipado fuerte, soporte completo para programación orientada a objetos y acceso a la biblioteca estándar de .NET.
- **Sistema de *ScriptableObjects*.** Permite almacenar datos configurables como activos del proyecto.
- **Soporte multiplataforma.** Permite compilar el proyecto para más de veinte plataformas.

- **Comunidad y recursos.** Cuenta con documentación oficial detallada, foros activos y un amplio ecosistema de *assets*.

Godot Engine

Godot es un motor de código abierto creado por Juan Linietsky y Ariel Manzur, publicado bajo licencia MIT. Sus características incluyen:

- **Arquitectura basada en nodos y escenas.** Toda entidad es un nodo dentro de un árbol de escenas.
- **Lenguaje GDScript.** Lenguaje propio con sintaxis similar a Python. También soporta C# y C++.
- **Código abierto.** Permite acceder y modificar el código fuente del motor.
- **Sistema de *TileMap* y *TileSet*.** Proporciona herramientas para el diseño de niveles 2D.
- **Sin costes de licencia.**

Unreal Engine

Unreal Engine, desarrollado por Epic Games, es un motor enfocado principalmente en gráficos de alta fidelidad para juegos 3D AAA:

- **Sistema *Blueprints*.** Permite programar mediante un sistema de scripting visual.
- **Lenguaje C++.** Ofrece rendimiento nativo y control de bajo nivel.
- **Orientación principal a 3D.** Carece de herramientas nativas específicas como *tile-maps*.
- **Peso del editor.** El editor tiene un tamaño considerable (más de 60 GB).

GameMaker

GameMaker, desarrollado por YoYo Games, es un motor orientado al desarrollo de videojuegos 2D:

- **Lenguaje GML (*GameMaker Language*).** Orientado a la simplicidad.
- **Especialización en 2D.** Herramientas nativas optimizadas para la creación de juegos bidimensionales.
- **Limitaciones en proyectos complejos.** Arquitectura menos flexible para proyectos con requerimientos de modularidad avanzada.
- **Licencia de pago.**

2.3.2 Tabla comparativa de motores de desarrollo

La elección del motor de desarrollo es un factor crítico para el éxito del proyecto. La siguiente tabla evalúa las alternativas más relevantes del mercado, comparando criterios técnicos clave —como el soporte nativo 2D y la gestión de datos— para justificar la idoneidad de la tecnología seleccionada para Ascension.

Criterio	Unity	Godot	Unreal Engine	GameMaker
Licencia	Gratuito (con restricciones)	MIT (libre)	Gratuito (regalías al 5 %)	Pago
Lenguaje principal	C#	GDScript / C#	C++ / Blueprints	GML
Soporte 2D nativo	Excelente	Bueno	Limitado	Excelente
Sistema de <i>Tilemaps</i>	Integrado y maduro	Integrado (Godot 4)	No nativo	Integrado
<i>ScriptableObjects</i>	Sí	Recursos equivalentes	<i>Data Assets</i>	No
Documentación y comunidad	Muy amplia	Creciente	Amplia (centrada en 3D)	Moderada
Soporte multiplataforma	+20 plataformas	+10 plataformas	+10 plataformas	+10 plataformas
Curva de aprendizaje	Moderada	Baja-moderada	Alta	Baja
Ideoneidad para <i>roguelikes</i> 2D	Alta	Alta	Baja	Moderada

Tabla 2.5: Comparativa de motores de desarrollo de videojuegos.

2.3.3 Justificación de la elección de Unity y C#

Se seleccionó Unity frente a Godot, Unreal Engine y GameMaker tras evaluar los criterios recogidos en la Tabla 2.5. La decisión se basa en cuatro requisitos técnicos concretos del proyecto:

1. **Madurez del sistema de *Tilemaps*.** La generación procedural de salas basada en *tilemaps* constituye una funcionalidad central del proyecto. El componente *Tilemap Collider2D* de Unity genera colisionadores de forma nativa a partir de la disposición de tiles, mientras que en Godot la creación de colisionadores por tile requería configuración manual adicional.
2. **Sistema de *ScriptableObjects*.** Resuelve un problema arquitectónico concreto: la separación entre datos configurables y lógica de ejecución. Sin este mecanismo, ajustar un parámetro de equilibrado obligaría a recompilar el proyecto.
3. **Lenguaje C#.** Ofrece tipado fuerte, soporte completo para programación orientada a objetos (herencia, polimorfismo, interfaces), manejo de excepciones, eventos y delegados. Estas características resultan fundamentales para la implementación de patrones de diseño como *Singleton*, *Observer* y *Strategy*.

4. **Documentación y soporte.** La documentación oficial de Unity, los foros de la comunidad y la abundancia de recursos formativos facilitan la resolución de problemas técnicos de forma autónoma.

2.3.4 Correspondencia entre funcionalidades de Unity/C# y subsistemas del proyecto

La implementación de los subsistemas en Ascension depende de características específicas del motor y del lenguaje C#. La siguiente tabla detalla estas funcionalidades y su equivalencia en motores alternativos, permitiendo comprender cómo la arquitectura del proyecto (basada en patrones como Observer o Strategy) se ve favorecida por las herramientas nativas de Unity.

Funcionalidad de Unity / C#	Subsistema que la utiliza	Godot	Unreal	GameMaker
TilemapCollider2D (colisionadores automáticos)	Generación procedural de salas (RoomGenerator)	Requiere config. manual por tile	No nativo para 2D	No aplicable
CompositeCollider2D (fusión de colisionadores)	Bordes perimetrales de sala	Equivalente parcial	No nativo para 2D	No aplicable
ScriptableObject + [CreateAssetMenu]	Datos de enemigos, armas, clases, tiles	Resource (menor integración)	Data Asset (requiere C++)	No disponible
Delegados y eventos (event Action<T>)	Patrón <i>Observer</i> en GameManager	Señales (<i>signals</i>)	Delegados multi-cast (C++)	No disponible
Herencia + virtual / override	Patrón <i>Template Method</i> (enemigos), <i>Strategy</i> (armas)	Herencia con <i>extends</i>	Herencia en C++	No soporta herencia
Corutinas (IEnumerator + yield return)	Invulnerabilidad, efectos temporales, espera	Equivalente (<i>await</i>)	Latent actions	Alarmas (limitadas)
DontDestroyOnLoad	Persistencia de <i>Singletons</i> entre escenas	autoload	GameInstance	Objetos persistentes
[SerializeField]	Exposición de campos privados al inspector	@export	UPROPERTY()	No aplicable
Physics2D.OverlapCircle / CompareTag	Detección de colisiones e identificación de entidades	Áreas 2D + grupos	Canales de colisión (C++)	Colisiones simples

Tabla 2.6: Correspondencia entre funcionalidades de Unity/C# utilizadas en el proyecto y sus equivalentes en motores alternativos.

2.3.5 Otras herramientas utilizadas

Además del motor de desarrollo, el proyecto ha contado con un ecosistema de herramientas auxiliares que optimizan la calidad del código, el control de versiones y el diseño visual. La siguiente tabla detalla este conjunto de herramientas y las razones técnicas de su selección.

Herramienta	Propósito	Justificación
Visual Studio	Entorno de desarrollo integrado (IDE)	Autocompletado inteligente para C# y Unity, depuración integrada, refactorización
LibreSprite	Creación de <i>pixel art</i>	Editor de sprites de código abierto con soporte para animaciones frame-by-frame
Git	Control de versiones	Seguimiento de cambios, gestión de ramas y recuperación de estados anteriores del proyecto
PlantUML	Diagramas UML (<i>Unified Modeling Language</i>)	Generación de diagramas UML a partir de texto plano
TextMesh Pro (Unity)	Renderizado de texto en UI	Sistema de renderizado de texto de alta calidad integrado en Unity

Tabla 2.7: Herramientas complementarias utilizadas en el desarrollo.

LibreSprite: creación de sprites y animación en *pixel art*

En un videojuego 2D con estética *pixel art*, la herramienta de creación gráfica condiciona directamente la calidad del resultado y la eficiencia del flujo de producción de activos. En *Ascension*, LibreSprite se emplea para la elaboración de *sprites* (personaje, enemigos, proyectiles y tiles) y para animaciones *frame-by-frame*.

Para contextualizar su elección, la Tabla 2.8 compara LibreSprite con dos herramientas habituales en proyectos 2D:

Criterio	LibreSprite	Aseprite	Figma
Enfoque principal	<i>Pixel art</i> y animación 2D	<i>Pixel art</i> y animación 2D	Diseño UI/UX (vectorial)
Animación <i>frame-by-frame</i>	Sí	Sí	Limitada (no orientada a sprites)
Control de píxel y paletas	Alto	Alto	Medio (depende de plugins/flujo)
Adecuación para sprites 16×16 / 32×32	Alta	Alta	Baja-moderada

Tabla 2.8: Comparativa de herramientas para producción de gráficos 2D.

Se seleccionó LibreSprite frente a Aseprite y Figma por dos razones concretas. En primer lugar, LibreSprite es *software* de código abierto y licencia libre. En segundo lugar, Figma se descartó porque su enfoque vectorial no está diseñado para edición *pixel-perfect* a resoluciones de 16×16 o 32×32 píxeles.

Desde el punto de vista de integración con el motor, el flujo de trabajo consiste en exportar los recursos desde LibreSprite en formato PNG (como sprites individuales o *spritesheets*) e importarlos en Unity configurando parámetros de *pixel art*: Pixels Per Unit coherente con

el tamaño de tile (16), filtrado sin suavizado (*Point*) y compresión desactivada para preservar la nitidez.

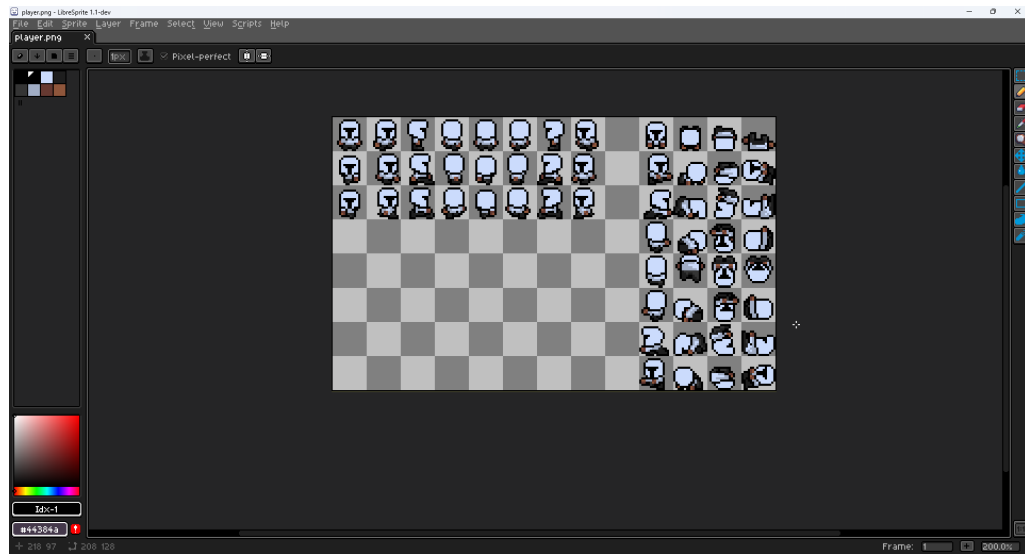


Figura 2.9: Captura de la interfaz de LibreSprite durante la edición de un *sprite* del juego *Ascension*, mostrando la paleta de colores y el lienzo de edición. *Elaboración propia.*

2.4 PATRONES DE DISEÑO EN EL DESARROLLO DE VIDEOJUEGOS

El desarrollo de videojuegos de complejidad media-alta requiere la aplicación de patrones de diseño (Gamma et al., 1994) [16] que faciliten la organización del código, la separación de responsabilidades y la evolución del sistema. A continuación se describen los patrones más relevantes en este contexto, que se han aplicado en el desarrollo de *Ascension*.

2.4.1 Patrón *Singleton*

El patrón *Singleton* establece que una clase disponga de una única instancia en toda la aplicación y proporciona un punto de acceso global a dicha instancia. Su uso implica un compromiso: simplifica el acceso a servicios globales pero introduce un acoplamiento implícito. Se seleccionó el patrón *Singleton* frente a la inyección de dependencias por una razón práctica: en un proyecto individual con cuatro gestores centrales, la infraestructura de un contenedor de inyección de dependencias habría añadido complejidad sin beneficio proporcional. En *Ascension*, se utiliza de forma controlada en los gestores centrales (*GameManager*, *ScoreManager*, *RoomFlowController*, *DamageBoostManager*), todos ellos persistentes entre escenas mediante `DontDestroyOnLoad`.

2.4.2 Patrón *Observer*

El patrón *Observer* define una relación de dependencia uno-a-muchos entre objetos. Se seleccionó este patrón frente a la alternativa directa (que el *GameManager* invocara explícitamente métodos de cada componente de interfaz) porque esta última habría creado una dependencia inversa que violaría el principio de inversión de dependencias. En *Ascension*,

el *GameManager* expone eventos como *OnGameStateChanged*, *OnRoomCleared* y *OnEnemy Killed*, a los que se suscriben los componentes de interfaz de usuario y otros subsistemas.

2.4.3 Patrón *Strategy*

El patrón *Strategy* define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables. Se seleccionó frente a la alternativa de una única clase *Weapon* con condicionales *if/else*. La clase base *Weapon* define la interfaz común (método *Attack()*), mientras que *MeleeWeapon* y *RangedWeapon* proporcionan implementaciones específicas. Esto permite añadir nuevos tipos de arma creando una nueva subclase sin modificar el código existente.

2.4.4 Patrón *Template Method*

El patrón *Template Method* define el esqueleto de un algoritmo en una operación, delegando en las subclases la definición de determinados pasos. En *Ascension*, la clase base *Enemy* define el ciclo de vida común de todos los enemigos, mientras que las subclases (*SlimeGreen*, *SlimeRed*, *SlimeBlue*) sobrescriben los métodos virtuales para definir su comportamiento específico de IA.

2.4.5 Arquitectura basada en componentes (Entidad-Componente)

Unity se fundamenta en una arquitectura basada en componentes, donde las entidades del juego (*GameObjects*) se definen como contenedores vacíos a los que se agregan componentes (*MonoBehaviour*) que encapsulan funcionalidades independientes. Esta arquitectura favorece la composición sobre la herencia.

2.4.6 Máquina de estados (*State Machine*)

Una máquina de estados define un conjunto finito de estados posibles y las transiciones válidas entre ellos. En *Ascension*, el *GameManager* implementa una máquina de estados que controla el flujo global del juego con los estados: *Menu*, *CharacterSelection*, *Playing*, *Paused*, *GameOver* y *Victory*.

2.5 SÍNTESIS DEL ESTADO DEL ARTE

El análisis realizado en este capítulo permite establecer las siguientes conclusiones:

1. El subgénero *roguelike* presenta retos técnicos de considerable complejidad que justifican su abordaje en un Trabajo de Fin de Grado, incluyendo la generación procedural de contenido, la implementación de inteligencia artificial reactiva y la gestión de múltiples subsistemas en tiempo real. Su creciente relevancia en la industria del videojuego, tanto en términos comerciales como en reconocimiento crítico, refuerza la pertinencia de su estudio.
2. Los títulos comerciales analizados (*The Binding of Isaac*, *Enter the Gungeon*, *Hades* y *Dead Cells*) proporcionan referencias de diseño sólidas en cuanto a estructura de

niveles, mecánicas de combate, sistemas de enemigos y experiencia de usuario, que se han considerado en el diseño de *Ascension*. El sistema de tiles con efectos constituye un elemento diferenciador del proyecto respecto a las referencias estudiadas.

3. Unity, combinado con el lenguaje C#, constituye la herramienta más adecuada para el desarrollo del proyecto, dada la madurez de su sistema de *tilemaps*, su soporte para *ScriptableObjects* y la disponibilidad de documentación y recursos.
4. Los patrones de diseño identificados (*Singleton*, *Observer*, *Strategy* y *Template Method*) constituyen herramientas fundamentales para la organización del código en proyectos de esta naturaleza y se han aplicado en el desarrollo del proyecto. Adicionalmente, se han empleado mecanismos de estructuración propios del motor (arquitectura basada en componentes) y una máquina de estados para el control del flujo global.

3 DISEÑO DEL SISTEMA

El presente capítulo describe el diseño del videojuego *Ascension*. Se abordan la terminología del dominio, los requisitos funcionales expresados mediante historias de usuario, la propuesta arquitectónica del sistema y los patrones de diseño aplicados. El objetivo es proporcionar una visión completa de las decisiones de diseño antes de abordar la implementación detallada de los subsistemas.

3.1 TERMINOLOGÍA UTILIZADA EN ESTE CAPÍTULO

Antes de abordar los aspectos técnicos del desarrollo, se definen los términos específicos del dominio de los videojuegos y del motor Unity que se utilizan a lo largo de este capítulo:

Tabla 3.1: Glosario de términos técnicos utilizados en este capítulo.

Término	Definición
<i>Sprite</i>	Imagen bidimensional que representa visualmente un elemento del juego (personaje, enemigo, tile, objeto). En <i>pixel art</i> , cada <i>sprite</i> se compone de un número reducido de píxeles con colores definidos.
<i>Tile</i>	Celda cuadrada de tamaño fijo que constituye la unidad básica de construcción de un nivel. Las salas de <i>Ascension</i> se componen de tiles de 16×16 píxeles organizados en una rejilla.
<i>Tilemap</i>	Rejilla bidimensional que almacena y renderiza tiles. Unity proporciona un componente <i>Tilemap</i> que gestiona automáticamente el posicionamiento y la visualización de los tiles.
<i>Hitbox</i>	Área invisible asociada a un objeto del juego que define la zona en la que se detectan impactos. Cuando el <i>hitbox</i> de un arma se superpone con el <i>hitbox</i> de un enemigo, se registra un golpe.
<i>Collider</i>	Componente de Unity que define la forma física de un objeto para la detección de colisiones. Un <i>BoxCollider2D</i> define un rectángulo; un <i>TilemapCollider2D</i> genera colisionadores automáticos a partir de los tiles de un <i>tilemap</i> .

Término	Definición
<i>Trigger</i>	Modo especial de un <i>collider</i> que detecta la superposición con otros objetos sin producir una respuesta física (sin empuje ni rebote). Se utiliza para detectar eventos como la entrada del jugador en una sala.
<i>Prefab</i>	Plantilla reutilizable de un objeto del juego con todos sus componentes preconfigurados. Al instanciar un <i>prefab</i> , se crea una copia independiente en la escena.
<i>ScriptableObject</i>	Contenedor de datos de Unity que permite almacenar configuraciones (estadísticas de enemigos, datos de armas, efectos de tiles) como activos del proyecto, editables desde el inspector sin necesidad de modificar código.
<i>Corutina</i>	Función de Unity que permite ejecutar código distribuido en múltiples fotogramas. Se utiliza para efectos temporales como el parpadeo de invulnerabilidad, el parpadeo rojo de muerte o la persistencia de efectos de tiles.
<code>Awake()</code> / <code>Start()</code>	Métodos del ciclo de vida de Unity que se ejecutan al inicializar un objeto. <code>Awake()</code> se llama al cargar el componente; <code>Start()</code> se ejecuta justo antes del primer fotograma.
<code>Update()</code> / <code>FixedUpdate()</code>	Métodos que Unity invoca de forma repetida. <code>Update()</code> se ejecuta una vez por fotograma (entrada y lógica general) y <code>FixedUpdate()</code> se ejecuta a intervalos fijos (física).
<code>Time.deltaTime</code>	Tiempo transcurrido desde el último fotograma. Se usa para que el movimiento y otros cálculos sean independientes de la tasa de fotogramas.
<code>Time.timeScale</code>	Factor que escala el tiempo del juego. Un valor cero detiene la simulación (útil para pausar); uno es tiempo normal.
<code>[SerializeField]</code>	Atributo de C# usado en Unity para exponer un campo privado a configuración sin hacerlo público, permitiendo ajustar valores sin modificar código.
<code>Instantiate(...)</code>	Función que crea una instancia en tiempo de ejecución a partir de una plantilla (<i>prefab</i>) u objeto existente.
<i>Rigidbody2D</i>	Componente de física de Unity que aplica simulación física bidimensional a un objeto (gravedad, velocidad, colisiones). En <i>Ascension</i> se utiliza con gravedad desactivada para movimiento cenital.
<code>DontDestroyOnLoad</code>	Método de Unity que impide que un objeto se destruya al cambiar de escena, permitiendo que gestores como <code>GameManager</code> persistan durante toda la ejecución.

3.2. REQUISITOS FUNCIONALES: HISTORIAS DE USUARIO

Término	Definición
Gizmos	Elementos gráficos de depuración visibles únicamente durante el desarrollo (no en el juego final). Se utilizan para visualizar rangos de ataque, áreas de aparición y otros parámetros de diseño.
PPU (<i>Pixels Per Unit</i>)	Número de píxeles del <i>sprite</i> que equivalen a una unidad del mundo del juego. En <i>Ascension</i> , el valor es dieciséis, lo que significa que un tile de 16×16 píxeles ocupa exactamente una unidad en el mundo.

3.2 REQUISITOS FUNCIONALES: HISTORIAS DE USUARIO

La captura de requisitos funcionales se ha realizado mediante Historias de Usuario (HU), siguiendo la plantilla estándar del marco de trabajo ágil: «*Como [rol], quiero [funcionalidad], para [beneficio]*». Cada historia de usuario incluye criterios de aceptación verificables, una estimación en puntos de historia y una prioridad asignada. Las historias se han organizado por subsistemas funcionales.

Los criterios de aceptación se utilizan como especificación de requisitos. En el alcance del presente trabajo, su comprobación se ha realizado de forma manual durante el desarrollo mediante ejecución y observación en el editor de Unity, sin instrumentación específica para métricas de rendimiento ni una batería sistemática de casos de prueba automatizados.

3.2.1 Sistema de combate

HU-001: Ataque cuerpo a cuerpo. *Como jugador, quiero atacar enemigos con un arma cuerpo a cuerpo mediante un gesto de barrido, para eliminar amenazas cercanas de forma intuitiva.* Criterios de aceptación: el ataque se ejecuta al mantener presionado el botón izquierdo del ratón; el arma realiza un arco de 180 grados; existe un tiempo de espera de 0,5 segundos entre ataques; los enemigos dentro del área de impacto reciben daño; se muestra retroalimentación visual mediante un efecto de rastro. Prioridad: alta. Estimación: cinco puntos.

HU-002: Ataque a distancia. *Como jugador, quiero atacar enemigos desde una distancia segura disparando proyectiles, para mantener una posición estratégica durante el combate.* Criterios: el disparo se ejecuta al hacer clic con el botón izquierdo del ratón; el proyectil se dirige hacia la posición del cursor; existe un tiempo de espera de 0,3 segundos entre disparos; los proyectiles tienen un tiempo de vida de tres segundos; los proyectiles colisionan y se destruyen al impactar contra enemigos o muros. Prioridad: alta. Estimación: cinco puntos.

HU-003: Sistema de daño. *Como jugador, quiero que los enemigos muestren retroalimentación visual al ser golpeados, para obtener información inmediata sobre la efectividad de los ataques.* Criterios: el enemigo muestra un destello rojo de 0,1 segundos al recibir daño; la barra de vida se actualiza instantáneamente; el enemigo es destruido cuando su vida llega a cero; se otorga puntuación por cada eliminación. Prioridad: media. Estimación: tres puntos.

HU-004: Recepción de daño del jugador. *Como jugador, quiero experimentar un periodo de invulnerabilidad temporal tras recibir daño, para disponer de tiempo para reaccionar y escapar de situaciones peligrosas.* Criterios: el jugador recibe daño al colisionar con enemigos o proyectiles enemigos; se activa un periodo de invulnerabilidad de un segundo; durante la invulnerabilidad, el sprite parpadea en blanco a doce Hz; la interfaz de corazones se actualiza inmediatamente. Prioridad: alta. Estimación: cinco puntos.

3.2.2 Sistema de movilidad

HU-005: Movimiento básico. *Como jugador, quiero mover el personaje en ocho direcciones utilizando las teclas WASD, para explorar las salas y esquivar ataques enemigos.* Criterios: el movimiento responde a las teclas W, A, S, D; el personaje se mueve en las ocho direcciones cardinales e intercardinales; la velocidad se normaliza para evitar movimiento diagonal más rápido; el personaje no puede atravesar muros. Prioridad: crítica. Estimación: tres puntos.

HU-006: Habilidad de esquiva (roll). *Como jugador, quiero realizar una esquiva rápida con invulnerabilidad temporal, para evitar patrones de ataque enemigos.* Criterios: la esquiva se activa con la tecla Espacio; el jugador se desplaza al doble de velocidad durante 0,4 segundos; durante la esquiva, el jugador es invulnerable a todo daño y atraviesa enemigos y proyectiles enemigos; existe un tiempo de espera de 0,6 segundos entre esquivas. Prioridad: alta. Estimación: cinco puntos.

HU-007: Interacción con tiles de velocidad. *Como jugador, quiero que la velocidad de movimiento se vea afectada por el tipo de suelo, para añadir variabilidad táctica al posicionamiento.* Criterios: los tiles de hielo incrementan la velocidad en un 50 %; los tiles de lodo la reducen en un 50 %; el efecto persiste durante cinco segundos tras abandonar el tile; los enemigos también se ven afectados. Prioridad: media. Estimación: cinco puntos.

3.2.3 Sistema de progresión de salas

HU-008: Transición entre salas. *Como jugador, quiero avanzar a la siguiente sala al derrotar a todos los enemigos e interactuar con las escaleras, para progresar en la partida.* Criterios: las escaleras se bloquean al entrar si hay enemigos; las escaleras se habilitan al derrotar a todos los enemigos; aparecen escaleras en una posición aleatoria de la sala; al presionar la tecla E sobre las escaleras se genera una nueva sala; la nueva sala presenta mayor dificultad. Prioridad: crítica. Estimación: ocho puntos.

HU-009: Bloqueo de escaleras durante combate. *Como jugador, quiero que las escaleras de la sala se bloqueen cuando hay enemigos activos, para que cada combate deba completarse antes de avanzar.* Criterios: las escaleras se bloquean automáticamente; se habilitan únicamente cuando todos los enemigos han sido eliminados. Prioridad: alta. Estimación: tres puntos.

3.2.4 Sistema de enemigos

HU-010: Enemigos con inteligencia artificial diferenciada. *Como jugador, quiero enfrentar enemigos con comportamientos diferenciados, para experimentar variedad en los de-*

saffos de combate. Criterios: existen tres arquetipos de enemigo (francotirador, perseguidor, saltador); cada arquetipo tiene un patrón de movimiento único; los enemigos no atacan inmediatamente tras aparecer (0,75 s de gracia); los enemigos respetan la física del entorno. Prioridad: crítica. Estimación: trece puntos.

Aclaración terminológica: en este documento, el término *inteligencia artificial* hace referencia a la lógica de control desarrollada específicamente para los enemigos (scripts en C# basados en máquinas de estados finitos (FSM) y eventos). Unity proporciona el entorno de ejecución (ciclo `Update()`, físicas y colisiones), pero no se utiliza un sistema de IA *integrado* del motor.

HU-011: Sistema de aparición de enemigos. *Como desarrollador, quiero que los enemigos aparezcan de forma segura y equilibrada en cada sala, para favorecer una experiencia de juego justa.* Criterios: los enemigos nunca aparecen a menos de 2,5 unidades del jugador; el sistema realiza 25 intentos de posicionamiento antes de utilizar una posición de respaldo; el número de enemigos escala con la profundidad de la sala; se soportan modos de aparición por cantidad fija o por presupuesto de coste. Prioridad: alta. Estimación: ocho puntos.

HU-012: Variantes de slimes. *Como jugador, quiero que cada tipo de slime tenga un color y comportamiento distintivo, para poder anticipar sus patrones de ataque.* Criterios: el slime verde mantiene distancia y dispara proyectiles; el slime rojo persigue al jugador para daño por contacto; el slime azul realiza saltos rápidos hacia el jugador; cada slime tiene sprites únicos. Prioridad: media. Estimación: ocho puntos.

3.2.5 Sistema de progresión y puntuación

HU-013: Sistema de puntuación. *Como jugador, quiero acumular puntos por los logros durante la partida.* Criterios: 100 puntos por sala completada; puntos variables por enemigo eliminado (daño del enemigo $\times 5$); 1.000 puntos de bonificación por completar el juego. Prioridad: media. Estimación: tres puntos.

HU-014: Historial de partidas. *Como jugador, quiero ver un registro de las mejores partidas anteriores.* Criterios: el historial almacena hasta 30 partidas; cada entrada muestra puntuación, clase, salas, enemigos, resultado y fecha; los datos persisten entre sesiones de juego; se puede limpiar el historial desde el menú principal. Prioridad: baja. Estimación: cinco puntos.

HU-015: Sistema de clases de personaje. *Como jugador, quiero elegir entre diferentes clases de personaje con estadísticas únicas.* Criterios: cada clase tiene estadísticas diferenciadas (vida, velocidad); cada clase comienza con un arma inicial específica; la selección se realiza en una escena dedicada. Prioridad: media. Estimación: cinco puntos.

HU-016: Tiles de potenciación. *Como jugador, quiero encontrar tiles especiales que mejoren temporalmente las capacidades del personaje.* Criterios: los tiles de potenciación aumentan el daño en +1 durante cinco segundos; los tiles de curación restauran un punto de vida; el efecto se aplica a todas las armas y proyectiles; los efectos se reinician al cambiar de sala. Prioridad: media. Estimación: cinco puntos.

3.2.6 Sistema de generación procedural

HU-017: Generación aleatoria de salas. *Como jugador, quiero que cada sala se genere de forma diferente en cada partida.* Criterios: cada sala tiene un tamaño base de 26×12 tiles; el suelo se compone de tiles normales y especiales distribuidos aleatoriamente; los muros forman un perímetro cerrado con colisionadores funcionales; la generación completa requiere menos de 100 ms. Prioridad: crítica. Estimación: trece puntos.

HU-018: Distribución de tiles especiales. *Como desarrollador, quiero controlar la rareza y distribución de tiles especiales, para equilibrar la dificultad.* Criterios: tiles de hielo y lodo aparecen en manchas de 4-9 tiles con un 12 % de probabilidad; tiles de potenciación aparecen individualmente con un 18 % de probabilidad (1-2 por sala); tiles de curación son extremadamente raros (2 % de probabilidad, máximo uno por sala). Prioridad: media. Estimación: ocho puntos.

HU-019: Sistema de variantes de tiles. *Como jugador, quiero ver variedad visual en los tiles del suelo.* Criterios: cada tipo de tile tiene múltiples sprites alternativos; la selección es determinista basada en la posición; se soportan hasta diez variantes por tipo de tile. Prioridad: baja. Estimación: cinco puntos.

3.2.7 Sistema de interfaz de usuario

HU-020: Representación de vida mediante corazones. *Como jugador, quiero ver la vida actual representada mediante corazones en la interfaz.* Criterios: cada corazón representa un punto de vida; la cantidad se ajusta dinámicamente según la clase seleccionada; la representación se actualiza instantáneamente al recibir o curar daño. Prioridad: alta. Estimación: cinco puntos.

HU-021: Menú de pausa. *Como jugador, quiero poder pausar el juego en cualquier momento.* Criterios: el menú se abre y cierra con la tecla ESC; el tiempo del juego se detiene (`Time.timeScale = 0`); se muestran opciones de reanudar y menú principal. Prioridad: alta. Estimación: tres puntos.

HU-022: Pantallas de fin de partida. *Como jugador, quiero ver una pantalla específica al morir o ganar, para confirmar el final de la partida.* Criterios: se muestra una pantalla de derrota o de victoria; las estadísticas se registran automáticamente en el historial. Prioridad: media. Estimación: cinco puntos.

3.2.8 Historias técnicas

HT-001: Sistema de persistencia. Implementación de persistencia de datos mediante `PlayerPrefs` para datos simples y serialización JSON para estructuras complejas, con validación de datos al cargar y límite de 30 entradas en el historial. Prioridad: media. Estimación: cinco puntos.

HT-002: Optimización de rendimiento. Uso de almacenamiento en caché de componentes en `Awake()` para evitar llamadas a `GetComponent()` en `Update()`; uso selectivo de `CollisionDetectionMode2D.Continuous`; minimización de búsquedas `GameObject.Find`. Prioridad: media. Estimación: ocho puntos.

HT-003: Sistema de registro y depuración. Implementación de mensajes de advertencia para configuraciones incorrectas, registros informativos en transiciones críticas de estado, y visualización de *Gizmos* durante el desarrollo para rangos de enemigos. Prioridad: baja. Estimación: tres puntos.

3.3 PROPUESTA ARQUITECTÓNICA

3.3.1 Visión general de la arquitectura

La arquitectura del sistema se fundamenta en el paradigma de programación orientada a componentes proporcionado por Unity Engine, combinado con patrones de diseño de la ingeniería del software clásica (*Singleton*, *Observer*, *Strategy*, *Template Method*). Se seleccionó esta combinación frente a una arquitectura puramente basada en herencia porque el sistema de componentes de Unity favorece la composición: cada entidad del juego se define como un `GameObject` al que se agregan componentes independientes, lo que permite reutilizar funcionalidades sin crear jerarquías de herencia profundas. Los patrones de diseño complementan esta arquitectura proporcionando soluciones probadas para problemas recurrentes (acceso global a gestores (*Singleton*), desacoplamiento entre lógica y presentación (*Observer*), intercambiabilidad de algoritmos (*Strategy*) y reutilización de comportamiento base (*Template Method*)) que el sistema de componentes por sí solo no resuelve.

La estructura resultante se organiza en dos capas diferenciadas:

- **Capa de lógica de juego.** Contiene los componentes que implementan las reglas del juego: `GameManager` (máquina de estados y progresión), `RoomFlowController` (secuenciación de salas), `RoomGenerator` (generación procedural de tiles y muros), `Enemy Manager` (aparición y seguimiento de enemigos), `PlayerController` (movimiento en ocho direcciones y esquiva), `PlayerHealth` (gestión de puntos de vida e invulnerabilidad), la jerarquía de enemigos (`Enemy`, `SlimeGreen`, `SlimeRed`, `SlimeBlue`) y el sistema de armas (`Weapon`, `MeleeWeapon`, `RangedWeapon`, `Projectile`). En términos generales, se evita que esta capa dependa directamente de la interfaz de usuario; cuando se requiere una actualización visual inmediata y acotada (por ejemplo, la vida en corazones), se utiliza una referencia explícita a un componente de presentación.
- **Capa de presentación.** Contiene los componentes de interfaz: `HeartDisplay` (visualización de vida mediante iconos de corazones), `ScoreDisplay` (puntuación en tiempo real), `PauseMenu` (menú de pausa con opciones de navegación), `GameOverScreen` y `VictoryScreen` (pantallas de fin de partida con estadísticas), y `ScoreboardMenuUI` (historial de puntuaciones con hasta treinta entradas). Estos componentes se suscriben a eventos emitidos por la capa de lógica y reaccionan a los cambios de estado sin que la capa de lógica conozca su existencia.

La comunicación entre ambas capas se realiza principalmente mediante el patrón *Observer*: los gestores centrales (*managers*) exponen eventos C# (definidos mediante delegados con nombre) a los que los componentes de presentación se suscriben durante su inicialización. Por ejemplo, cuando el `GameManager` cambia su estado a `Paused`, emite el evento `On`

GameStateChanged; el PauseMenu, suscrito a este evento, activa su panel de interfaz y la escala de tiempo del juego se detiene. Para actualizaciones de interfaz muy concretas y de baja complejidad (como la visualización de corazones) se emplea una referencia directa (Player Health -> HeartDisplay) por simplicidad, evitando introducir infraestructura de eventos adicional para un caso puntual.

3.3.2 Flujo de escenas

El proyecto se estructura en tres escenas de Unity que representan las fases del flujo de juego:

1. **MainMenu.** Escena del menú principal, con opciones para iniciar partida, consultar el historial de puntuaciones y salir del juego.
2. **ClassSelection.** Escena de selección de clase de personaje, donde el jugador elige entre las clases disponibles antes de comenzar la partida.
3. **GameScene.** Escena principal del juego, donde se desarrolla toda la acción. La generación procedural de salas se realiza dentro de esta misma escena, regenerando el contenido del *tilemap* sin necesidad de cargar escenas adicionales.

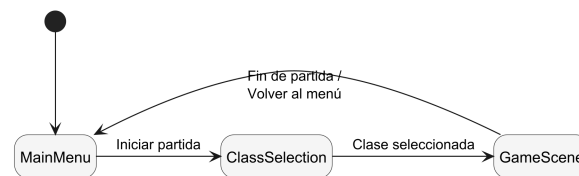


Figura 3.2: Diagrama del flujo de escenas. *Elaboración propia.*

3.3.3 Gestores centrales (*managers*)

El sistema utiliza cuatro gestores centrales que implementan el patrón *Singleton*. Tres de ellos (GameManager, ScoreManager y DamageBoostManager) persisten entre cambios de escena mediante `DontDestroyOnLoad` (directiva que evita que Unity destruya el objeto al cargar otra escena); el `RoomFlowController` opera como *Singleton* de escena, re-instanciándose en cada escena de juego y re-enlazándose a los componentes necesarios mediante el evento `SceneManager.sceneLoaded`:

Gestor	Responsabilidad
GameManager	Control de estados del juego, progresión, transiciones entre escenas
ScoreManager	Cálculo de puntuación, persistencia del historial de partidas
RoomFlowController	Control del bucle de generación y progresión de salas
DamageBoostManager	Gestión de bonificaciones temporales de daño

Tabla 3.2: Gestores centrales del sistema.

3.3.4 Diagrama de dependencias

La estructura de dependencias entre los módulos principales del sistema se organiza de la siguiente forma. El diagrama muestra la relación jerárquica entre componentes: cada

3.3. PROPUESTA ARQUITECTÓNICA

nodo depende de los nodos situados debajo de él en la jerarquía. Las flechas indican dependencias de creación, referencia directa o suscripción a eventos. Los nodos marcados con (*ScriptableObject*) representan activos de datos configurables desde el inspector de Unity.

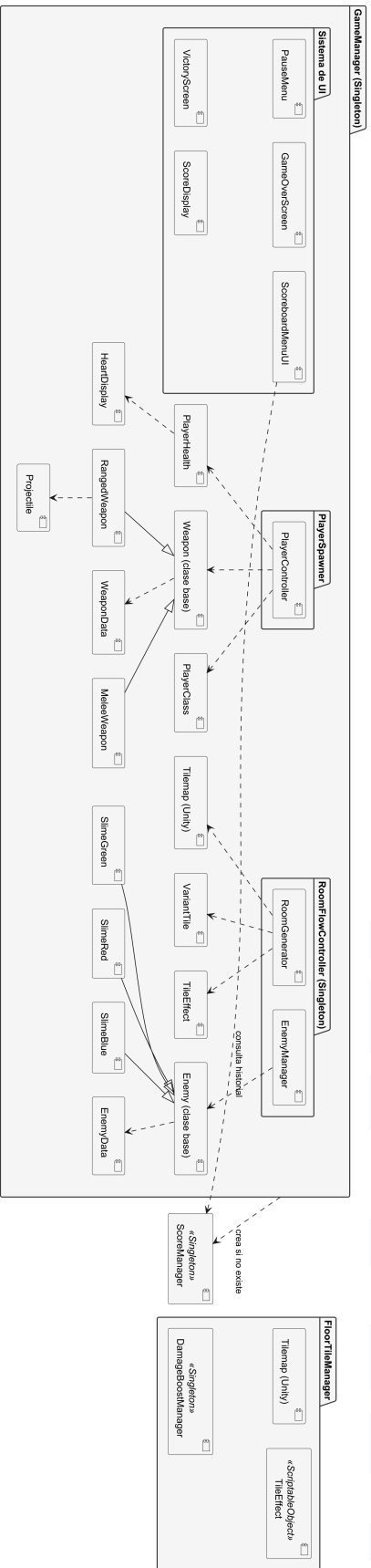


Figura 3.3: Diagrama de dependencias entre los módulos del sistema. *Elaboración propia.*

3.3.5 Flujo de producción: del activo gráfico al juego

Esta sección describe el proceso completo de producción seguido para transformar los activos gráficos (*sprites*) en entidades funcionales dentro del juego. El flujo abarca desde la creación e importación de los *sprites* hasta la configuración de los *GameObjects* con sus componentes de física, colisión y animación, y su posterior ensamblaje en la escena del juego.

Importación y configuración de *sprites*

Los activos gráficos del proyecto se crearon en LibreSprite como *pixel art* a resoluciones de 16x16, 32x32 y 64x64 píxeles, según el tipo de elemento (tiles, personajes y enemigos, respectivamente). Tras la exportación en formato PNG, la importación en Unity requiere una configuración específica para preservar la estética del *pixel art*:

- **Pixels Per Unit (PPU):** se establece en 16 para que cada tile de 16x16 píxeles ocupe exactamente una unidad del mundo del juego. Esta correspondencia simplifica el cálculo de posiciones en el *tilemap* y garantiza la coherencia entre la cuadrícula de tiles y las coordenadas del sistema de física.
- **Filtrado:** se configura como *Point (no filter)* para evitar el suavizado de bordes entre píxeles, preservando la apariencia nítida del *pixel art*.
- **Compresión:** se desactiva o se establece en modo sin pérdida (*None*) para evitar artefactos visuales que distorsionarían los bordes de los *sprites*.
- **Modo de *sprite*:** para personajes y enemigos con múltiples fotogramas de animación, se utiliza el modo *Multiple*, que permite particionar una hoja de *sprites* (*spritesheet*) en fotogramas individuales mediante el editor de *sprites* de Unity.

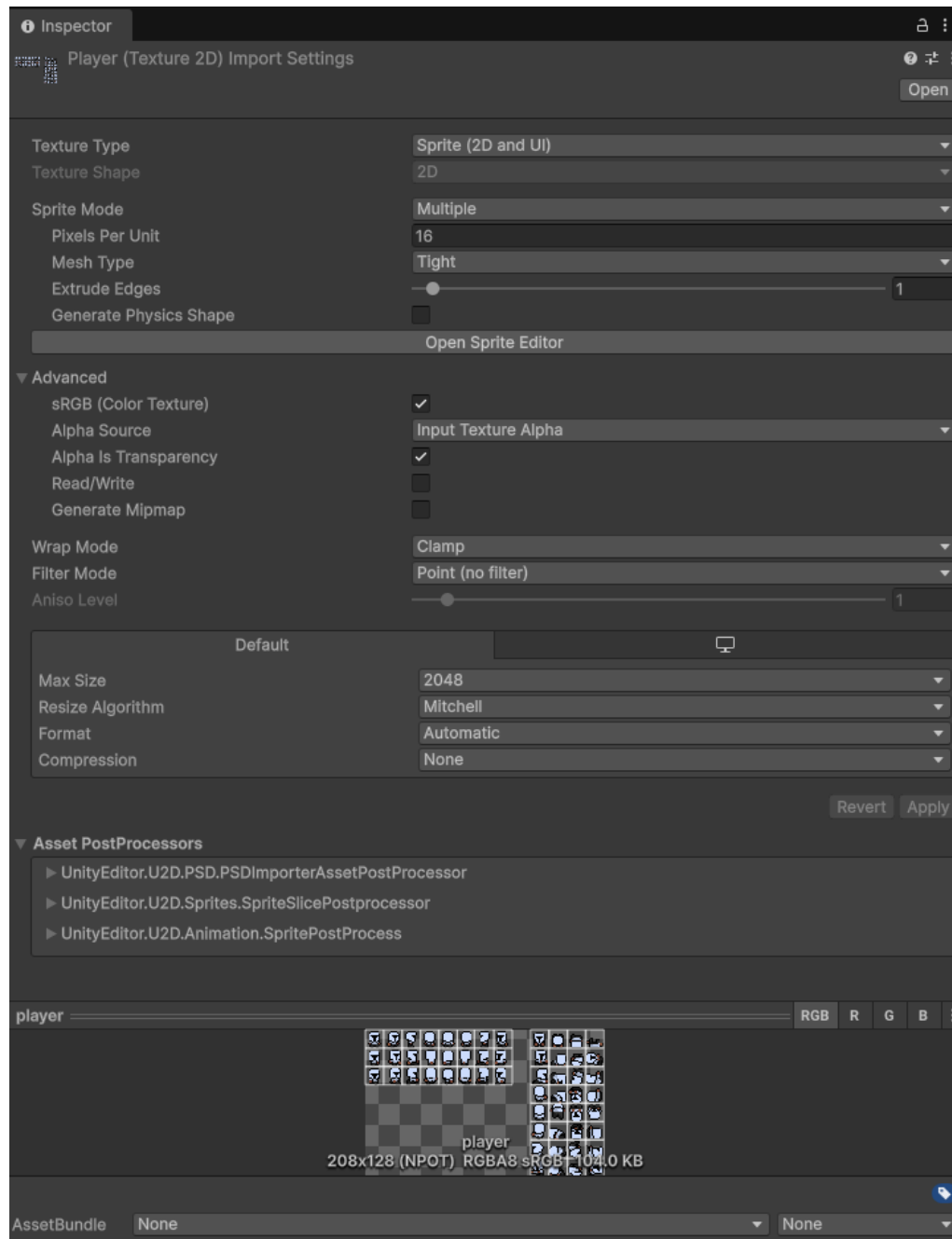


Figura 3.4: Configuración de importación de un *sprite* en el inspector de Unity, mostrando los parámetros PPU, filtrado y compresión. *Elaboración propia.*

Creación de *GameObjects* y configuración de componentes

Cada entidad del juego se construye como un *GameObject* de Unity al que se agregan los componentes necesarios para su funcionamiento. A continuación se describe la configuración típica para los principales tipos de entidad.

Jugador:

- **SpriteRenderer**: renderiza el *sprite* del personaje con el orden de capa adecuado.
- **Rigidbody2D**: habilita la simulación física con gravedad desactivada (movimiento cenital), detección de colisiones continua y rotación congelada.
- **CircleCollider2D**: define el área física del personaje para la detección de colisiones con muros y enemigos. Se ha usado un colisionador circular en la zona central y desplazado hacia abajo para que las colisiones con muros sean más naturales (evitando que el jugador pueda quedar atrapado por colisionadores rectangulares en esquinas o bordes).
- **PlayerController** (script): gestiona el movimiento y la esquivas.
- **PlayerHealth** (script): gestiona los puntos de vida y la invulnerabilidad.
- **Animator**: controla las transiciones entre estados de animación (reposo, movimiento, esquivas, daño).

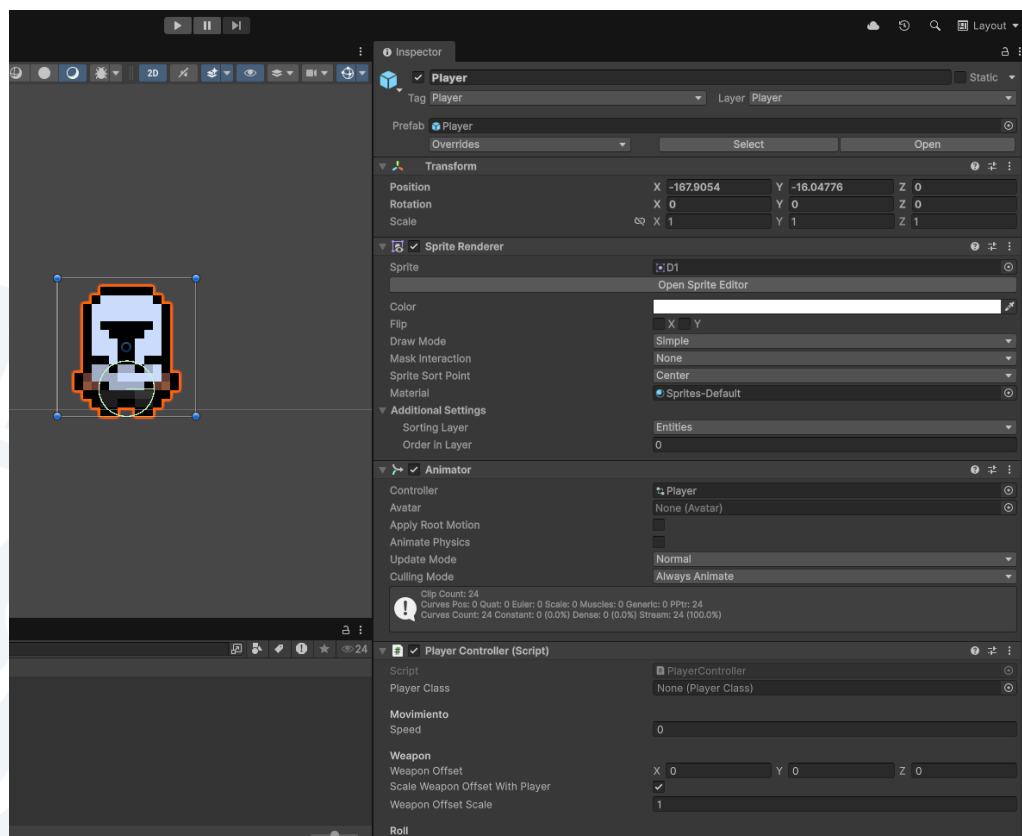


Figura 3.5: Vista del inspector de Unity mostrando los componentes configurados en el *GameObject* del jugador: *SpriteRenderer*, *Rigidbody2D*, *CircleCollider2D*, scripts y *Animator*. *Elaboración propia*.

Enemigo (ejemplo: Slime Verde):

- **SpriteRenderer**: renderiza el *sprite* del enemigo.

- Rigidbody2D: física 2D sin gravedad, con detección de colisiones continua.
- CircleCollider2D: define el área de contacto del enemigo. Se utiliza un colisionador circular en lugar de rectangular porque la forma redondeada del *slime* produce colisiones más naturales al contacto con el jugador y con los muros.
- SlimeGreen (script): implementa la IA del enemigo francotirador.
- Animator: gestiona las animaciones de movimiento, ataque y muerte.

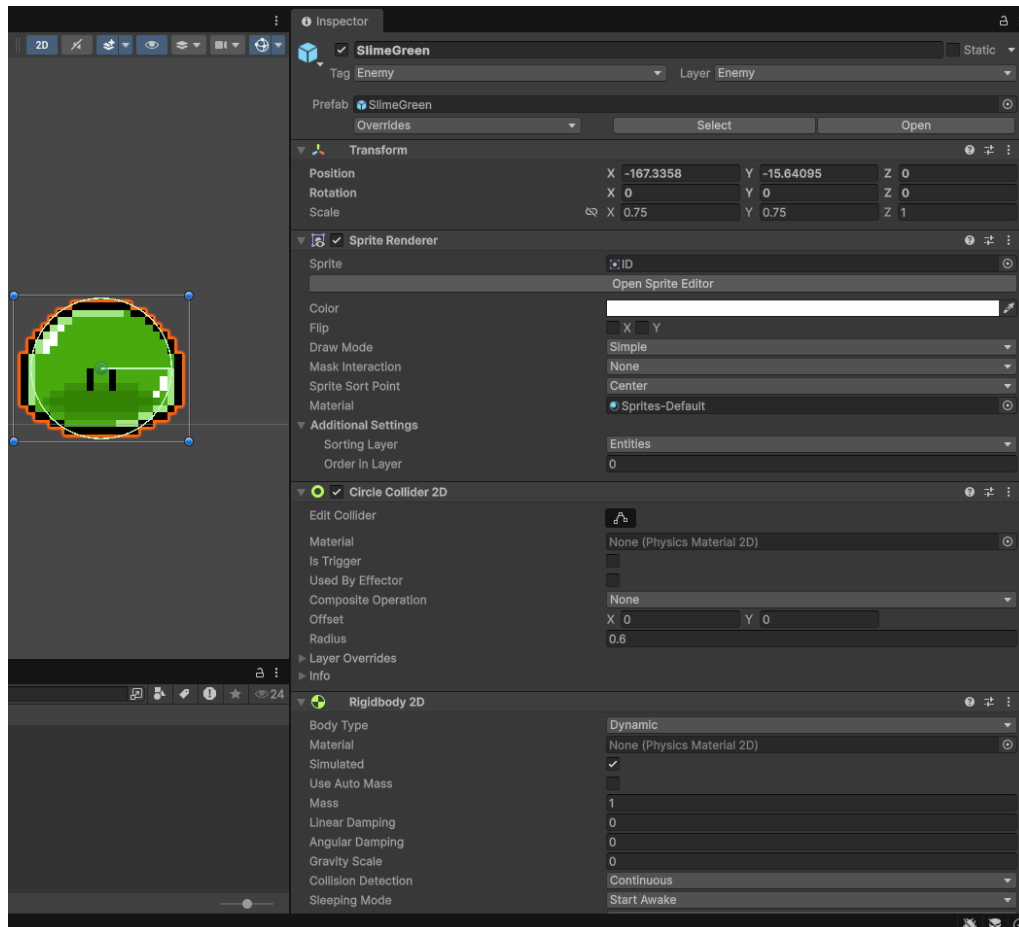


Figura 3.6: Vista del inspector de Unity mostrando los componentes de un enemigo *Slime Verde*, con detalle del *CircleCollider2D* y sus dimensiones. *Elaboración propia.*

Configuración de *colliders* y *hitboxes*

La detección de colisiones se organiza en dos categorías:

- **Colliders físicos** (modo no *trigger*): producen respuesta física (empuje, bloqueo). Se utilizan en los muros perimetrales de las salas (TilemapCollider2D + Composite Collider2D en el *tilemap* de muros) y en el cuerpo del jugador y los enemigos, impidiendo que atraviesen paredes u otros objetos sólidos.

- **Colliders de tipo trigger** (modo *Is Trigger* activado): detectan superposición sin respuesta física. Se utilizan para la detección de impactos de armas cuerpo a cuerpo (el *hitbox* del arma), la detección de proyectiles contra enemigos o muros, la activación de escaleras al presionar la tecla de interacción y la detección de tiles especiales bajo el jugador o los enemigos.

La configuración del *TilemapCollider2D* resulta especialmente relevante para la generación procedural. Al añadir un componente *TilemapCollider2D* al *tilemap* de muros, Unity genera automáticamente un colisionador por cada tile presente en ese *tilemap*. Combinado con un *CompositeCollider2D*, los colisionadores individuales se fusionan en un único colisionador perimetral continuo, lo que reduce el coste computacional de la simulación física y evita que el jugador pueda deslizarse entre los bordes de tiles adyacentes.

Configuración de *tilemaps* y tiles

El sistema de niveles se construye sobre dos *tilemaps* de Unity superpuestos en la misma escena:

1. **Tilemap de muros** (*wallTilemap*): contiene los tiles de muro que forman el perímetro de cada sala. Incluye los componentes *TilemapCollider2D* y *CompositeCollider2D* para generar la barrera física.
2. **Tilemap de suelo** (*floorTilemap*): contiene los tiles de suelo (normales y especiales) que componen el área jugable. No genera colisionadores, ya que el suelo no bloquea el movimiento.

Cada tile se configura como un activo de tipo *VariantTile* (clase derivada de *TileBase* de Unity) que almacena múltiples variantes visuales del mismo tipo de tile. Al colocar un tile, el sistema selecciona una variante de forma determinista según la posición, lo que produce variabilidad visual sin aleatorización explícita.

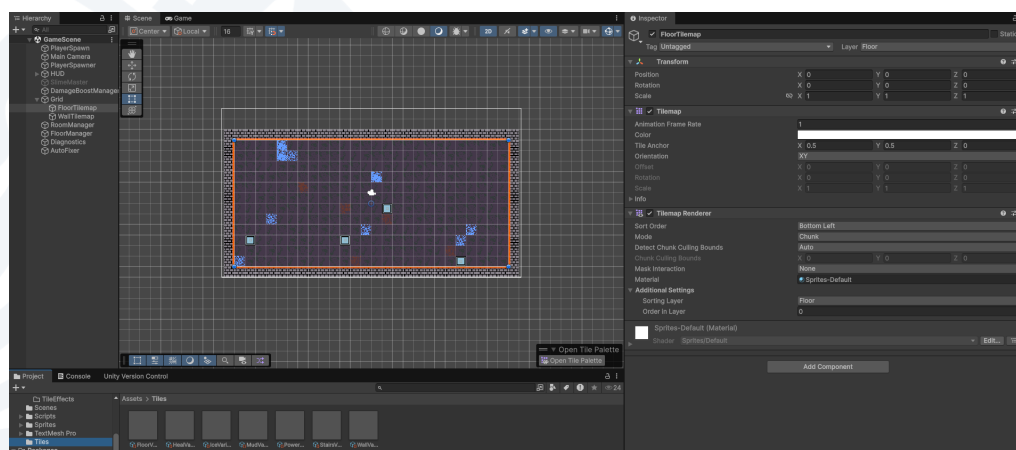


Figura 3.7: Editor de *tilemaps* de Unity mostrando las dos capas (muros y suelo). *Elaboración propia.*

Sistema de animaciones

Las animaciones del juego se implementan mediante el sistema de animación de Unity (Animator + AnimationClip). Cada personaje y enemigo dispone de un controlador de animación (Animator Controller) que define los estados de animación posibles y las transiciones entre ellos.

Para el jugador, los estados de animación incluyen:

- **Reposo** (*idle*): animación cíclica de dos fotogramas con cadencia lenta.
- **Movimiento** (*walk*): animación cíclica de cuatro fotogramas. La dirección del movimiento (arriba, abajo, izquierda, derecha) se controla mediante parámetros del *Animator* (*moveX*, *moveY*), lo que permite seleccionar la animación correspondiente a la dirección actual sin necesidad de un *blend tree* completo.
- **Esquiva** (*roll*): animación de tres fotogramas que se reproduce una vez durante la duración de la esquiva.

Para los enemigos, los estados de animación incluyen reposo, movimiento y muerte, con transiciones controladas por parámetros del *Animator* que los scripts de IA actualizan en cada fotograma.

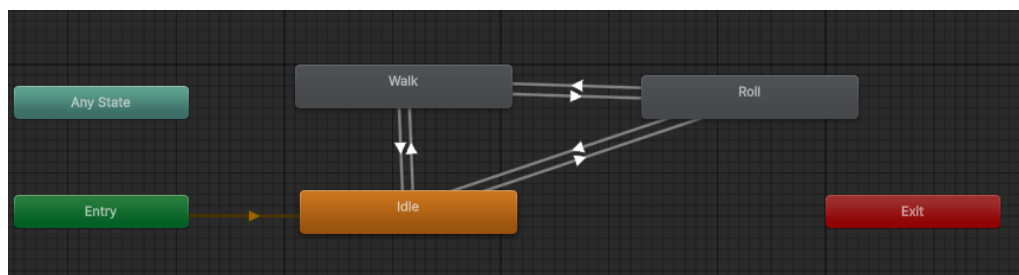


Figura 3.8: Controlador de animación (*Animator Controller*) del jugador en Unity, mostrando los estados y las transiciones entre animaciones de reposo, movimiento y esquiva. *Elaboración propia.*

De los componentes al flujo de juego

Una vez configurados los *GameObjects* individuales con sus componentes, el flujo de juego se construye mediante la coordinación de estos elementos:

1. El *PlayerSpawner* instancia el *prefab* del jugador en la posición inicial de la sala.
2. El *RoomGenerator* genera la estructura de tiles de la sala sobre los dos *tilemaps* (muros y suelo).
3. El *EnemyManager* instancia los *prefabs* de los enemigos en posiciones seguras dentro de la sala.
4. El *RoomFlowController* coordina la secuencia: bloqueo de escaleras -> combate -> habilitación de escaleras -> interacción -> siguiente sala.



3.4. PATRONES Y MECANISMOS DE DISEÑO APLICADOS

5. El `GameManager` supervisa el estado global y gestiona las transiciones entre menú, juego, pausa, victoria y derrota.

Cada *prefab* contiene la configuración completa de su *GameObject* (componentes, valores de campos serializados, referencias a *ScriptableObjects*), de modo que al instanciarlo se obtiene una copia funcional del elemento sin necesidad de configuración adicional en tiempo de ejecución.

3.4 PATRONES Y MECANISMOS DE DISEÑO APLICADOS

En el apartado de estado del arte se presentaron los patrones de diseño seleccionados para el proyecto y la fundamentación teórica de cada uno. En este apartado se describe su aplicación concreta en la arquitectura de *Ascension*: para cada patrón se presenta el código resultante y se analizan los detalles de implementación específicos del proyecto.

3.4.1 Patrón *Singleton*

Como se fundamentó en el apartado correspondiente del estado del arte, el patrón *Singleton* se seleccionó para los cuatro gestores centrales frente a la inyección de dependencias. Los tres gestores persistentes (`GameManager`, `ScoreManager`, `DamageBoostManager`) comparten la siguiente estructura:

```
1 public static GameManager Instance { get; private set; }
2
3 void Awake() {
4     if (Instance == null) {
5         Instance = this;
6         DontDestroyOnLoad(gameObject);
7     } else {
8         Destroy(gameObject);
9     }
10 }
```

El `RoomFlowController` aplica la misma estructura de instancia única, pero omite `DontDestroyOnLoad` porque sus referencias a componentes de escena (`RoomGenerator`, `EnemyManager`, `Tilemap`) se invalidan al cambiar de escena; en su lugar, se re-instancia en cada escena de juego y reconstruye sus referencias en el evento `SceneManager.sceneLoaded`.

3.4.2 Patrón *Observer*

Como se describió en el análisis de patrones del estado del arte, el patrón *Observer* desacopla el `GameManager` de los componentes de interfaz. Su implementación concreta utiliza los eventos `OnGameStateChanged`, `OnRoomCleared` y `OnEnemyKilled`:

```
1 // En GameManager (emisor)
2 public delegate void GameStateChanged(GameState newState);
3 public static event GameStateChanged OnGameStateChanged;
4
5 // En PauseMenu (receptor)
6 void Start() {
7     EnsureRuntimeUIIfMissing();
8     WireButtonsOnce();
9 }
```

```

9   GameManager.OnGameStateChanged += OnGameStateChanged;
10 }
11
12 private void OnGameStateChanged(GameState newState) {
13     EnsureRuntimeUIIfMissing();
14     if (pauseMenuPanel == null) return;
15     pauseMenuPanel.SetActive(newState == GameState.Paused);
16 }

```

El GameManager desconoce cuántos ni cuáles componentes están suscritos, lo que permite añadir nuevos componentes reactivos (por ejemplo, un sistema de audio que reproduzca un efecto al completar una sala) sin modificar el código del gestor.

3.4.3 Patrón Strategy

Como se describió en el análisis de patrones, el patrón *Strategy* estructura el sistema de armas mediante subclases especializadas de Weapon. Cada tipo requiere campos específicos distintos (swingAngle para melee, bulletPrefab para ranged), lo que confirma la decisión de evitar condicionales internos:

```

1  // Weapon es una clase concreta con Attack() virtual (cuerpo vacío);
2  // las subclases lo sobrescriben con su propio comportamiento.
3  public class Weapon : MonoBehaviour {
4      public virtual void Attack() { } // no-op de forma predeterminada
5  }
6
7  public class MeleeWeapon : Weapon {
8      public override void Attack() {
9          // Ataque de barrido con hitbox
10     }
11 }
12
13 public class RangedWeapon : Weapon {
14     public override void Attack() {
15         // Disparo de proyectil
16     }
17 }

```

El PlayerController interactúa con cualquier arma con la interfaz Weapon.Attack(), sin necesidad de conocer el tipo concreto. Añadir un nuevo tipo de arma solo requiere crear una nueva subclase sin modificar el código existente.

3.4.4 Patrón Template Method

Como se describió en el análisis de patrones, el patrón *Template Method* estructura la jerarquía de enemigos. Sin este patrón, las alternativas serían: (a) tres clases independientes duplicando ~180 líneas de lógica común, o (b) una única clase de ~800 líneas con condicionales switch(enemyType):

```

1  public class Enemy : MonoBehaviour {
2      protected virtual void Start() {
3          // Inicializacion base: vida, fisica, gracia
4      }
5
6      protected virtual void Update() {
7          // Logica base: verificacion de gracia, efectos de tiles
8      }
9  }

```

```
9
10 protected virtual void Die() {
11     // Muerte base: notificación, animación, destrucción
12 }
13
14
15 public class SlimeGreen : Enemy {
16     protected override void Update() {
17         base.Update();
18         // Logica específica: evaluar distancia, disparar
19     }
20 }
```

La clase base `Enemy` define el esqueleto de comportamiento común (gestión de vida, detección de contacto, muerte, interacción con tiles), mientras que cada subclase especializa únicamente la inteligencia artificial, permitiendo añadir nuevos tipos de enemigo sin modificar las clases existentes.

3.4.5 Arquitectura basada en componentes (Entidad-Componente)

Además de los patrones clásicos, el análisis del estado del arte introdujo la arquitectura de componentes de Unity. Su aplicación en *Ascension* responde al siguiente problema:

Problema. Las entidades del juego (jugador, enemigos, proyectiles) requieren combinaciones variables de funcionalidades: un enemigo necesita física, renderizado y lógica de IA, pero no control de entrada; el jugador necesita control de entrada, física, vida y arma, pero no lógica de IA.

Alternativa descartada. Herencia múltiple de una jerarquía profunda (`Entity > MovableEntity > DamageableEntity > PlayerEntity`). Se descartó porque C# no soporta herencia múltiple y porque las jerarquías profundas producen clases base sobrecargadas con funcionalidad que no todas las subclases necesitan (el denominado *fragile base class problem*).

Aplicación en el proyecto: Toda la arquitectura del juego se fundamenta en la composición de componentes proporcionada por Unity.



Figura 3.9: Diagrama de la composición del jugador. *Elaboración propia.*

Justificación: Favorece la composición sobre la herencia. Cada componente encapsula una responsabilidad única (movimiento, vida, física, renderizado), lo que permite reutilizar componentes entre entidades distintas y modificar una funcionalidad sin afectar a las demás. Por ejemplo, el componente `PlayerHealth` podría reutilizarse en un futuro NPC aliado sin necesidad de heredar de `PlayerController`.

3.4.6 Máquina de estados (State Machine)

Como se introdujo en el análisis del estado del arte, el `GameManager` implementa una máquina de estados. La decisión de diseño responde al siguiente problema:

Problema. El flujo global del juego atraviesa múltiples fases (menú, selección de clase, partida, pausa, fin de partida) con transiciones condicionadas. Sin una estructura explícita, el código del GameManager tendería a acumular variables booleanas (`isPaused`, `isGameOver`, `isInMenu`) cuyas combinaciones serían difíciles de mantener y probar.

Alternativa descartada. Variables booleanas independientes para cada estado. Se descartó porque n booleanos producen 2^n combinaciones posibles, la mayoría de las cuales representan estados inválidos (por ejemplo, `isPaused == true` y `isGameOver == true` simultáneamente). Una enumeración con seis valores permite exactamente seis estados válidos, eliminando combinaciones ilógicas por construcción.

Aplicación en el proyecto: Control del flujo global del juego en GameManager.

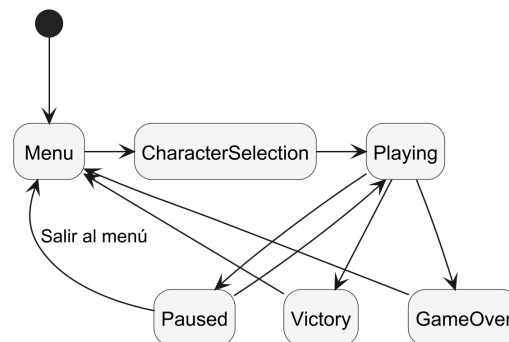


Figura 3.10: Diagrama de los estados del juego. *Elaboración propia.*

Justificación: Modela de forma explícita los estados válidos del juego y las transiciones permitidas entre ellos, reduciendo la probabilidad de estados no contemplados y facilitando la comprensión del flujo del juego.

3.4.7 Interacción entre patrones: análisis de flujos transversales

Los patrones descritos en las secciones anteriores no operan de forma aislada, sino que se combinan en cadenas de interacción que atraviesan múltiples subsistemas. A continuación se analizan tres flujos representativos que ilustran cómo la combinación de patrones produce un diseño cohesivo con bajo acoplamiento.

Flujo 1: Muerte de un enemigo (*Template Method* + *Observer* + *Singleton*)

Cuando un enemigo es eliminado, intervienen tres patrones en cadena. El patrón *Template Method* proporciona el método `Die()` en la clase base `Enemy`, que todas las subclases heredan sin sobrescribir. Este método invoca al `GameManager` (accesible globalmente mediante el patrón *Singleton*) a través del método `NotifyEnemyKilled()`. A su vez, el `GameManager` emite el evento `OnEnemyKilled` del patrón *Observer*, al que están suscritos los componentes de interfaz de usuario.

```

1 // (1) Template Method: Enemy.Die(), heredado por SlimeGreen, SlimeRed, SlimeBlue
2 protected virtual void Die() {
3     if (isDead) return;
4     isDead = true;
5
6     // (2) Singleton: acceso global al GameManager

```

```

7   if (GameManager.Instance != null) {
8       int scoreValue = enemyData != null ? enemyData.damage * 5 : 10;
9       GameManager.Instance.NotifyEnemyKilled(scoreValue);
10  }
11
12  if (animator != null && animator.runtimeAnimatorController != null && hasAnimDieTrigger)
13  {
14      animator.SetTrigger("Die");
15      Destroy(gameObject, 0.5f);
16  } else {
17      Destroy(gameObject);
18  }
19
20  // (3) Observer: GameManager emite evento que la UI recibe
21  public void NotifyEnemyKilled(int scoreValue = 10) {
22      enemiesKilled++;
23      ScoreManager.Instance?.Add(scoreValue); // Singleton: acceso al ScoreManager
24      OnEnemyKilled?.Invoke(enemiesKilled); // Observer: notifica a la UI
25  }

```

La cadena completa es: **Template Method** (el esqueleto de `Die()` se define una sola vez) -> **Singleton** (acceso al gestor central sin propagación de referencias) -> **Observer** (desacoplamiento entre la lógica de muerte y la interfaz de usuario). Si se hubiera implementado sin estos patrones, cada subclase de enemigo habría necesitado duplicar la lógica de muerte, mantener una referencia explícita al `GameManager` (pasada por constructor o asignada en el inspector) y conocer directamente los componentes de UI que deben actualizarse.

Flujo 2: Ataque del jugador (**Strategy + ScriptableObject + Singleton**)

Cuando el jugador ataca, intervienen el patrón *Strategy*, el sistema de *ScriptableObjects* y el patrón *Singleton*. El `PlayerController` invoca `weapon.Attack()` a través de la interfaz común de la clase base `Weapon (Strategy)`, sin conocer si el arma es cuerpo a cuerpo o a distancia. La implementación concreta (definida en `MeleeWeapon` o `RangedWeapon`) consulta los parámetros del *ScriptableObject* `WeaponData` para obtener el daño base y, en el caso de las armas a distancia, el prefab del proyectil. El proyectil, a su vez, consulta al `DamageBoost Manager (Singleton)` para aplicar las bonificaciones de daño de los tiles de potenciación.

```

1  // (1) Strategy: el PlayerController no distingue entre tipos de arma
2  weapon.Attack(); // Polimorfismo: ejecuta MeleeWeapon.Attack() o RangedWeapon.Attack()
3
4  // (2) ScriptableObject: el arma consulta sus parametros de un activo externo
5  public override void Attack() {
6      // ...
7      projectileScript.damage = weaponData.damage; // Dato del ScriptableObject
8  }
9
10 // (3) Singleton: el proyectil consulta al gestor de bonificacion
11 void Start() {
12     damage = DamageBoostManager.Instance.GetBoostedDamage(damage); // Singleton
13 }

```

Flujo 3: Cambio de estado del juego (**State Machine + Observer + Singleton**)

La transición de estado del juego ejemplifica la interacción entre la máquina de estados, el patrón *Observer* y el patrón *Singleton*. Cuando el jugador pulsa la tecla de pausa, el `Pause Menu` accede al `GameManager` mediante el *Singleton* y solicita un cambio de estado. La máquina de estados del `GameManager` procesa la transición (modifica `Time.timeScale`, actualiza la

variable `currentState`) y emite el evento `OnGameStateChanged` del patrón *Observer*. Todos los componentes suscritos (incluido el propio `PauseMenu` que originó la petición) reaccionan al evento de forma independiente.

```
1 // (1) Singleton: acceso al gestor central desde cualquier componente
2 GameManager.Instance.ChangeState(GameState.Paused);
3
4 // (2) State Machine: procesamiento de la transición
5 public void ChangeState(GameState newState) {
6     currentState = newState;
7     Time.timeScale = newState == GameState.Playing ? 1f : 0f;
8
9     // (3) Observer: notificación a todos los suscriptores
10    OnGameStateChanged?.Invoke(newState);
11 }
12
13 // (4) Suscriptores independientes reaccionan sin acoplamiento
14 // PauseMenu:
15 private void OnGameStateChanged(GameState state) {
16     EnsureRuntimeUIIfMissing();
17     if (pauseMenuPanel == null) return;
18     pauseMenuPanel.SetActive(state == GameState.Paused);
19 }
20 // GameOverScreen:
21 private void OnGameStateChanged(GameState state) {
22     gameOverPanel.SetActive(state == GameState.GameOver);
23 }
```

Esta interacción demuestra una propiedad arquitectónica: el emisor del evento (`GameManager`) no conoce ni la cantidad ni la identidad de los suscriptores, lo que permite añadir nuevos componentes reactivos (por ejemplo, un futuro sistema de audio que silencie la música al pausar) sin modificar el código del `GameManager` ni de los suscriptores existentes.

4 DESARROLLO DE ASCENSION

El presente capítulo constituye el núcleo técnico de la memoria y describe la implementación detallada de cada subsistema del videojuego *Ascension*. Partiendo del diseño presentado previamente (requisitos, arquitectura y patrones de diseño), se documenta cada módulo con sus correspondientes fragmentos de código, explicaciones línea por línea y diagramas de interacción. Al final del capítulo se describen dos sistemas desarrollados pero pendientes de integración completa en el flujo de juego.

El desarrollo de cada subsistema siguió la metodología iterativa e incremental descrita en la introducción. Las secciones de este capítulo presentan el resultado final de cada módulo tras las iteraciones realizadas; al inicio de cada sección se incluye un breve párrafo de **evolución iterativa** que documenta los cambios de diseño más relevantes que se produjeron durante el proceso de desarrollo, proporcionando trazabilidad entre la metodología adoptada y las decisiones técnicas finales.

4.1 SISTEMA DE GESTIÓN CENTRAL

Evolución iterativa. El sistema de gestión central atravesó tres iteraciones significativas. En la primera versión, el `GameManager` gestionaba directamente la generación de salas, la aparición de enemigos y la puntuación en una única clase monolítica de más de 500 líneas. La validación funcional reveló que esta concentración de responsabilidades dificultaba la depuración (un error en la generación de salas afectaba a la lógica de puntuación) y violaba el principio de responsabilidad única. En la segunda iteración se extrajeron el `RoomFlow Controller`, el `ScoreManager` y el `DamageBoostManager` como gestores independientes, cada uno con su propio ciclo de vida *Singleton*. La tercera iteración introdujo el sistema de eventos (`OnGameStateChanged`, `OnRoomCleared`, `OnEnemyKilled`) para desacoplar la comunicación entre gestores: en la versión anterior, el `GameManager` invocaba métodos concretos de cada componente de UI, lo que obligaba a modificar el gestor central cada vez que se añadía un nuevo elemento a la interfaz.

4.1.1 GameManager: controlador principal del juego

El `GameManager` constituye el componente central del sistema. Implementa el patrón *Singleton* con el objetivo de mantener una única instancia, y gestiona el ciclo de vida completo de una partida mediante una máquina de estados finitos (FSM). Sus responsabilidades incluyen el control de estados del juego, la gestión de progreso (salas completadas, enemigos eliminados), las transiciones entre escenas y la coordinación de los subsistemas.

Máquina de estados. El flujo del juego se modela mediante una enumeración que define los estados válidos:

```
1 public enum GameState {  
2     Menu,  
3     CharacterSelection ,  
4     Playing ,  
5     Paused ,  
6     GameOver ,  
7     Victory  
8 }
```

Las transiciones entre estados se gestionan mediante un método centralizado que modifica el estado actual, ajusta la escala de tiempo del juego y notifica a los observadores suscritos:

```
1 public void ChangeState(GameState newState) {  
2     if (currentState == newState) return;    // Evita transiciones redundantes  
3  
4     currentState = newState;  
5  
6     switch (newState) {  
7         case GameState.Playing :  
8             Time.timeScale = 1f;                // Tiempo normal: fisica y animaciones activas  
9             isPaused = false;  
10            break;  
11        case GameState.Paused :  
12            Time.timeScale = 0f;                // Detiene la simulacion completa  
13            isPaused = true;  
14            break;  
15        case GameState.GameOver :  
16        case GameState.Victory :  
17            Time.timeScale = 0f;                // Congela el juego tras el resultado  
18            break;  
19    }  
20  
21    OnGameStateChanged?.Invoke(newState);    // Notifica a todos los suscriptores  
22 }
```

Línea por línea:

- `if (currentState == newState) return;` previene notificaciones duplicadas si un componente solicita transitar al estado en el que ya se encuentra el juego, lo que evitaría efectos colaterales como reactivar paneles de UI que ya están visibles.
- `Time.timeScale = 0f;` detiene toda la simulación dependiente del tiempo escalado (física del `Rigidbody2D`, animaciones del `Animator`, corutinas que usen `WaitForSeconds`). Se seleccionó `Time.timeScale` frente a la alternativa de desactivar componentes individuales porque proporciona una pausa global con una sola asignación, sin necesidad de iterar sobre todos los objetos activos de la escena.
- `OnGameStateChanged?.Invoke(newState);` emite el evento del patrón *Observer*. El operador `?.` evita una excepción si ningún componente está suscrito al evento. Los componentes de UI (`PauseMenu`, `GameOverScreen`) se suscriben a este evento en su método `Start()` y reaccionan automáticamente al cambio de estado.

Figura 4.1 - Máquina de estados del GameManager

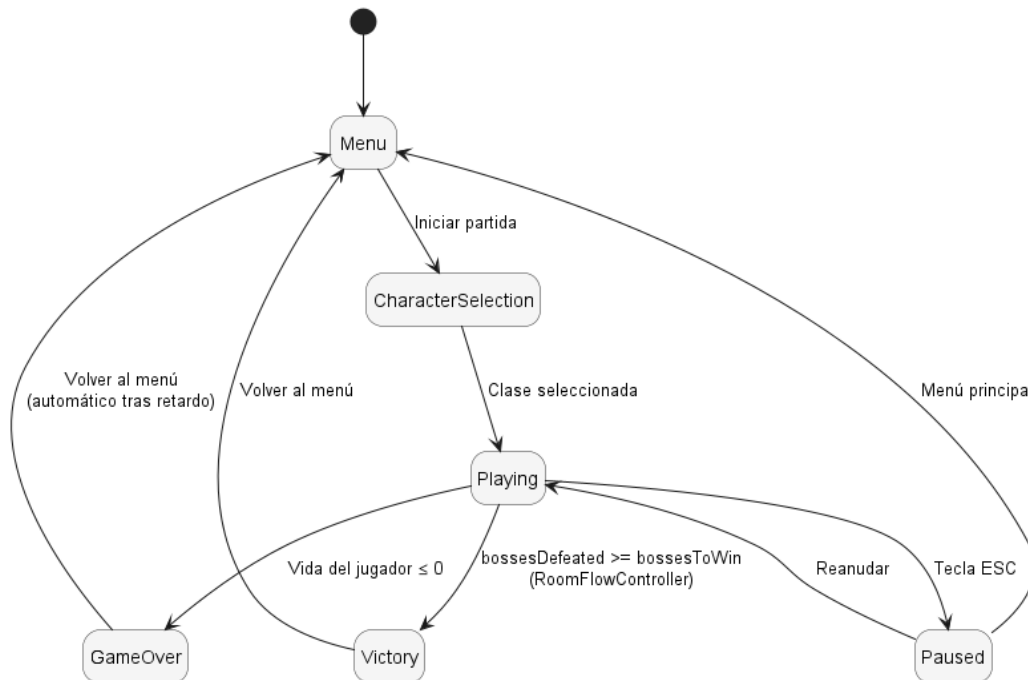


Figura 4.1: Diagrama de estados UML del GameManager. *Elaboración propia.*

Patrón Singleton. La inicialización del *Singleton* se realiza en el método `Awake()`, con el objetivo de que solo exista una instancia del gestor y que esta persista entre cambios de escena:

```

1 void Awake() {
2     if (Instance == null) {
3         Instance = this;
4
5         if (ScoreManager.Instance == null) {
6             new GameObject("ScoreManager").AddComponent<ScoreManager>();
7         }
8         DontDestroyOnLoad(gameObject);
9     } else {
10        Destroy(gameObject);
11        return;
12    }
13 }

```

La comprobación `if (Instance == null)` previene la creación de instancias duplicadas cuando se vuelve a cargar una escena que contenga un `GameManager` en su jerarquía. Adicionalmente, si no existe un `ScoreManager` activo, el `GameManager` lo crea de forma automática, de forma que el sistema de puntuación esté disponible. La llamada a `DontDestroyOnLoad` se realiza al final del bloque para que tanto el `GameManager` como el `ScoreManager` recién creado persistan entre cambios de escena. Si ya existe una instancia, el duplicado se destruye y se retorna inmediatamente con `return`.

Sistema de eventos. El `GameManager` expone tres eventos principales que implementan el patrón *Observer*:

Evento	Momento de emisión	Suscriptores típicos
OnGameStateChanged	Al cambiar el estado del juego	PauseMenu, pantallas de fin de partida, sistemas de UI
OnRoomCleared	Al completar una sala	RoomFlowController, Score Display
OnEnemyKilled	Al eliminar un enemigo	ScoreManager, contadores de UI

Tabla 4.1: Eventos del GameManager.

Inicio de partida y coordinación entre gestores. El método `StartNewRun()` constituye el punto de entrada de cada nueva partida y ejemplifica la coordinación entre los cuatro gestores *Singleton* del sistema. Al iniciar una ejecución, el GameManager reinicia sus propios contadores y delega el reinicio de cada subsistema en el gestor correspondiente:

```

1 public void StartNewRun() {
2     currentLevel = 1;
3     roomsCleared = 0;
4     enemiesKilled = 0;
5
6     RoomFlowController.Instance?.ResetRunProgress(); // Reinicia profundidad de sala y
7     // secuencia de jefes
8     ScoreManager.Instance?.ResetScore(); // Pone la puntuacion a cero
9     DamageBoostManager.Instance?.ResetBoost(); // Elimina bonificaciones de dano
10    // residuales
11
12    ChangeState(GameState.Playing); // Transita a Playing -> Time.
13    timeScale = 1
14 }

```

La secuencia de reinicio sigue un orden específico: primero se reinician los datos del GameManager, después los de cada gestor dependiente y, finalmente, se cambia el estado a Playing. Este orden garantiza que los subsistemas estén en un estado limpio *antes* de que la transición de estado notifique a los suscriptores del evento `OnGameStateChanged`, incluida la interfaz de usuario, que al recibir la notificación de estado Playing consulta los contadores, que ya deben estar a cero. Si el cambio de estado se realizase antes de los reinicios, los componentes de UI mostrarían brevemente los datos de la partida anterior.

El siguiente diagrama resume la cadena de coordinación entre gestores al iniciar una nueva partida:

4.1. SISTEMA DE GESTIÓN CENTRAL

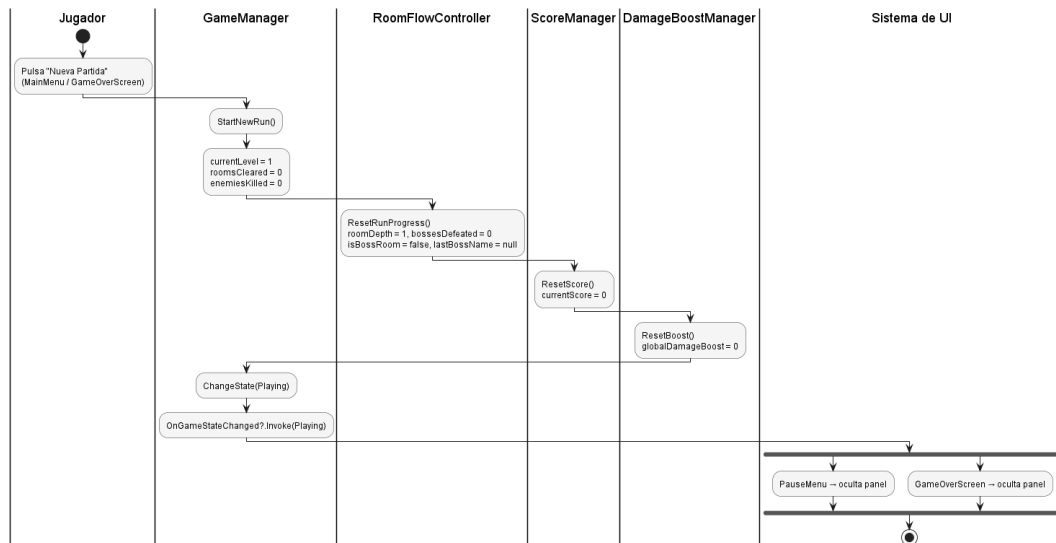


Figura 4.2: Diagrama de la coordinación entre gestores al iniciar una nueva partida. *Elaboración propia.*

Sistema de progresión. La puntuación se calcula según las siguientes reglas: 100 puntos por cada sala completada, una cantidad variable por cada enemigo eliminado (calculada como el daño base del enemigo multiplicado por 5), y una bonificación de 1.000 puntos en caso de completar el juego con victoria.

Notificación de sala completada y eliminación de enemigo. Los métodos `NotifyRoomCleared()` y `NotifyEnemyKilled()` completan el ciclo de coordinación entre el `GameManager` y los subsistemas dependientes:

```

1 public void NotifyRoomCleared() {
2     roomsCleared++;
3     ScoreManager.Instance?.Add(100);           // 100 puntos por sala
4     OnRoomCleared?.Invoke(roomsCleared);       // Notifica a RoomFlowController y UI
5 }
6
7 public void NotifyEnemyKilled(int scoreValue = 10) {
8     enemiesKilled++;
9     ScoreManager.Instance?.Add(scoreValue);    // Puntos = dano del enemigo 5
10    OnEnemyKilled?.Invoke(enemiesKilled);      // Actualiza contadores de UI
11 }

```

El flujo completo de una sala (desde la aparición de enemigos hasta la transición a la siguiente sala) involucra la siguiente cadena de invocaciones entre gestores:

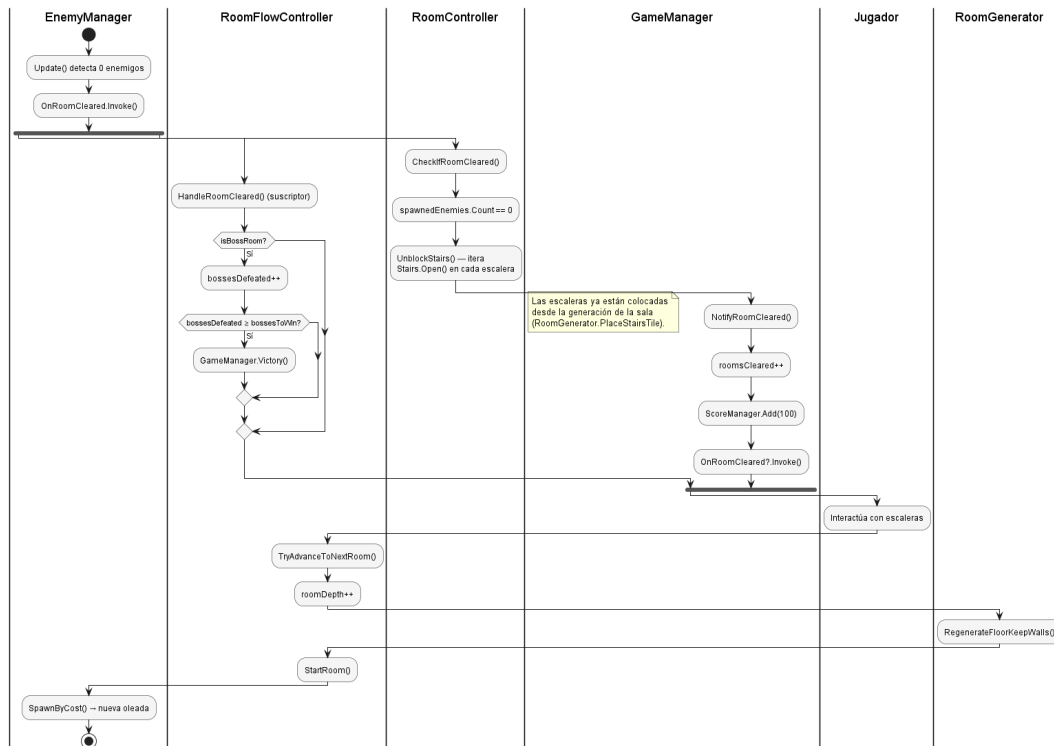


Figura 4.3: Diagrama del flujo de completar una sala. *Elaboración propia.*



Figura 4.4: Captura de pantalla del menú principal de *Ascension*, mostrando las opciones de nueva partida, tabla de puntuaciones y salir. *Elaboración propia.*

4.1.2 RoomFlowController: control de progresión de salas

El RoomFlowController gestiona el bucle central de generación y progresión de salas. Su ciclo de funcionamiento se describe mediante el siguiente diagrama:

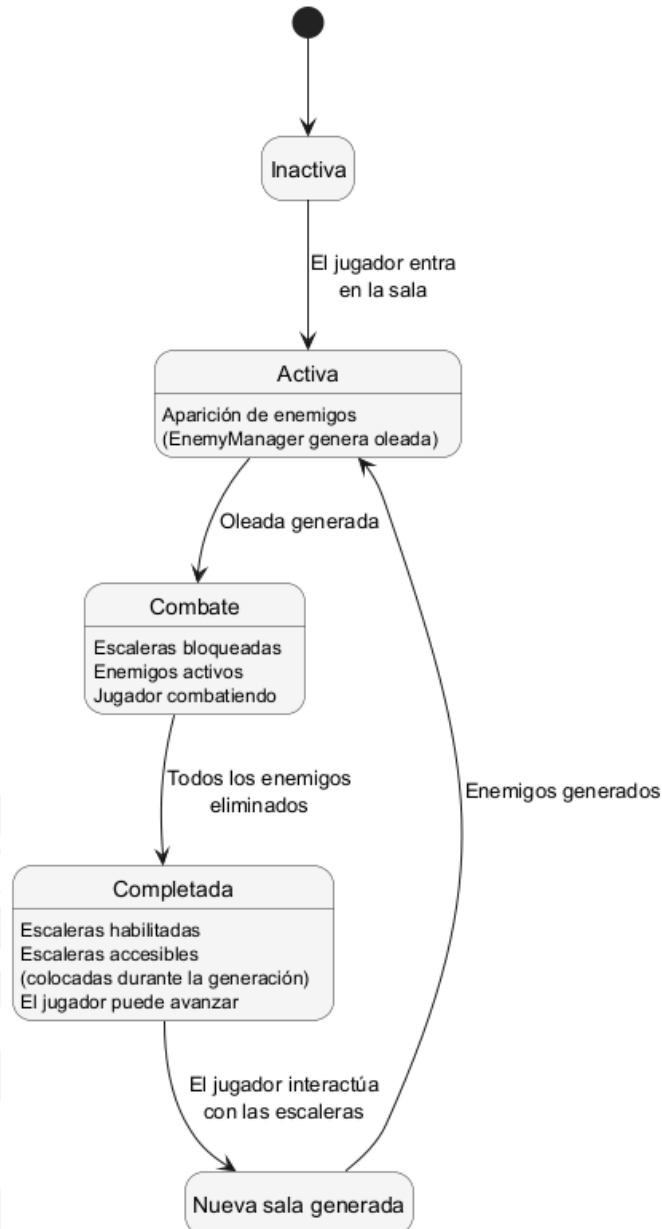


Figura 4.5: Diagrama del ciclo de funcionamiento de generación de salas. *Elaboración propia.*

Sistema de jefes periódicos. Cada cinco salas completadas, la siguiente sala contiene un enemigo jefe en lugar de una oleada estándar. En este proyecto, un “jefe” no introduce una IA completamente nueva, sino que se define como una **variante del enemigo base con atributos incrementados** (*escalado*): mayor tamaño visual (multiplicador de escala de 2,4×)

y mayor resistencia (multiplicador de vida de ocho veces). Este enfoque permite introducir picos de dificultad de forma controlada sin añadir un subsistema adicional de IA de jefes.

- Jefe 1: Slime verde (francotirador) escalado
- Jefe 2: Slime rojo (perseguidor) escalado
- Jefe 3: Slime azul (saltador) escalado

4.1.3 ScoreManager: sistema de puntuación y persistencia

El ScoreManager implementa la persistencia de datos de partidas mediante PlayerPrefs y serialización JSON. PlayerPrefs es un mecanismo de almacenamiento local simple (pares clave–valor) proporcionado por Unity, adecuado para guardar preferencias y pequeños volúmenes de datos. La serialización a JSON convierte las estructuras del historial en texto estructurado para poder almacenarlas y recuperarlas de forma consistente entre ejecuciones del juego. Cada entrada del historial almacena la siguiente información:

```
1 public class ScoreEntry {  
2     public long utcTicks;           // Marca de tiempo  
3     public int score;               // Puntuacion final  
4     public int roomsCleared;        // Salas completadas  
5     public int enemiesKilled;       // Enemigos eliminados  
6     public string playerClass;      // Clase utilizada  
7     public string result;           // "Victory" o "GameOver"  
8 }
```

El historial se limita a un máximo de 30 entradas, realizando limpieza automática de las entradas más antiguas cuando se alcanza este límite. Los datos se persisten entre sesiones de juego mediante su serialización a formato JSON y su almacenamiento en PlayerPrefs.

4.2 SISTEMA DEL JUGADOR

Evolución iterativa. El sistema del jugador experimentó ajustes relevantes durante el desarrollo. La primera implementación del movimiento utilizaba `GetAxis` (con suavizado) en lugar de `GetAxisRaw`, lo que generaba una latencia perceptible entre la pulsación de tecla y el inicio del movimiento; las sesiones de juego revelaron que esta latencia era inadecuada para un *roguelike* de acción, y se sustituyó por `GetAxisRaw`. El sistema de esquiva (*roll*) se incorporó en una iteración posterior a la implementación del sistema de enemigos: las primeras pruebas de combate demostraron que el jugador carecía de opciones defensivas activas, ya que la única forma de evitar daño era el movimiento. La invulnerabilidad post-daño (parpadeo a 12 Hz) se añadió tras detectar que los enemigos podían infligir varios impactos en fotogramas consecutivos, resultando en muertes instantáneas que el jugador percibía como injustas.

4.2.1 PlayerController: control de movimiento

El PlayerController gestiona el movimiento del personaje, la habilidad de esquiva y la inicialización basada en la clase seleccionada. El sistema de movimiento soporta ocho

direcciones y normaliza el vector resultante para evitar que el movimiento diagonal sea más rápido que el movimiento en ejes cardinales:

```
1 void Update() {
2     if (isRolling) return;
3     float hor = Input.GetAxisRaw("Horizontal");
4     float ver = Input.GetAxisRaw("Vertical");
5     velocity = new Vector3(hor, ver, 0).normalized * speed;
6     if (canRoll && Input.GetKeyDown(rollKey)) {
7         Vector2 rollDir = new Vector2(hor, ver).normalized;
8         StartCoroutine(DoRoll(rollDir));
9     }
10 }
11
12 void FixedUpdate() {
13     if (!isRolling && rigidbody != null) {
14         rigidbody.MovePosition(rigidbody.position + (Vector2)(velocity * Time.fixedDeltaTime)
15     );
16 }
```

Línea por línea:

- **Línea 1:** Si el jugador está ejecutando una acción evasiva, se ignora toda la entrada de movimiento; el *roll* tiene su propia lógica de velocidad gestionada por la corutina.
- **Líneas 2-3:** `GetAxisRaw` devuelve -1, 0 o 1 sin suavizado de transición, lo que produce un movimiento inmediato y reactivo. Se seleccionó `GetAxisRaw` frente a `GetAxis` porque este último aplica un suavizado temporal que introduce latencia perceptible entre la pulsación de tecla y el movimiento, inadecuado para un *roguelike* de acción en tiempo real donde la precisión de movimiento es crítica.
- **Línea 4:** `.normalized` convierte el vector a magnitud 1 antes de multiplicar por `speed`. Esta normalización resuelve un problema clásico de los videojuegos 2D: sin ella, el movimiento diagonal (por ejemplo, W+D) tendría magnitud $\sqrt{2} \approx 1,41$, lo que haría al personaje un 41 % más rápido en diagonal que en ejes cardinales.
- **Líneas 5-7:** La esquiva solo se activa si el jugador puede esquivar (`canRoll`) y si la tecla se pulsa (`GetKeyDown`, no `GetKey`). La dirección del *roll* se calcula a partir de la entrada actual, y se inicia la corutina `DoRoll(rollDir)` que gestiona la velocidad, la invulnerabilidad y la travesía de colisiones.
- **Líneas 8-10:** La aplicación de movimiento se realiza en `FixedUpdate()` mediante `MovePosition`, que desplaza al jugador basándose en la velocidad calculada multiplicada por `fixedDeltaTime`. Se utiliza `MovePosition` en lugar de asignar `linearVelocity` porque este método respeta la interpolación del `Rigidbody2D` y produce un movimiento más suave. `FixedUpdate()` se ejecuta a una frecuencia fija (50 Hz de forma predeterminada) independiente de la tasa de fotogramas.

Inicialización basada en clase. Al instanciar al jugador, el `PlayerController` recibe un `ScriptableObject` de tipo `PlayerClass` que define las estadísticas iniciales del personaje y el arma con la que comienza la partida:

```

1 public void Initialize() {
2     baseSpeed = playerClass.speed;
3     speed = baseSpeed;
4
5     // Instanciar arma inicial
6     GameObject weaponInstance = Instantiate(
7         playerClass.startingWeaponData.weaponPrefab,
8         transform.position,
9         Quaternion.identity,
10        transform
11    );
12
13    // Inicializar script del arma
14    Weapon weaponScript = weaponInstance.GetComponent<Weapon>();
15    weaponScript.weaponData = playerClass.startingWeaponData;
16    weaponScript.Initialize();
17 }

```

Este diseño permite añadir nuevas clases de personaje sin modificar el código del *controller*, ya que toda la configuración se define en activos de tipo *ScriptableObject* editables desde el inspector de Unity.

Sistema de esquiva (roll). La mecánica de esquiva se implementa mediante una corutina que aplica un multiplicador de velocidad temporal, activa la invulnerabilidad del jugador y desactiva las colisiones con enemigos y proyectiles durante un periodo configurable. Al finalizar la esquiva, el sistema restaura los valores originales de velocidad, vulnerabilidad y colisiones:

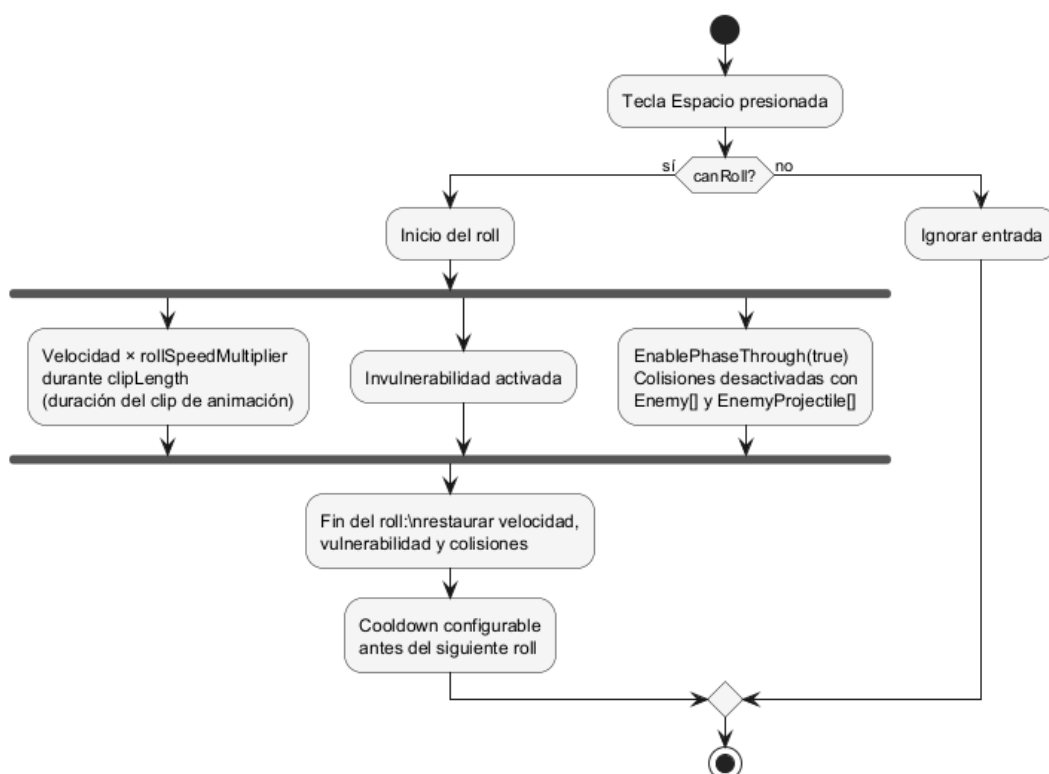


Figura 4.6: Diagrama del funcionamiento de la esquiva. *Elaboración propia.*

Travesía de colisiones (*phase-through*). Durante la esquivia, el jugador atraviesa a los enemigos y sus proyectiles. El método `EnablePhaseThrough` utiliza `Physics2D.IgnoreCollision` de Unity para desactivar selectivamente las colisiones entre el colisionador del jugador y los de cada enemigo y proyectil enemigo activos en la escena:

```

1 private void EnablePhaseThrough(bool enable) {
2     Collider2D playerCollider = GetComponent<Collider2D>();
3     if (playerCollider == null) return;
4
5     Enemy[] enemies = FindObjectsByType<Enemy>(FindObjectsSortMode.None);
6     foreach (Enemy enemy in enemies) {
7         if (enemy != null) {
8             Collider2D enemyCollider = enemy.GetComponent<Collider2D>();
9             if (enemyCollider != null)
10                Physics2D.IgnoreCollision(playerCollider, enemyCollider, enable);
11        }
12    }
13
14    EnemyProjectile[] projectiles = FindObjectsByType<EnemyProjectile>(FindObjectsSortMode.
15    None);
16    foreach (EnemyProjectile proj in projectiles) {
17        if (proj != null) {
18            Collider2D projCollider = proj.GetComponent<Collider2D>();
19            if (projCollider != null)
20                Physics2D.IgnoreCollision(playerCollider, projCollider, enable);
21        }
22    }
23 }

```

Se seleccionó `Physics2D.IgnoreCollision` frente a la alternativa de cambiar la capa del jugador (*layer*) durante la esquivia, porque el cambio de capa afectaría a todas las interacciones del jugador (incluidas paredes y tiles), mientras que `IgnoreCollision` permite desactivar selectivamente solo las colisiones con enemigos y proyectiles, manteniendo intactas las colisiones con el entorno.

4.2.2 PlayerHealth: sistema de vida

El `PlayerHealth` gestiona los puntos de vida del jugador, el sistema de invulnerabilidad temporal tras recibir daño y la detección de muerte. El método principal de recepción de daño ilustra el flujo completo:

```

1 public void TakeDamage(int amount) {
2     if (isInvulnerable || (playerController != null && playerController.IsInvulnerable()))
3         return;
4
5     currentHealth -= amount;
6     currentHealth = Mathf.Clamp(currentHealth, 0, maxHealth);
7
8     UpdateHearts();
9
10    if (currentHealth <= 0) {
11        Die();
12    } else {
13        if (damageCoroutine != null)
14            StopCoroutine(damageCoroutine);
15        damageCoroutine = StartCoroutine(DamageInvulnerabilityCoroutine());
16    }
17 }

```

La verificación inicial comprueba si el jugador se encuentra en estado de invulnerabilidad, ya sea por haber recibido daño previamente (invulnerabilidad temporal) o por estar ejecutando una esquiva; la comprobación `playerController != null` previene errores si la referencia aún no está inicializada. La función `Mathf.Clamp` evita que la vida descienda por debajo de cero o exceda el máximo. Tras actualizar la representación visual de los corazones, se evalúa si el jugador ha muerto; en caso contrario, se cancela cualquier corutina de parpadeo en curso (`damageCoroutine`) antes de iniciar una nueva, evitando conflictos visuales cuando el jugador recibe impactos consecutivos.

Efecto visual de parpadeo. Durante el periodo de invulnerabilidad, el sprite del jugador parpadea alternando entre su color original y blanco, y opcionalmente alternando su visibilidad, a una frecuencia configurable (de forma predeterminada, doce Hz). Si existe una corutina de parpadeo anterior en curso, se cancela antes de iniciar la nueva para evitar conflictos:

```

1 private IEnumerator DamageInvulnerabilityCoroutine() {
2     isInvulnerable = true;
3
4     if (spriteRenderers == null || spriteRenderers.Length == 0)
5         CacheSpriteRenderers();
6
7     float end = Time.time + Mathf.Max(0f, invulnerableTime);
8     float blinkPeriod = blinkFrequencyHz > 0f ? (1f / blinkFrequencyHz) : 0.1f;
9     float nextToggle = 0f;
10    bool white = true;
11    bool visible = true;
12
13    while (Time.time < end) {
14        if (Time.time >= nextToggle) {
15            nextToggle = Time.time + blinkPeriod;
16            white = !white;
17            visible = !visible;
18        }
19
20        for (int i = 0; i < spriteRenderers.Length; i++) {
21            var sr = spriteRenderers[i];
22            if (sr == null) continue;
23            sr.enabled = visibilityBlinkOnDamage ? visible : true;
24            sr.color = white ? Color.white : originalColors[i];
25        }
26
27        yield return null;
28    }
29
30    for (int i = 0; i < spriteRenderers.Length; i++) {
31        if (spriteRenderers[i] == null) continue;
32        spriteRenderers[i].color = originalColors[i];
33        spriteRenderers[i].enabled = true;
34    }
35
36    isInvulnerable = false;
37    damageCoroutine = null;
38 }

```

La instrucción `yield return null` cede la ejecución hasta el siguiente fotograma sin bloquear el hilo principal del juego. El final del efecto se controla comparando `Time.time` (tiempo global del juego) con el instante `end`, lo que permite que la duración de la invulnerabilidad sea consistente independientemente de la tasa de fotogramas.

El cambio de color y visibilidad se aplica a todos los `SpriteRenderer` del jugador (incluidos los de sus objetos hijos, almacenados en caché por `CacheSpriteRenderers()`) para que el parpadeo afecte al personaje completo. La variable `visibilityBlinkOnDamage` permite alternar no solo el color, sino también la visibilidad (`sr.enabled`) de cada renderer, produciendo un efecto de parpadeo más notorio. Al finalizar la corutina, se restablece el estado original de todos los renderers y se marca `damageCoroutine = null` para permitir la gestión correcta de corutinas superpuestas.

Diagrama del ciclo de vida del jugador

El ciclo de vida del jugador integra los subsistemas de movimiento (`PlayerController`), vida (`PlayerHealth`) y esquiva, atravesando los siguientes estados durante una partida:

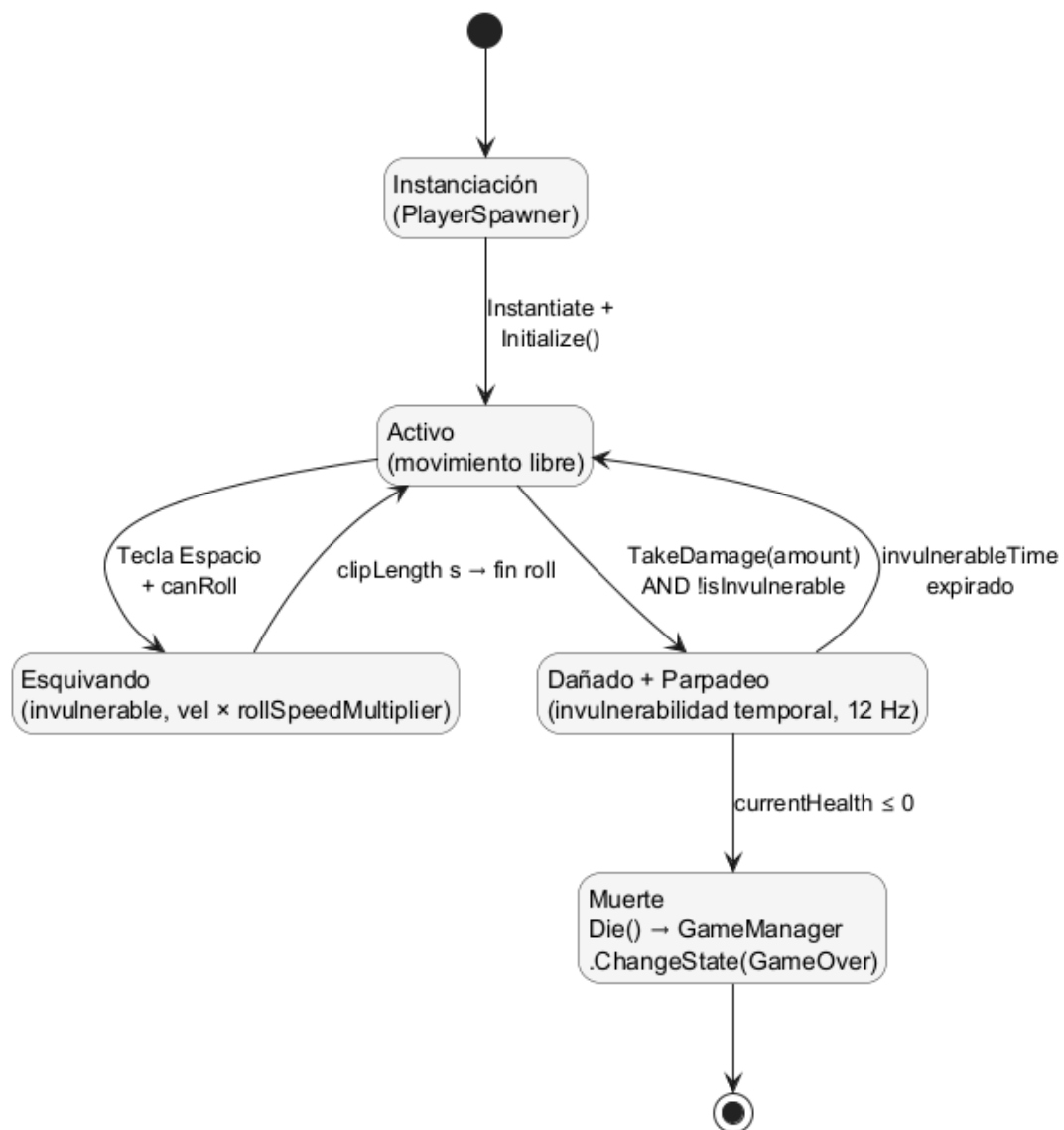


Figura 4.7: Ciclo de vida del jugador. *Elaboración propia.*

Este diagrama muestra que el jugador solo puede recibir daño en el estado **Activo** (ni durante la esquivas ni durante la invulnerabilidad post-daño), lo que proporciona dos mecanismos de protección distintos: uno proactivo (esquivas voluntaria) y otro reactivo (invulnerabilidad automática tras daño). Cuando la vida alcanza cero, se ejecuta un parpadeo rojo rápido (cinco alternancias rojo/blanco a 0,08 s por cambio, duración total de 0,8 s) que proporciona retroalimentación visual de muerte antes de notificar al `GameManager` para que transite al estado `GameOver`, ejemplificando la interacción *Observer + State Machine* descrita en el capítulo de diseño.

4.2.3 PlayerSpawner: instanciación del jugador

El `PlayerSpawner` se encarga de instanciar al jugador al inicio de la escena de juego, aplicando la clase seleccionada en la escena anterior:

```
1 void Start() {  
2     if (ClassSelectionManager.SelectedClass == null) {  
3         Debug.LogError("No se ha seleccionado ninguna clase.");  
4         SceneManager.LoadScene("ClassSelection");  
5         return;  
6     }  
7  
8     GameObject player = Instantiate(  
9         playerPrefab,  
10        spawnPoint.position,  
11        Quaternion.identity  
12    );  
13  
14    var controller = player.GetComponent<PlayerController>();  
15    controller.playerClass = ClassSelectionManager.SelectedClass;  
16    controller.Initialize();  
17 }
```

Si el jugador accede a la escena de juego sin haber seleccionado una clase (por ejemplo, durante la depuración), el sistema detecta esta situación y redirige automáticamente a la escena de selección.

4.2.4 PlayerClass: datos de configuración mediante *ScriptableObjects*

Los datos de cada clase de personaje se definen mediante un *ScriptableObject*. Se seleccionó este mecanismo frente a dos alternativas: (a) codificar las estadísticas directamente en variables del `PlayerController`, lo que obligaría a duplicar y recompilar el script para cada combinación de parámetros; y (b) cargar los datos desde un fichero externo (JSON o XML), lo que añadiría lógica de lectura, deserialización y validación sin integración con el inspector de Unity.

El *ScriptableObject* resuelve ambos problemas: es un activo nativo del proyecto que se edita directamente desde el inspector, se serializa automáticamente con el proyecto, y se vincula al `PlayerController` mediante una referencia en el campo `playerClass`. Esta decisión tiene una implicación arquitectónica directa: para añadir una nueva clase de personaje, basta con crear un nuevo activo de tipo `PlayerClass` desde el menú del editor (Roguelike/Player Class), configurar sus estadísticas y asignarle un arma inicial, sin modificar una sola línea del código del `PlayerController`.



4.3. SISTEMA DE ENEMIGOS

```
1 [CreateAssetMenu(fileName = "PlayerClass", menuName = "Roguelike/PlayerClass")]
2 public class PlayerClass : ScriptableObject {
3     public string className;           // Nombre visible de la clase (p. ej., "Guerrero")
4     public int maxHealth = 5;          // Puntos de vida maximos
5     public float speed = 32f;          // Velocidad base en unidades/segundo
6     public WeaponData startingWeaponData; // Referencia al arma inicial (otro
7     ScriptableObject)
```

Línea por línea:

- `[CreateAssetMenu(...)]`: atributo de Unity que añade una entrada al menú contextual del editor, permitiendo crear instancias de este *ScriptableObject* directamente desde Assets > Create > Roguelike > PlayerClass.
- `className`: cadena que identifica la clase en la interfaz de usuario (pantalla de selección y estadísticas finales).
- `maxHealth = 5`: valor predeterminado que diferencia las clases; el guerrero podría tener 7 y el mago 3.
- `speed = 32f`: velocidad base en unidades del mundo por segundo; a 16 PPU (píxeles por unidad), equivale a 512 píxeles por segundo.
- `startingWeaponData`: referencia a otro *ScriptableObject* (*WeaponData*) que define el arma con la que la clase comienza la partida. Esta composición de *ScriptableObjects* permite que la misma arma pueda reutilizarse en múltiples clases sin duplicar datos.

4.3 SISTEMA DE ENEMIGOS

Evolución iterativa. El sistema de enemigos fue el subsistema que más iteraciones requirió. Inicialmente, cada tipo de enemigo se implementó como una clase independiente sin herencia compartida, lo que provocó duplicación de la lógica de vida, daño y muerte en tres archivos. La refactorización hacia la jerarquía con *Template Method* (clase base *Enemy* + subclases) se realizó en la cuarta iteración del módulo. El sistema de periodos de gracia (`contactDamageGraceSeconds`, `aiGraceSeconds`) se descubrió durante las pruebas de juego: sin este mecanismo, los enemigos dañaban al jugador en el fotograma de aparición. El limitador de velocidad (`CapToPlayerSpeed`) surgió de la misma forma: el *SlimeGreen* en estado de huida se movía más rápido que el jugador, haciendo imposible la persecución. Ambos mecanismos se integraron en la clase base *Enemy* para que todos los tipos de enemigo los heredasen automáticamente.

4.3.1 Enemy: clase base y patrón *Template Method*

La clase *Enemy* define el comportamiento base compartido por todos los tipos de enemigo del juego. Se seleccionó el patrón *Template Method* frente a la alternativa de implementar cada enemigo como una clase independiente sin herencia. Esta alternativa se descartó porque los tres tipos de enemigo comparten un núcleo de funcionalidad significativo: gestión de puntos de vida (recepción de daño, muerte), periodos de gracia tras la aparición, interacción con

tiles de velocidad, detección de contacto con el jugador y notificación al `GameManager` al morir. Duplicar esta lógica en tres clases independientes habría generado aproximadamente 180 líneas de código redundante y habría introducido riesgo de inconsistencias: si se modificase la lógica de daño en un tipo de enemigo pero no en los demás, se producirían discrepancias difíciles de detectar.

El *Template Method* resuelve este problema: la clase base `Enemy` define el esqueleto de comportamiento común, y cada subclase sobrescribe exclusivamente el método `Update()` para definir su IA específica.

Sistema de gracia al aparecer. Para evitar que los enemigos ataquen al jugador inmediatamente tras aparecer (lo que resultaría en una experiencia injusta), se implementa un sistema de periodos de gracia:

```
1 [SerializeField] private float contactDamageGraceSeconds = 0.75f;
2 [SerializeField] private float aiGraceSeconds = 0.75f;
3
4 protected bool IsAIEnabled => Time.time >= aiEnabledTime;
```

En este contexto, `Time.time` representa el tiempo transcurrido desde el inicio de la ejecución del juego. La IA se habilita cuando dicho tiempo supera un instante programado (`aiEnabledTime`), de modo que el enemigo “espera” un breve intervalo antes de comenzar a actuar.

Durante los primeros 0,75 segundos tras su aparición, el enemigo no puede infligir daño por contacto ni ejecutar su lógica de inteligencia artificial. Este intervalo proporciona al jugador una ventana de reacción justa.

Interacción con tiles de velocidad. Los enemigos, al igual que el jugador, se ven afectados por los tiles de velocidad del suelo:

```
1 public void SetTileSpeedMultiplier(float multiplier) {
2     tileSpeedMultiplier = Mathf.Clamp(multiplier, 0.1f, 5f);
3 }
```

Esta mecánica añade profundidad táctica al combate, ya que el jugador puede utilizar las zonas de lodo para ralentizar a los enemigos perseguidores o las zonas de hielo para alejarse rápidamente.

Sistema de muerte. Al alcanzar cero puntos de vida, el enemigo notifica al `GameManager` para el cálculo de puntuación y ejecuta un parpadeo rojo de retroalimentación visual antes de destruirse:

```
1 protected virtual void Die() {
2     if (isDead) return;
3     isDead = true;
4
5     if (rb != null) {
6         rb.linearVelocity = Vector2.zero;
7         rb.bodyType = RigidbodyType2D.Kinematic;
8     }
9     if (animator != null)
10        animator.enabled = false;
11
12     if (GameManager.Instance != null) {
13         int scoreValue = enemyData != null ? enemyData.damage * 5 : 10;
14         GameManager.Instance.NotifyEnemyKilled(scoreValue);
15     }
```

4.3. SISTEMA DE ENEMIGOS

```

15     }
16
17     StartCoroutine ( DeathBlinkCoroutine () );
18 }
19
20 private IEnumerator DeathBlinkCoroutine () {
21     int blinks = 4;
22     float interval = 0.08f;
23
24     for (int i = 0; i < blinks; i++) {
25         SetSpriteRenderersColor (Color.red);
26         yield return new WaitForSeconds (interval);
27         SetSpriteRenderersColor (Color.white);
28         yield return new WaitForSeconds (interval);
29     }
30
31     if (animator != null && animator.runtimeAnimatorController != null && hasAnimDieTrigger)
32     {
33         animator.enabled = true;
34         animator.SetTrigger ("Die");
35         Destroy (gameObject, 0.5f);
36     } else {
37         Destroy (gameObject);
38     }
39 }

```

Antes de notificar al GameManager, el método `Die()` congela al enemigo: se anula la velocidad del `Rigidbody2D` y se cambia su tipo a `Kinematic` para impedir que la física lo desplace (incluido el *dash* del `SlimeBlue`, cuyo `FixedUpdate` sigue asignando velocidad mientras `isDashing` sea verdadero). Además, se desactiva el `Animator` para que el `sprite` quede estático mirando hacia la última dirección, evitando que la animación de movimiento continúe durante el parpadeo. La corutina `DeathBlinkCoroutine()` alterna el color de todos los `SpriteRenderer` entre rojo y blanco cuatro veces (intervalo de 0,08 s por cambio, duración total de 0,64 s), proporcionando retroalimentación visual inmediata de eliminación. Tras el parpadeo, si el `Animator` dispone de un `trigger Die`, se reactiva para reproducir la animación de muerte y el objeto se destruye tras 0,5 s; en caso contrario, se destruye inmediatamente.

Daño por contacto. Los enemigos infligen daño continuo al jugador mientras permanecen en contacto con él, verificado mediante `OnCollisionStay2D`. El sistema respeta tanto el periodo de gracia al aparecer como el estado de esquiva del jugador:

```

1 protected virtual void OnCollisionStay2D (Collision2D collision) {
2     if (isDead) return;
3
4     if (collision.gameObject.CompareTag ("Player")) {
5         PlayerController pc = collision.gameObject.GetComponent <PlayerController> ();
6         if (pc != null && pc.IsRolling) return; // Sin dano durante roll
7
8         if (Time.time < contactDamageEnabledTime) return; // Periodo de gracia
9
10        if (Time.time >= lastDamageTime + damageRate) {
11            PlayerHealth ph = collision.gameObject.GetComponent <PlayerHealth> ();
12            if (ph != null && enemyData != null) {
13                ph.TakeDamage (enemyData.damage);
14                lastDamageTime = Time.time;
15            }
16        }
17    }
18 }

```

La comprobación `pc.IsRolling` conecta el sistema de contacto con la mecánica de es-
quiva documentada anteriormente: durante la ejecución de un *roll*, el jugador no recibe daño
por contacto, proporcionando un uso defensivo a la mecánica de esquivas.

4.3.2 SlimeGreen: enemigo francotirador

El SlimeGreen implementa un comportamiento de combate a distancia. Se diseñó este
arquetipo para obligar al jugador a adoptar una postura ofensiva: frente a un enemigo que
huye al ser perseguido, el jugador no puede limitarse a esquivar, sino que debe cerrar la
distancia activamente o utilizar armas a distancia. La alternativa descartada era crear un se-
gundo enemigo perseguidor con estadísticas diferentes, pero esto habría reducido la variedad
táctica al ofrecer el mismo tipo de desafío con números distintos.

Su inteligencia artificial sigue un patrón de **máquina de estados finitos (FSM) implícita
con tres estados**, evaluados en cada fotograma mediante la distancia euclidiana al juga-
dor. Aunque los estados no se representan como una enumeración explícita en el código (a
diferencia del `GameState` del `GameManager`), el comportamiento resultante es funcionalmen-
te equivalente a una FSM formal. La siguiente tabla define los estados, las condiciones de
transición y las acciones asociadas:

Estado	Condición de entrada	Acción ejecutada	Transición de salida
Huida (<i>Escape</i>)	<code>distancia < escape</code> <code>Distance (4 u)</code>	Movimiento en direc- ción opuesta al jugador a $1,5 \times$ velocidad base, limitado por <code>CapTo</code> <code>PlayerSpeed</code>	-> Disparo si distancia ≥ 4 u; -> Acercamien- to si distancia > 12 u
Disparo (<i>Shoot</i>)	<code>escapeDistance <=</code> <code>distancia <= max</code> <code>Distance (4–12 u)</code>	Detención completa (<code>rb.linearVelocity</code> $=$ <code>Vector2.zero</code>) + disparo de proyectil si <code>Time.time >= next</code> <code>ShootTime</code>	-> Huida si distancia $<$ 4 u; -> Acercamiento si distancia > 12 u
Acercamiento (<i>Ap- proach</i>)	<code>distancia > max</code> <code>Distance (12 u)</code>	Movimiento hacia el ju- gador a velocidad base, limitado por <code>CapTo</code> <code>PlayerSpeed</code>	-> Disparo si distancia ≤ 12 u; -> Huida si dis- tancia < 4 u

Tabla 4.2: Estados, transiciones y acciones de la FSM del SlimeGreen.

El siguiente diagrama de estados formaliza las transiciones entre los tres estados:

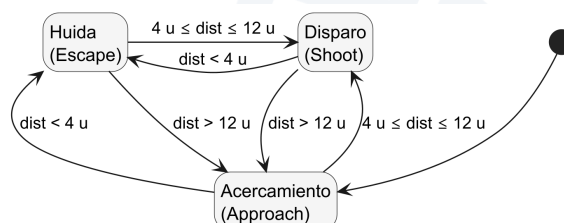


Figura 4.8: Diagrama de la FSM del SlimeGreen. *Elaboración propia.*

La implementación concreta en el método `Update()` del `SlimeGreen` muestra cómo la evaluación de distancia en cada fotograma sustituye a una tabla de transiciones explícita:

```

1 protected override void Update() {
2     base.Update(); // (1) Logica base: gracia, tiles
3
4     if (isDead || player == null) return; // (2) Guardas de seguridad
5
6     if (!IsAIEnabled) { // (3) Periodo de gracia (0,75 s)
7         StopMoving();
8         return;
9     }
10
11     float distanceToPlayer = Vector2.Distance( // (4) Distancia euclidiana
12         transform.position, player.position);
13
14     if (distanceToPlayer < escapeDistance) { // (5) FSM: transicion a Huida
15         Escape();
16     } else if (distanceToPlayer > maxDistance) { // (6) FSM: transicion a
17         Acercamiento
18         Approach();
19     } else { // (7) FSM: estado Disparo
20         StopMoving();
21         if (Time.time >= nextShootTime) {
22             Shoot();
23         }
24     }
25
26     UpdateFallbackAnimation(); // (8) Animacion procedural de
    respaldo
  }

```

Línea por línea:

- **(1)** `base.Update()`: invoca el `Update()` de la clase base `Enemy`, que gestiona la interacción con tiles de velocidad y la detección de daño por contacto. Esta llamada es obligatoria en cualquier subclase que sobrescriba `Update()`, ya que sin ella los periodos de gracia y los modificadores de velocidad por tile dejarían de funcionar, un ejemplo concreto de cómo el patrón *Template Method* impone un contrato entre la clase base y las subclases.
- **(2)** Las guardas `isDead` y `player == null` previenen la ejecución de lógica de IA sobre enemigos ya eliminados o cuando la referencia al jugador se ha perdido (por ejemplo, durante la transición de sala).
- **(3)** `IsAIEnabled` es una propiedad de la clase base `Enemy` que compara `Time.time` con `aiEnabledTime`. Durante los primeros 0,75 segundos tras la aparición, esta propiedad devuelve `false`, y el enemigo permanece inactivo: esta ventana impide que el enemigo dispare al jugador en el mismo fotograma en que aparece.
- **(4)** `Vector2.Distance` calcula la distancia euclidiana $d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$ entre el enemigo y el jugador. Esta métrica se evalúa en cada fotograma (60 veces por segundo con VSync activado), lo que produce transiciones de estado inmediatas cuando el jugador cruza un umbral de distancia.
- **(5-7)** Las tres ramas del condicional implementan las transiciones de la FSM. La prioridad de evaluación es relevante: la condición de huida se evalúa *antes* que la de disparo,

garantizando que la autopreservación tiene prioridad sobre el ataque cuando el jugador está muy cerca.

- **(8)** `UpdateFallbackAnimation()`: aplica una animación procedural de respaldo (oscilación de escala al moverse, pulso al disparar) cuando el Animator no dispone de un AnimatorController asignado. Esto garantiza retroalimentación visual incluso en configuraciones incompletas.

El comportamiento resultante es el de un enemigo que busca mantener una distancia óptima respecto al jugador: si este se acerca demasiado, el slime verde retrocede; si se aleja demasiado, el slime verde se aproxima; cuando se encuentra a la distancia ideal, se detiene y dispara proyectiles con una cadencia configurable.

Análisis de la complejidad computacional de la FSM. La evaluación de la FSM tiene coste $O(1)$ por fotograma: una llamada a `Vector2.Distance` (una raíz cuadrada) y tres comparaciones. Para los tres tipos de enemigo simultáneamente y una oleada típica de 8-12 enemigos, el coste total por fotograma es $O(n)$ donde n es el número de enemigos activos, despreciable frente al coste del renderizado y la simulación física.

Sistema de disparo. El método de disparo instancia un proyectil, le asigna una dirección calculada hacia la posición actual del jugador y le aplica una velocidad configurable. A continuación se muestra el código completo con comentarios exhaustivos:

```

1 private void Shoot() {
2     // Verificación defensiva: si no hay prefab asignado, registrar advertencia
3     if (projectilePrefab == null) {
4         Debug.LogWarning("[SlimeGreen] No hay projectilePrefab asignado!");
5         return;
6     }
7
8     // Calcular direccion normalizada desde el punto de disparo hacia el jugador.
9     Vector2 direction = (player.position - shootPoint.position).normalized;
10
11    // Instanciar proyectil en la posicion del punto de disparo,
12    // sin rotacion (Quaternion.identity): la rotacion se aplica por velocidad.
13    GameObject projectile = Instantiate(
14        projectilePrefab, shootPoint.position, Quaternion.identity);
15
16    // Aplicar multiplicadores de escala: escala de ajuste fino (20 % por defecto)
17    // combinada con la escala de boss (> 1.0 para jefes).
18    float finalScaleMultiplier = projectileScaleMultiplier * bossProjectileScale;
19    projectile.transform.localScale *= finalScaleMultiplier;
20
21    // Asignar velocidad al Rigidbody2D del proyectil.
22    // Se garantiza una velocidad minima segura y se aplica el multiplicador (+15 %).
23    Rigidbody2D projRb = projectile.GetComponent<Rigidbody2D>();
24    float safeBaseSpeed = Mathf.Max(projectileSpeed, minProjectileBaseSpeed);
25    float finalProjectileSpeed = safeBaseSpeed * projectileSpeedMultiplier;
26    if (projRb != null) {
27        projRb.linearVelocity = direction * finalProjectileSpeed;
28    }
29
30    // Asignar dano y escala externa al componente EnemyProjectile.
31    EnemyProjectile projScript = projectile.GetComponent<EnemyProjectile>();
32    if (projScript != null) {
33        projScript.damage = projectileDamage;
34        projScript.externalScaleMultiplier = finalScaleMultiplier;
35    }
36

```

```

37 // Actualizar la direccion del animator y activar animacion de disparo.
38 SetAnimatorDirection(direction);
39 TriggerShootAnimation();
40
41 // Programar el siguiente disparo: Time.time + cooldown.
42 nextShootTime = Time.time + shootCooldown;
43 }

```

Mecánica de rebote contra muros. Un aspecto diferenciador del SlimeGreen es su comportamiento al colisionar con los muros perimetrales durante el estado de huida. En lugar de quedarse atascado contra el muro (lo que lo haría trivial de eliminar), el enemigo calcula una nueva dirección de escape mediante reflexión vectorial:

```

1 private void BounceFromWall(Collision2D collision) {
2     if (collision.contactCount == 0) return;
3
4     // Dirección deseada: alejarse del jugador
5     Vector2 awayFromPlayer = (transform.position - player.position).normalized;
6     // Normal de la superficie del muro en el punto de contacto
7     Vector2 normal = collision.GetContact(0).normal;
8
9     // Reflexión: dirección de movimiento reflejada respecto a la normal del muro
10    Vector2 reflected = Vector2.Reflect(lastMoveDirection, normal).normalized;
11
12    // Candidata: promedio entre la dirección de escape y la reflexión.
13    Vector2 candidate = (awayFromPlayer + reflected).normalized;
14
15    // Protección contra vector nulo: si el promedio se anula,
16    // usar la reflexión pura o la dirección de escape como respaldo.
17    if (candidate.sqrMagnitude < 0.0001f) {
18        candidate = reflected.sqrMagnitude > 0.0001f ? reflected : awayFromPlayer;
19    }
20
21    // Corrección iterativa: si la candidata apunta hacia el jugador,
22    // probar la tangente al muro como alternativa.
23    for (int i = 0; i < wallBounceAttempts; i++) {
24        Vector2 toPlayer = (player.position - transform.position).normalized;
25        if (Vector2.Dot(candidate, toPlayer) < 0f) break; // Ya se aleja: aceptar
26
27        candidate = new Vector2(-normal.y, normal.x).normalized;
28        if (Vector2.Dot(candidate, awayFromPlayer) < 0f) candidate = -candidate;
29    }
30
31    // Aplicar la nueva direccion
32    lastMoveDirection = candidate;
33    if (rb != null) {
34        rb.linearVelocity = lastMoveDirection * lastAppliedMoveSpeed;
35    }
36 }

```

Este algoritmo de rebote utiliza la fórmula de reflexión vectorial $\vec{r} = \vec{d} - 2(\vec{d} \cdot \vec{n})\vec{n}$, donde $\vec{d}$ es la dirección de movimiento y $\vec{n}$ es la normal de la superficie. La corrección iterativa garantiza que el enemigo no se dirija hacia el jugador tras el rebote, preservando la coherencia del comportamiento de huida.

Limitador de velocidad. Para evitar que ningun enemigo pueda ser mas rapido que el jugador (lo que haria imposible la evasión), se implementa un limitador de velocidad:

```

1 private float CapToPlayerSpeed(float desiredSpeed) {
2     if (playerController != null) {
3         float playerSpeed = playerController.CurrentSpeed;

```

```
4     if (playerSpeed > 0f)
5         return Mathf.Min(desiredSpeed, playerSpeed);
6     }
7     return desiredSpeed;
8 }
```

4.3.3 SlimeRed: enemigo perseguidor

El SlimeRed implementa el comportamiento más directo de los tres tipos de enemigo. Su única estrategia consiste en calcular la dirección hacia el jugador y aplicar una velocidad constante en esa dirección:

```
1 private void ChasePlayer() {
2     Vector2 direction = (player.position - transform.position).normalized;
3
4     if (rb != null) {
5         rb.linearVelocity = direction * (moveSpeed * TileSpeedMultiplier);
6     } else {
7         transform.position = Vector2.MoveTowards(
8             transform.position,
9             player.position,
10            (moveSpeed * TileSpeedMultiplier) * Time.deltaTime
11        );
12     }
13
14     SetAnimatorAttacking(false);
15 }
```

El código contempla dos modos de movimiento: mediante Rigidbody2D (preferido, ya que respeta las colisiones físicas) y mediante MoveTowards (como respaldo en caso de que el componente de física no esté disponible). El multiplicador TileSpeedMultiplier, heredado de la clase base Enemy, permite que el efecto de los tiles de velocidad se aplique correctamente. La llamada a SetAnimatorAttacking(false) –método de la clase base que verifica la existencia del parámetro IsAttacking en el animator– indica que el enemigo está persiguiendo y no atacando, lo que permite al animator reproducir la animación de movimiento correspondiente.

4.3.4 SlimeBlue: enemigo saltador

El SlimeBlue implementa un comportamiento basado en saltos rápidos (desplazamientos rápidos, *dash*). A diferencia del SlimeGreen, cuya FSM tiene tres estados, el SlimeBlue implementa una **FSM de dos estados** con una separación clara entre la fase de espera y la fase de desplazamiento:

4.3. SISTEMA DE ENEMIGOS

Estado	Condición de entrada	Duración	Acción
Espera (<i>Idle</i>)	isDashing == false	Indefinida (hasta que se cumplan condiciones)	Enemigo estático (rb.linearVelocity = Vector2.zero); evalúa distancia al jugador y <i>cooldown</i>
Salto (<i>Dash</i>)	minDashDistance <= dist <= maxDashDistance AND Time.time >= lastDashTime + dashCooldown	Exactamente 0,4 s (dashDuration)	Desplazamiento rectilíneo a 12 u/s en dirección fija

Tabla 4.3: Estados y transiciones de la FSM del SlimeBlue.

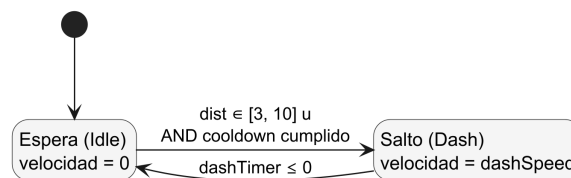


Figura 4.9: Diagrama de la FSM del SlimeBlue. *Elaboración propia.*

El método `Update()` del `SlimeBlue` implementa esta FSM de forma explícita, separando claramente las dos ramas de ejecución:

```

1 protected override void Update() {
2     base.Update(); // (1) Logica base heredada
3
4     if (isDead || player == null) return; // (2) Guardas de seguridad
5
6     if (!IsAIEnabled) { // (3) Gracia: 0,75 s sin IA
7         if (rb != null) rb.linearVelocity = Vector2.zero;
8         return;
9     }
10
11     if (isDashing) { // (4) Estado SALTO
12         SetAnimatorDirection(dashDirection); // Orientar animacion
13         ApplyDashFacing(); // Voltrear todos los sprites
14         dashTimer -= Time.deltaTime; // Decrementar temporizador
15         if (dashTimer <= 0f) {
16             EndDash(); // Transicion: Salto ->
17             Espera
18         }
19     } else { // (5) Estado ESPERA
20         float distanceToPlayer = Vector2.Distance(
21             transform.position, player.position);
22
23         if (distanceToPlayer >= minDashDistance &&
24             distanceToPlayer <= maxDashDistance &&
25             Time.time >= lastDashTime + dashCooldown) { // (6) Condicion de transicion
26             StartDash(); // Transicion: Espera ->
27             Salto
28         }
29     }
30 }

```

```

29 FlipSprite (); // (7) Orientacion visual
30 }

```

Línea por línea:

- **(1)** `base.Update()`: obligatorio para heredar la gestión de gracia y tiles de la clase base `Enemy` (patrón *Template Method*).
- **(4)** En el estado **Salto**, se actualizan la dirección del animator (`SetAnimatorDirection`) y el volteo de todos los `SpriteRenderer` del enemigo (`ApplyDashFacing`) en cada fotograma para reflejar la orientación del salto. El temporizador `dashTimer` se decrementa en cada fotograma. Cuando alcanza cero, se invoca `EndDash()`, que transita al estado **Espera** poniendo `isDashing = false`, deteniendo la velocidad y desactivando el flag de ataque del animator (`SetAnimatorAttacking(false)`).
- **(5-6)** En el estado **Espera**, se evalúan tres condiciones conjuntas: distancia mínima (3 u), distancia máxima (10 u) y *cooldown* cumplido (2 s desde el último salto). Las tres deben cumplirse simultáneamente para iniciar un nuevo salto.

La aplicación de velocidad se realiza en `FixedUpdate()`, separada de la lógica de decisión en `Update()`:

```

1 private void FixedUpdate() {
2     if (isDashing) {
3         rb.linearVelocity = dashDirection * (dashSpeed * TileSpeedMultiplier);
4     } else if (!isDead) {
5         rb.linearVelocity = Vector2.zero; // Estatico entre saltos
6     }
7 }

```

La separación `Update()` / `FixedUpdate()` es una decisión arquitectónica deliberada. La lógica de decisión de la FSM (¿debo saltar?) se ejecuta en `Update()` porque depende de la entrada del sistema y de temporizadores. La aplicación de velocidad al `Rigidbody2D` se ejecuta en `FixedUpdate()` porque el motor de física de Unity opera a frecuencia fija (50 Hz por defecto), y aplicar velocidad en `Update()` produciría movimiento inconsistente a distintas tasas de fotogramas.

La dirección del salto se calcula en el momento de su inicio (`StartDash()`) y permanece fija durante toda la duración del desplazamiento. Esto permite al jugador anticipar la trayectoria del salto y esquivarlo, introduciendo una componente táctica en el enfrentamiento contra este tipo de enemigo. Los parámetros del salto son: velocidad de doce unidades por segundo, duración de 0,4 segundos y tiempo de espera de dos segundos entre saltos consecutivos.



Figura 4.10: Captura de pantalla de *Ascension* durante un encuentro con un enemigo jefe, mostrando el incremento visual de tamaño respecto a los enemigos estándar y los proyectiles de mayor escala. *Elaboración propia.*

Visualización durante el desarrollo. Se implementan *Gizmos* para facilitar el ajuste del rango de salto:

```

1 private void OnDrawGizmosSelected() {
2     Gizmos.color = Color.yellow;
3     Gizmos.DrawWireSphere(transform.position, minDashDistance);
4
5     Gizmos.color = Color.cyan;
6     Gizmos.DrawWireSphere(transform.position, maxDashDistance);
7 }

```

4.3.5 EnemyManager: gestión de aparición de enemigos

El *EnemyManager* es responsable de la instanciación de enemigos en cada sala, la gestión de referencias a los enemigos activos y la detección de sala completada. Soporta dos modos de aparición:

Modo por presupuesto de coste. El sistema dispone de un presupuesto numérico y selecciona enemigos aleatoriamente (ponderados por peso) hasta agotar el presupuesto. Este mecanismo implementa una curva de dificultad **emergente**: a medida que el presupuesto crece con la profundidad de la mazmorra, el sistema puede instanciar combinaciones más peligrosas (por ejemplo, dos slimes verdes y tres rojos en lugar de solo cuatro rojos), sin necesidad de tablas de dificultad codificadas manualmente.

```

1 public void SpawnByCost(Rect area, int maxCost, int depth) {
2     int remaining = maxCost; // (1) Presupuesto disponible
3     int safety = 200;
4     int spawned = 0;
5
6     while (remaining >= minCost && safety-- > 0) { // (2) Bucle de aparición
7         int totalWeight = 0; // (3) Seleccion ponderada

```

```

8      foreach (var a in available)
9          if (a.cost <= remaining) totalWeight += a.weight;
10         if (totalWeight == 0) break;
11
12         int roll = Random.Range(0, totalWeight);
13         GameObject chosen = null; int chosenCost = 0;
14         foreach (var a in available) {
15             if (a.cost > remaining) continue;
16             roll -= a.weight;
17             if (roll < 0) { chosen = a.prefab; chosenCost = a.cost; break; }
18         }
19         if (chosen == null) break;
20
21         Vector2 pos = GetSafeSpawnPosition(area);           // (4) Posicion segura
22         var instance = Instantiate(chosen, pos,             // (5) Instanciacion
23                                   Quaternion.identity);
24
25         remaining -= chosenCost;                             // (6) Decrementar presupuesto
26         spawned++;
27     }
28 }

```

Línea por línea:

- **(1)** `remaining` representa el presupuesto disponible para la sala actual. El presupuesto base se incrementa con `depth` (profundidad de la mazmorra), produciendo oleadas progresivamente más intensas.
- **(2)** El bucle continúa mientras quede presupuesto suficiente para al menos el enemigo más barato (`minCost`). La variable `safety` previene bucles infinitos en caso de error de configuración (por ejemplo, si todos los costes de enemigos exceden el presupuesto restante).
- **(3)** La selección ponderada recorre la lista de enemigos cuyo coste no exceda el presupuesto restante, acumulando sus pesos. Un valor aleatorio (`roll`) determina cuál se selecciona, de modo que enemigos con mayor peso aparecen con mayor frecuencia: un slime rojo (peso alto, coste bajo) aparece más a menudo que un slime verde (peso medio, coste alto), reflejando la intención de diseño de que el enemigo francotirador sea menos común.
- **(4-5)** El posicionamiento seguro garantiza que ningún enemigo aparezca sobre el jugador.
- **(6)** Decrementar el presupuesto tras instanciar produce un reparto orgánico: con presupuesto alto, el sistema puede «gastar» en enemigos caros o repartir en muchos baratos, produciendo composiciones de oleada variadas en cada sala.

Posicionamiento seguro. La función `GetSafeSpawnPosition` evita que los enemigos aparezcan demasiado cerca del jugador, realizando hasta 25 intentos de posicionamiento aleatorio antes de recurrir a una posición de respaldo:

```

1 private Vector2 GetSafeSpawnPosition(Rect area) {
2     Vector2 fallback = new Vector2(
3         Random.Range(area.xMin, area.xMax),
4         Random.Range(area.yMin, area.yMax));

```

4.3. SISTEMA DE ENEMIGOS

```

5
6     var player = GameObject.FindGameObjectWithTag("Player");
7     if (player == null) return fallback;
8
9     Vector2 playerPos = player.transform.position;
10    float minDistSqr = safeSpawnRadius * safeSpawnRadius;
11
12    for (int i = 0; i < safeSpawnAttempts; i++) {
13        Vector2 candidate = new Vector2(
14            Random.Range(area.xMin, area.xMax),
15            Random.Range(area.yMin, area.yMax));
16        if ((candidate - playerPos).sqrMagnitude >= minDistSqr)
17            return candidate;
18    }
19
20    return fallback;
21 }

```

Detección de sala completada. En cada fotograma, el gestor elimina las referencias a enemigos destruidos y evalúa si todos los enemigos han sido eliminados. Cuando esto ocurre, emite un evento que desencadena la habilitación de las escaleras de progresión:

```

1 void Update() {
2     if (!roomActive) return;
3
4     PruneTracked(); // Eliminar referencias nulas
5
6     if (trackedEnemies.Count == 0) {
7         roomActive = false;
8         roomCleared = true;
9
10        if (!suppressClearEvent) {
11            OnRoomCleared?.Invoke();
12        }
13    }
14 }

```

4.3.6 Configuración de enemigos: EnemyData

Los datos de cada tipo de enemigo se definen mediante *ScriptableObjects*. Se seleccionó este mecanismo frente a la codificación de estadísticas en los propios scripts de enemigo por la misma razón expuesta para los datos de armas: durante el desarrollo, los parámetros de equilibrado de los enemigos (vida, daño, velocidad, coste de aparición) se ajustaron iterativamente más de diez veces, y cada ajuste con *ScriptableObjects* es inmediato y no requiere recompilación.

```

1 [CreateAssetMenu(menuName = "Enemies/EnemyData")]
2 public class EnemyData : ScriptableObject {
3     public string enemyName; // Identificador visible ("Slime Verde")
4     public int maxHealth = 3; // Puntos de vida maximos
5     public int damage = 1; // Dano infligido por contacto o proyectil
6     public float speed = 2f; // Velocidad base en unidades/segundo
7     public Sprite sprite; // Sprite principal del enemigo
8     public int enemyCost; // Coste de aparicion en el sistema de presupuesto
9 }

```

El campo `enemyCost` merece una explicación específica: define cuánto "cuesta" instanciar este enemigo dentro del sistema de presupuesto del `EnemyManager`. Un slime rojo (perseguidor, fácil de esquivar) tiene un coste bajo, mientras que un slime verde (francotirador, más

peligroso) tiene un coste superior. El presupuesto total del `EnemyManager` se incrementa con la profundidad de la mazmorra, produciendo oleadas progresivamente más intensas sin necesidad de tablas de dificultad codificadas manualmente. Esta decisión reduce el acoplamiento entre el sistema de dificultad y los tipos de enemigo: añadir un nuevo tipo de enemigo con su `enemyCost` correspondiente integra automáticamente ese enemigo en la curva de dificultad sin modificar el código del `EnemyManager`.

4.3.7 Síntesis comparativa de los tipos de enemigo

La siguiente tabla resume las diferencias de comportamiento, parámetros y rol táctico entre los tres tipos de enemigo, mostrando cómo la jerarquía basada en el patrón *Template Method* permite diferenciar la IA sin duplicar la lógica base compartida:

Aspecto	SlimeRed (perseguidor)	SlimeGreen (francotirador)	SlimeBlue (saltador)
Comportamiento de IA	Persecución directa constante	FSM de 3 estados (huir, acercarse, disparar)	Ciclo de espera y salto (<i>dash</i>)
Método principal sobrescrito	<code>Update()</code> -> <code>ChasePlayer()</code>	<code>Update()</code> -> <code>Escape()</code> / <code>Approach()</code> / <code>Shoot()</code>	<code>Update()</code> -> <code>StartDash()</code> / <code>EndDash()</code>
Movimiento	<code>rb.linearVelocity</code> hacia el jugador	<code>rb.linearVelocity</code> según zona de distancia	<code>rb.linearVelocity</code> solo durante el salto
Distancia al jugador	Siempre se acerca (0 m ideales)	Mantiene distancia óptima (4–12 unidades)	Rango de activación (3–10 unidades)
Mecánica de daño	Contacto directo	Proyectil a distancia (<code>EnemyProjectile</code>)	Contacto durante el salto
Velocidad base	<code>moveSpeed × TileSpeedMultiplier</code>	Limitada por <code>CapToPlayerSpeed</code>	<code>dashSpeed</code> = 12 u/s durante 0,4 s
Tiempo de espera	Ninguno (acción continua)	<code>shootCooldown</code> entre disparos	<code>dashCooldown</code> = 2 s entre saltos
Coste de aparición	Bajo (fácil de esquivar)	Alto (peligroso a distancia)	Medio (predecible pero rápido)
Rol táctico	Presión constante, obliga a moverse	Obliga a cerrar distancia o usar arma a distancia	Requiere anticipación y esquiva
Contramedida del jugador	Arma cuerpo a cuerpo y kiting	Arma a distancia o persecución rápida	Esquiva (<i>roll</i>) y posicionamiento

Tabla 4.4: Comparativa de comportamiento, parámetros y rol táctico de los tres tipos de enemigo.

La diversidad de estos tres arquetipos, combinada con el sistema de presupuesto de coste del `EnemyManager` y la variabilidad de los tiles especiales, produce situaciones de combate tácticas y diferenciadas en cada sala sin necesidad de un repertorio extenso de tipos de enemigo.

4.3.8 Diagrama de flujo de decisión de la IA por tipo de enemigo

El siguiente diagrama de flujo unificado muestra la lógica de decisión completa que ejecuta cada tipo de enemigo en cada fotograma, incluyendo la lógica heredada de la clase base `Enemy` (común a los tres) y la lógica específica de cada subclase:

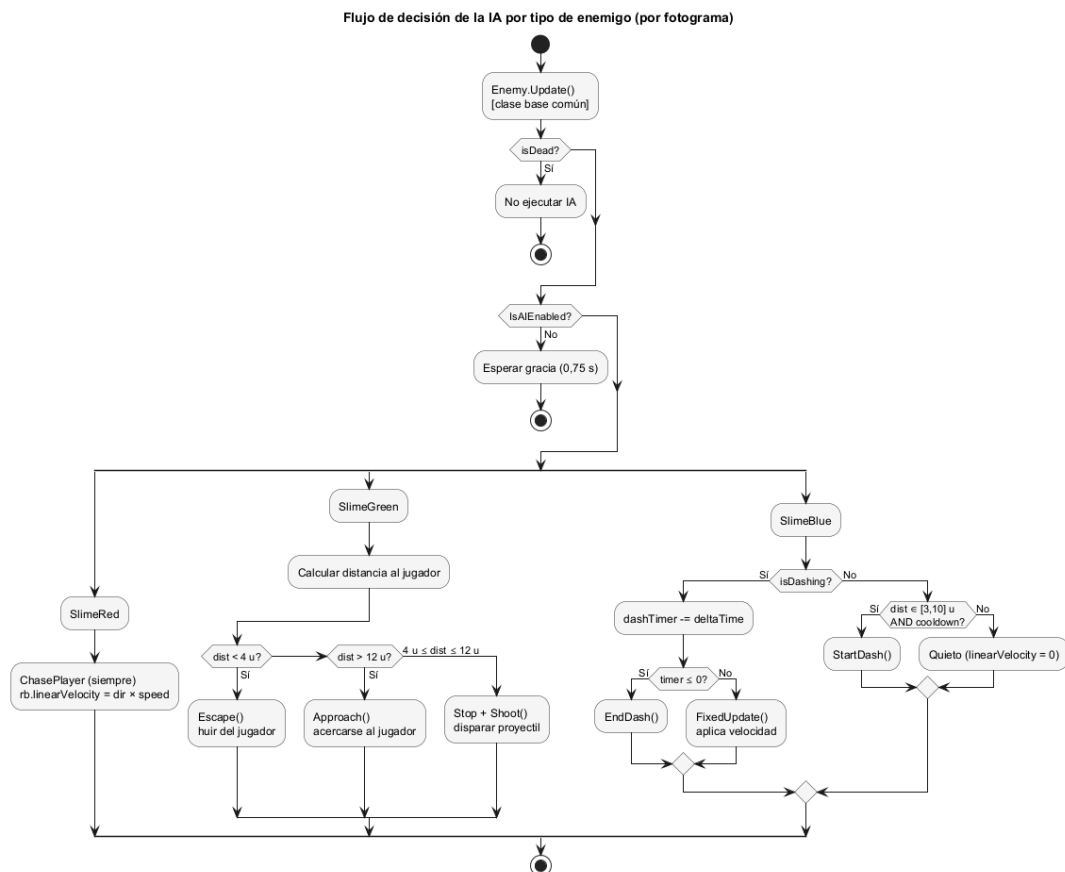


Figura 4.11: Diagrama de flujo de decisión de la IA por tipo de enemigo. *Elaboración propia.*

Este diagrama evidencia una propiedad del diseño: la lógica heredada de la clase base `Enemy` (verificaciones de muerte, gracia y tiles) se ejecuta siempre *antes* que la lógica específica de cada subclase, garantizando que las comprobaciones de seguridad son comunes e inmutables –un ejemplo concreto del contrato impuesto por el patrón *Template Method*.

Figura 4.5 - Jerarquía de clases de enemigos (patrón Template Method)

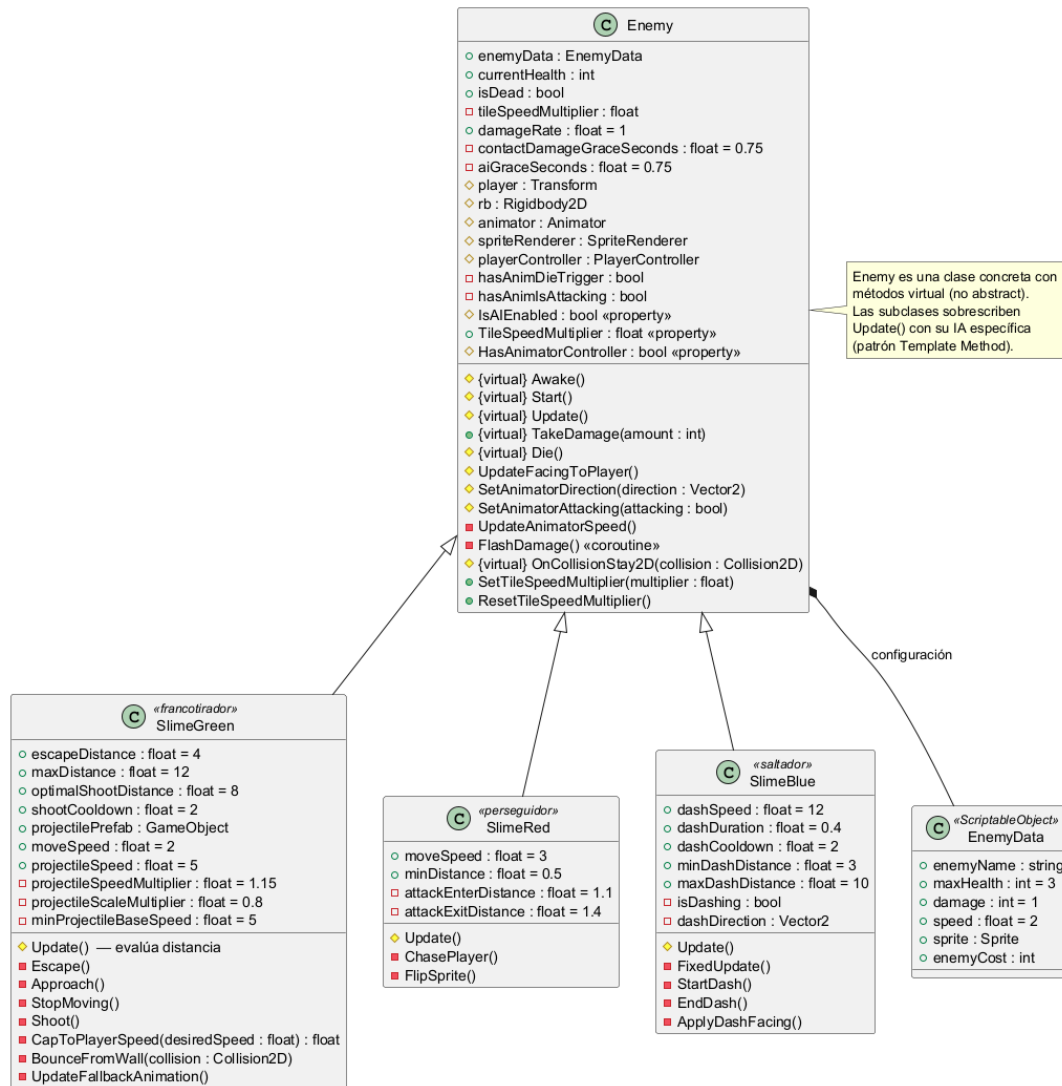


Figura 4.12: Diagrama de clases UML de la jerarquía de enemigos. *Elaboración propia.*

Figura 4.6 - Secuencia: flujo de combate (melee y distancia)

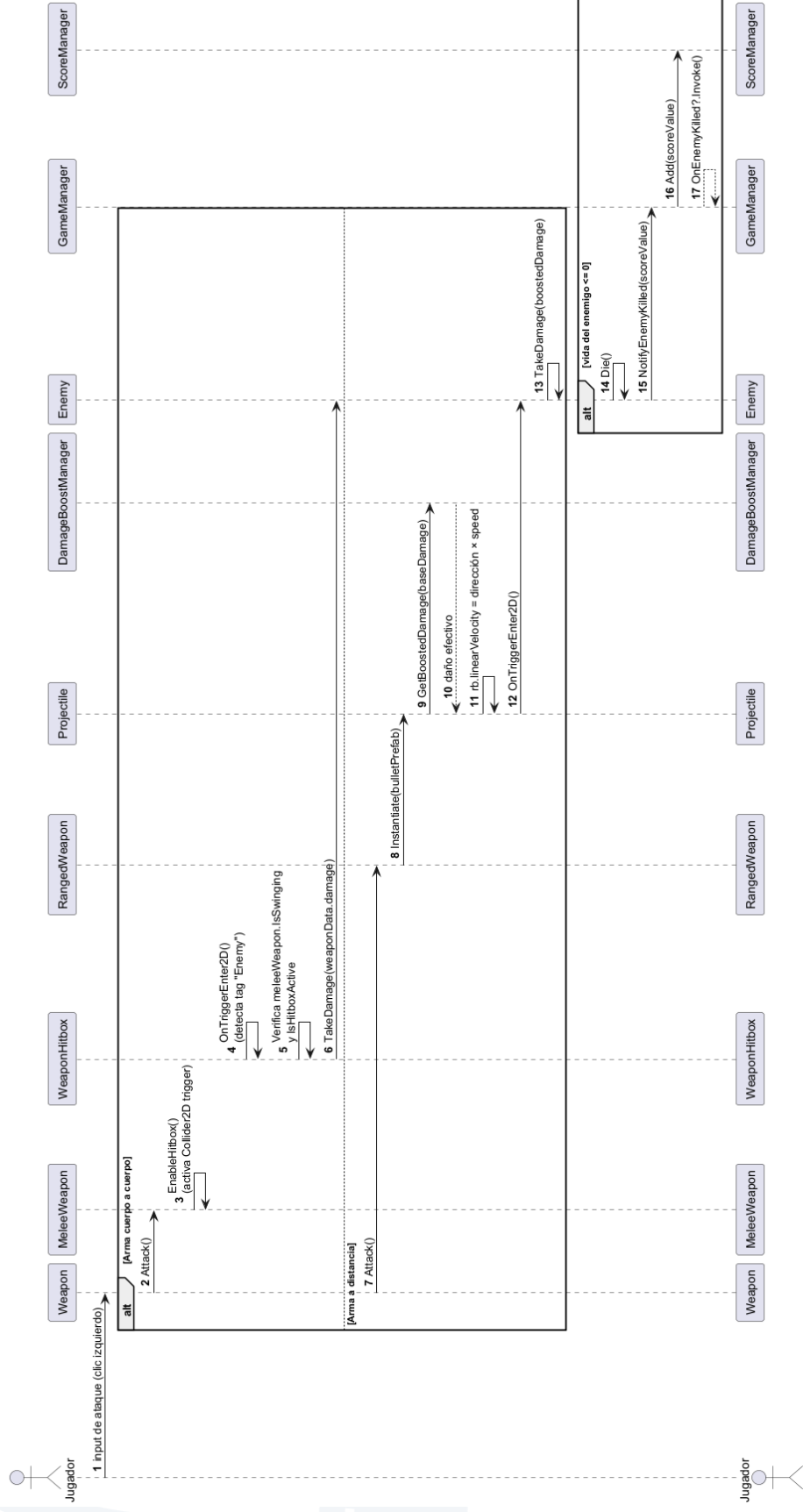


Figura 4.13: Diagrama de secuencia UML del flujo de combate. *Elaboración propia.*

4.4 SISTEMA DE ARMAS Y COMBATE

Evolución iterativa. El sistema de armas pasó por dos rediseños. La primera versión implementaba una clase única *Weapon* con un enumerado *WeaponType* y bloques condicionales que diferenciaban el comportamiento cuerpo a cuerpo del comportamiento a distancia; esta estructura se volvió difícil de mantener al añadir los efectos visuales (*weapon trail*, parpadeo de hitbox), ya que cada tipo de arma requería campos y lógica distintos dentro de la misma clase. La refactorización al patrón *Strategy* (con *Weapon* como clase base y *MeleeWeapon* / *RangedWeapon* como estrategias concretas) resolvió este problema. La animación de barrido del arma cuerpo a cuerpo se rediseñó de interpolación lineal a sinusoidal tras observar que el movimiento uniforme transmitía una sensación mecánica carente del «peso» visual esperado en un arma.

4.4.1 *Weapon*: clase base del sistema de armas y patrón *Strategy*

El sistema de armas implementa el patrón *Strategy*. Se seleccionó este patrón frente a la alternativa de una única clase *Weapon* con un enumerado *WeaponType* y bloques condicionales (`if (type == Melee) ... else if (type == Ranged) ...`). Esta alternativa se descartó por dos razones: en primer lugar, cada tipo de arma requiere campos específicos distintos (`swingAngle` y `swingDuration` para las armas cuerpo a cuerpo, `bulletPrefab` y `attackCooldown` para las armas a distancia), lo que produciría una clase con campos que solo son relevantes para uno de los tipos; en segundo lugar, añadir un tercer tipo de arma obligaría a modificar la lógica interna de la clase existente, violando el principio de apertura-cierre.

El patrón *Strategy* resuelve ambos problemas: la clase base *Weapon* define la interfaz común (`Attack()`, rotación hacia el cursor, órbita alrededor del jugador) y las subclases *MeleeWeapon* y *RangedWeapon* encapsulan únicamente los campos y la lógica relevantes para su tipo de ataque.

Sistema de rotación y órbita. El arma sigue la posición del cursor del ratón y orbita alrededor del jugador:

```
1 private void RotateTowardsMouse() {  
2     Vector3 mouseWorldPosition = camera.ScreenToWorldPoint(Input.mousePosition);  
3     mouseWorldPosition.z = 0;  
4  
5     Vector2 direction = (mouseWorldPosition - transform.position).normalized;  
6     float angle = Mathf.Atan2(direction.y, direction.x) * Mathf.Rad2Deg;  
7  
8     float spriteOrientationOffset = -90f;  
9     transform.rotation = Quaternion.Euler(0, 0, angle + spriteOrientationOffset);  
10 }
```

La conversión de la posición del ratón de coordenadas de pantalla a coordenadas del mundo se realiza mediante `Camera.ScreenToWorldPoint`. El ángulo de rotación se calcula mediante `Mathf.Atan2(direction.y, direction.x)`, que retorna el ángulo en radianes del vector dirección respecto al eje X positivo; la multiplicación por `Mathf.Rad2Deg` convierte a grados para aplicarlo como rotación del `transform`. El desplazamiento de -90° (`spriteOrientationOffset`) compensa que los sprites de las armas están orientados hacia arriba

en su fichero de origen: sin este ajuste, un ángulo de 0° apuntaría el sprite hacia la derecha en lugar de hacia arriba.

Se seleccionó el cálculo trigonométrico (Atan2 + rotación Euler) frente a la alternativa de usar `Quaternion.LookRotation`, porque esta última está orientada al espacio 3D y requiere una conversión adicional para obtener una rotación exclusivamente en el eje Z, lo que introduce complejidad innecesaria en un proyecto 2D.

4.4.2 MeleeWeapon: armas cuerpo a cuerpo

El `MeleeWeapon` implementa un ataque de barrido (*swing*) con un arco configurable de 180 grados y una animación procedural:

```
1 protected override void Update() {
2     base.Update();
3
4     // Cancelar swing si el jugador suelta el boton del raton a mitad de ataque
5     if (!Input.GetMouseButton(0) && isSwinging) {
6         StopSwinging();
7         return;
8     }
9
10    if (isSwinging) {
11        swingTimer += Time.deltaTime;
12        float progress = swingTimer / swingDuration;
13
14        if (progress >= 1f) {
15            StopSwinging();
16        } else {
17            float smoothProgress = Mathf.Sin(progress * Mathf.PI);
18            float currentSwingOffset = Mathf.Lerp(
19                -swingAngle / 2,
20                swingAngle / 2,
21                smoothProgress
22            );
23
24            Vector3 currentRotation = transform.localEulerAngles;
25            currentRotation.z = swingStartAngle + currentSwingOffset;
26            transform.localRotation = Quaternion.Euler(currentRotation);
27        }
28    }
29 }
```

La animación del barrido utiliza una curva sinusoidal (`Mathf.Sin`) para producir un movimiento suave que acelera al inicio y desacelera al final, resultando en una sensación de peso y naturalidad. Se seleccionó la interpolación sinusoidal frente a una interpolación lineal (`Mathf.Lerp` con progreso lineal) porque esta última produce un movimiento uniforme que transmite una sensación mecánica, carente del «peso» visual que se espera de un arma cuerpo a cuerpo. Si el jugador suelta el botón del ratón durante el barrido, este se cancela inmediatamente mediante `StopSwinging()`, que centraliza la desactivación de la hitbox y el rastro visual (`WeaponTrail`) en un único método, evitando la duplicación de lógica de limpieza.

Línea por línea del fragmento:

- `swingTimer += Time.deltaTime`: acumula el tiempo transcurrido desde el inicio del barrido, independientemente de la tasa de fotogramas.

- `float progress = swingTimer / swingDuration`: normaliza el progreso del barrido al rango `[0, 1]`, donde 0 es el inicio y 1 es el final.
- `Mathf.Sin(progress * Mathf.PI)`: convierte el progreso lineal en una curva sinusoidal. Cuando `progress = 0`, el seno vale 0; cuando `progress = 0.5`, vale 1 (máximo desplazamiento); cuando `progress = 1`, vuelve a 0.
- `Mathf.Lerp(-swingAngle/2, swingAngle/2, smoothProgress)`: mapea el progreso sinusoidal a un ángulo de rotación dentro del rango `[-90°, +90°]`, produciendo el arco de 180° configurable.
- `StopSwinging()`: al completar el barrido (`progress >= 1`) o al soltar el botón, desactiva la hitbox, detiene el rastro visual y marca `isSwinging = false`.

Detección de impacto. Durante el barrido, la hitbox del arma detecta colisiones con enemigos. Si existen componentes `WeaponHitbox` hijos, la detección se delega a ellos para evitar daño duplicado; en caso contrario, el propio `MeleeWeapon` verifica las condiciones de activación:

```

1 private void OnTriggerEnter2D(Collider2D other) {
2     // Si hay WeaponHitbox hijos, ellos gestionan el dano
3     if (weaponHitboxes != null && weaponHitboxes.Length > 0) return;
4     if (!isSwinging || !hitboxesActive) return;
5     if (hitbox == null || !hitbox.enabled) return;
6
7     if (other.CompareTag("Enemy") && weaponData != null) {
8         Enemy enemy = other.GetComponent<Enemy>();
9         if (enemy != null)
10             enemy.TakeDamage(weaponData.damage);
11     }
12 }

```

4.4.3 RangedWeapon: armas a distancia

El `RangedWeapon` implementa el ataque mediante la instanciación de proyectiles:

```

1 public override void Attack() {
2     if (Time.time < lastAttackTime + attackCooldown) return;
3
4     if (weaponData?.bulletPrefab != null) {
5         lastAttackTime = Time.time;
6
7         Vector3 spawnPosition = spawner?.position ?? transform.position;
8         Quaternion spawnRotation = transform.rotation;
9
10        GameObject bullet = Instantiate(
11            weaponData.bulletPrefab,
12            spawnPosition,
13            spawnRotation
14        );
15
16        Projectile projectileScript = bullet.GetComponent<Projectile>();
17        if (projectileScript != null) {
18            projectileScript.damage = weaponData.damage;
19            projectileScript.speed = 10f;
20        }
21    }
22 }

```

El operador `?.` (operador condicional nulo) evita excepciones cuando una referencia es nula. El operador `??` (operador de coalescencia nula) proporciona un valor de respaldo cuando el valor principal es nulo.

4.4.4 Projectile: comportamiento de proyectiles

El componente `Projectile` controla el movimiento lineal del proyectil, su autodestrucción por tiempo de vida y la integración con el sistema de bonificación de daño:

```
1 void Start() {  
2     if (DamageBoostManager.Instance != null) {  
3         damage = DamageBoostManager.Instance.GetBoostedDamage(damage);  
4     }  
5  
6     if (rb != null) {  
7         rb.linearVelocity = transform.up * speed;  
8     }  
9  
10    Destroy(gameObject, lifetime);  
11 }
```

La consulta al `DamageBoostManager` permite que los proyectiles se beneficien de las bonificaciones de daño otorgadas por los tiles de potenciación. La llamada a `Destroy(gameObject, lifetime)` programa la destrucción automática del proyectil tras un tiempo configurable, evitando la acumulación de objetos fuera de los límites de la sala.

Detección de impacto. Al colisionar, el proyectil aplica daño a los enemigos y se destruye al contactar con paredes o enemigos. Los proyectiles del jugador ignoran explícitamente las armas del propio jugador para evitar colisiones involuntarias:

```
1 void OnTriggerEnter2D(Collider2D other) {  
2     if (other.CompareTag("Player") || other.CompareTag("Weapon")  
3         || other.GetComponent<Weapon>() != null)  
4         return;  
5  
6     if (other.CompareTag("Enemy")) {  
7         Enemy enemy = other.GetComponent<Enemy>();  
8         if (enemy != null)  
9             enemy.TakeDamage(damage);  
10    }  
11  
12    if (other.CompareTag("Wall") || other.CompareTag("Enemy"))  
13        Destroy(gameObject);  
14 }
```

La comprobación combinada de la etiqueta `"Weapon"` y del componente `Weapon` proporciona una doble verificación: la etiqueta cubre los casos estándar, mientras que la comprobación de componente cubre prefabs que pudieran no tener la etiqueta asignada.

4.4.5 WeaponData: configuración de armas mediante *ScriptableObjects*

Los datos de cada arma se definen mediante *ScriptableObjects*, manteniendo la separación entre datos y lógica. Al igual que con `PlayerClass` y `EnemyData`, se seleccionó este mecanismo frente a la codificación directa de parámetros en los scripts de arma. Esta decisión permite que el mismo prefab de `MeleeWeapon` se reutilice con distintas configuraciones

(espada, hacha, lanza) simplemente asignando un *ScriptableObject* diferente, sin duplicar prefabs ni código:

```
1 [CreateAssetMenu(menuName = "Items/WeaponData")]
2 public class WeaponData : ScriptableObject {
3     public string weaponName;           // Nombre visible del arma
4     public int damage;                  // Dano base por impacto
5     public float attackSpeed;           // Velocidad de ataque (tiempo de espera entre ataques)
6     public Sprite sprite;               // Sprite visual del arma
7     public GameObject bulletPrefab;     // Prefab del proyectil (solo para armas a distancia;
8     null para melee)
9     public GameObject weaponPrefab;     // Prefab del GameObject completo del arma
10 }
```

Línea por línea:

- **damage:** daño base que puede ser modificado en tiempo de ejecución por el *Damage BoostManager* cuando el jugador se encuentra sobre un tile de potenciación.
- **bulletPrefab:** referencia condicional, solo las armas a distancia la utilizan; las armas cuerpo a cuerpo la dejan como `null`. El código de *RangedWeapon.Attack()* verifica esta referencia antes de instanciar proyectiles mediante el operador condicional nulo (`?.`).
- **weaponPrefab:** referencia al prefab preconfigurado que incluye el componente *Melee Weapon* o *RangedWeapon*, los colisionadores, el *SpriteRenderer* y los efectos visuales.

4.4.6 Flujo completo de daño: del ataque a la puntuación

El sistema de combate involucra a cinco componentes distribuidos en tres subsistemas (armas, enemigos y gestión central). El siguiente diagrama describe la cadena completa de invocaciones que se produce cuando el jugador ataca a un enemigo, desde la entrada del jugador hasta la actualización de la puntuación:

4.4. SISTEMA DE ARMAS Y COMBATE

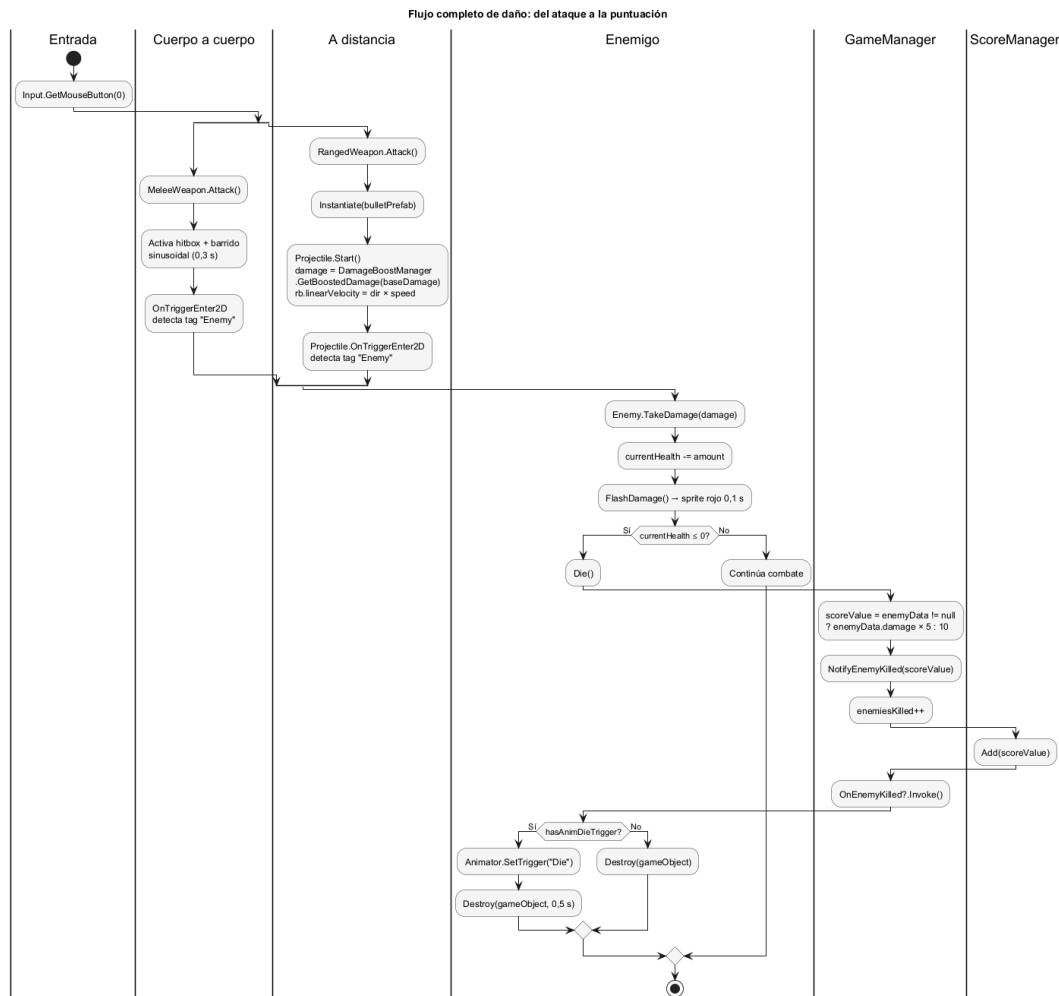


Figura 4.14: Diagrama de flujo del sistema de daño. *Elaboración propia.*

Este flujo evidencia la separación de responsabilidades aplicada: cada componente ejecuta exclusivamente su tarea (el arma ataca, el enemigo recibe daño, el gestor contabiliza) y la comunicación entre capas se realiza mediante invocaciones directas tipadas y eventos del patrón *Observer*. La alternativa (que el arma calculase ella misma la puntuación, o que el enemigo accediese directamente al marcador de la UI) habría introducido dependencias transversales entre subsistemas que no comparten responsabilidad.

Figura 4.8 - Sistema de armas (patrón Strategy)

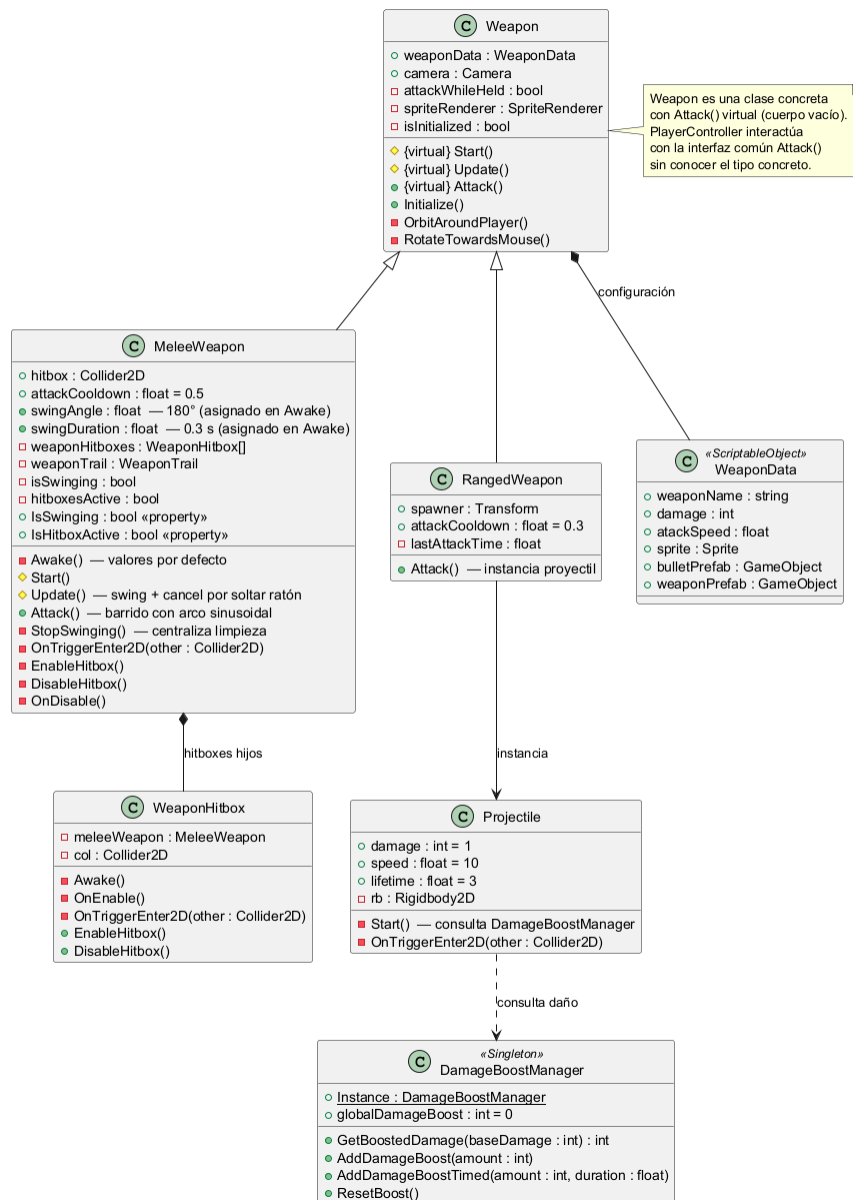


Figura 4.15: Diagrama de clases UML del sistema de armas. *Elaboración propia.*

4.5 SISTEMA DE GENERACIÓN PROCEDURAL

Evolución iterativa. La generación procedural fue el subsistema con mayor impacto arquitectónico derivado de la iteración. El diseño inicial cargaba una nueva escena de Unity para cada sala, lo que provocaba tiempos de carga de 0,5 a 1,5 segundos que rompían el ritmo de juego. El rediseño hacia la regeneración del *tilemap* dentro de la misma escena (`RegenerateFloorKeepWalls`) eliminó estos tiempos de carga y modificó las responsabilidades del `RoomFlowController`. Adicionalmente, la distribución de tiles especiales evolucionó de una colocación puramente aleatoria (tile a tile) a un sistema de *clusters* por expansión



radial: la distribución aleatoria producía tiles aislados cuyo efecto era imperceptible durante el juego, mientras que los *clusters* de 4-9 tiles generan zonas tácticas significativas que el jugador puede utilizar o evitar de forma deliberada.

4.5.1 RoomGenerator: generación de salas

El RoomGenerator constituye uno de los componentes de mayor complejidad del proyecto. Es responsable de la generación procedural de las salas del juego, incluyendo la estructura de muros, la distribución de tiles de suelo con variantes visuales y la colocación de tiles especiales con efectos.

Se seleccionó un enfoque de rejilla rectangular fija frente a alternativas más sofisticadas como BSP (*Binary Space Partitioning*) o WFC (*Wave Function Collapse*) por tres razones que afectan directamente a la jugabilidad y la complejidad de implementación:

1. **Simplicidad de posicionamiento de enemigos.** Con una rejilla rectangular fija de límites conocidos, cualquier coordenada dentro del perímetro de muros constituye una posición válida para la aparición de enemigos. Con BSP, las salas tendrían formas irregulares conectadas por pasillos estrechos, lo que requeriría verificaciones adicionales de accesibilidad y de línea de visión entre el jugador y el punto de aparición.
2. **Contención automática.** Los bordes perimetrales de muros generan colisionadores automáticos mediante el TilemapCollider2D, impidiendo que el jugador abandone el área de juego sin lógica de contención adicional.
3. **Variabilidad táctica mediante tiles.** La variabilidad entre salas no reside en la forma geométrica del espacio, sino en la distribución de tiles especiales dentro de él. Este enfoque combina la previsibilidad estructural (el jugador siempre sabe el tamaño de la sala) con la imprevisibilidad táctica (la ubicación de zonas de hielo, lodo y potenciación cambia en cada sala).

Configuración de la sala. Cada sala se genera sobre una rejilla de 26×12 tiles, con un borde perimetral de muros de 1 tile de grosor:

```
1 [SerializeField] private int roomWidth = 26;  
2 [SerializeField] private int roomHeight = 12;  
3 [SerializeField] private int wallBorderThicknessTiles = 1;
```

Algoritmo de generación. El proceso de generación de una sala sigue una secuencia de cinco pasos. A continuación se muestra la estructura real del método GenerateRoom() de RoomGenerator.cs, simplificada para mayor claridad:

```
1 public void GenerateRoom() {  
2     bool clearWalls = true; // En el proyecto real, esta decisión se pasa como parámetro.  
3  
4     // 1. Limpiar tilemaps anteriores  
5     if (clearWalls) wallTilemap.ClearAllTiles();  
6     floorTilemap.ClearAllTiles();  
7  
8     // 2. Rellenar suelo y colocar tiles especiales en clusters  
9     for (int x = 0; x < roomWidth; x++)  
10        for (int y = 0; y < roomHeight; y++) {
```

```

11     Vector3Int pos = new Vector3Int(x, y, 0) + roomOffset;
12     floorTilemap.SetTile(pos, GetRandomNormalFloorTile());
13 }
14 PlaceSpecialTilesClustered(); // hielo, lodo, curacion, potenciacion
15
16 // 3. Colocar escaleras (en cada sala, en posicion aleatoria con margen)
17 PlaceStairsTile();
18
19 if (!clearWalls) return; // Caso RegenerateFloorKeepWalls(): no se repintan muros.
20
21 // 4. Generar borde de muros perimetrales (bucles inline)
22 int wallT = Mathf.Max(1, wallBorderThicknessTiles);
23
24 // Muro inferior / superior (incluye esquinas)
25 for (int t = 1; t <= wallT; t++) {
26     int yBottom = -t;
27     int yTop = roomHeight - 1 + t;
28     for (int x = -wallT; x <= (roomWidth - 1 + wallT); x++) {
29         wallTilemap.SetTile(new Vector3Int(x, yBottom, 0) + roomOffset, wallTile);
30         wallTilemap.SetTile(new Vector3Int(x, yTop, 0) + roomOffset, wallTile);
31     }
32 }
33
34 // Muro izquierdo / derecho (sin duplicar las filas ya pintadas arriba/abajo)
35 for (int t = 1; t <= wallT; t++) {
36     int xLeft = -t;
37     int xRight = roomWidth - 1 + t;
38     for (int y = 0; y < roomHeight; y++) {
39         wallTilemap.SetTile(new Vector3Int(xLeft, y, 0) + roomOffset, wallTile);
40         wallTilemap.SetTile(new Vector3Int(xRight, y, 0) + roomOffset, wallTile);
41     }
42 }
43
44 // 5. Actualizar colisionadores
45 RefreshWallColliders();
46 EnsureBoundaryCollidersSetup();
47 }

```

Paso 1: Limpieza. Se eliminan todos los tiles existentes en ambos *tilemaps* (muros y suelo) mediante `wallTilemap.ClearAllTiles()` y `floorTilemap.ClearAllTiles()`, preparando la rejilla para una nueva generación.

Paso 2: Generación de suelo con clusters. Este paso constituye el núcleo del algoritmo de generación procedural. Primero se rellena toda la superficie interior con tiles normales seleccionados aleatoriamente entre las variantes disponibles. A continuación, se aplica el algoritmo de expansión por *clusters* para colocar los tiles especiales.

Algoritmo de expansión por clusters. Los tiles de hielo y lodo se distribuyen en manchas orgánicas de entre 4 y 9 tiles, generadas mediante un algoritmo de expansión por adyacencia de 4 direcciones desde un centro aleatorio. El método `PaintCluster` implementa esta lógica:

```

1 private void PaintCluster(Vector2Int center, int targetCount, int maxRadius,
2                           TileBase tile, HashSet<Vector2Int> reserved, int padding) {
3     int minX = padding;
4     int maxX = roomWidth - 1 - padding;
5     int minY = padding;
6     int maxY = roomHeight - 1 - padding;
7
8     bool InBounds(Vector2Int c) =>
9         c.x >= minX && c.x <= maxX && c.y >= minY && c.y <= maxY;

```

4.5. SISTEMA DE GENERACIÓN PROCEDURAL

```

10  int Manhattan(Vector2Int a, Vector2Int b) =>
11      Mathf.Abs(a.x - b.x) + Mathf.Abs(a.y - b.y);
12
13  var painted = new List<Vector2Int>(targetCount);
14  if (InBounds(center) && reserved.Add(center)) {
15      painted.Add(center);
16      floorTilemap.SetTile(new Vector3Int(center.x, center.y, 0) + roomOffset, tile);
17  }
18
19  Vector2Int[] dirs = { Vector2Int.right, Vector2Int.left,
20                      Vector2Int.up, Vector2Int.down };
21  int safety = 200;
22  while (painted.Count < targetCount && safety-- > 0) {
23      if (painted.Count == 0) break;
24      var baseCell = painted[Random.Range(0, painted.Count)];
25      var next = baseCell + dirs[Random.Range(0, dirs.Length)];
26
27      if (!InBounds(next)) continue;
28      if (Manhattan(next, center) > maxRadius) continue;
29      if (!reserved.Add(next)) continue;
30
31      painted.Add(next);
32      floorTilemap.SetTile(new Vector3Int(next.x, next.y, 0) + roomOffset, tile);
33  }
34  }

```

El algoritmo comienza seleccionando una posición aleatoria como centro del *cluster*. A continuación, expande el *cluster* seleccionando en cada iteración una celda pintada al azar y desplazándose en una de las cuatro direcciones cardinales. La función local `InBounds` verifica que la posición candidata se encuentra dentro del margen útil del suelo (excluyendo un borde de padding celdas), y el `HashSet<Vector2Int> reserved` impide que una misma celda se pinte dos veces o sea reclamada por *clusters* diferentes. La restricción de distancia Manhattan respecto al centro garantiza que el *cluster* no se extienda más allá del radio máximo, produciendo manchas compactas de forma irregular y orgánica.

Este *pipeline* se ejecuta también (con una variante) cuando el jugador avanza a la siguiente sala dentro de la misma escena: `RegenerateFloorKeepWalls()` omite los pasos 4 y 5 (los muros perimetrales no cambian) y únicamente regenera el suelo, los tiles especiales y las escaleras, reduciendo el coste computacional de la transición.

Probabilidades de aparición de tiles especiales:

Tipo de tile	Probabilidad	Formato	Cantidad por sala
Curación (<i>Heal</i>)	2 %	Tile individual	Máximo 1
Hielo (<i>Ice</i>)	12 %	Cluster de 4-9 tiles	Variable
Lodo (<i>Mud</i>)	12 %	Cluster de 4-9 tiles	Variable
Potenciación (<i>PowerUp</i>)	18 %	Tile individual	1-2

Tabla 4.5: Probabilidades y formato de distribución de tiles especiales.

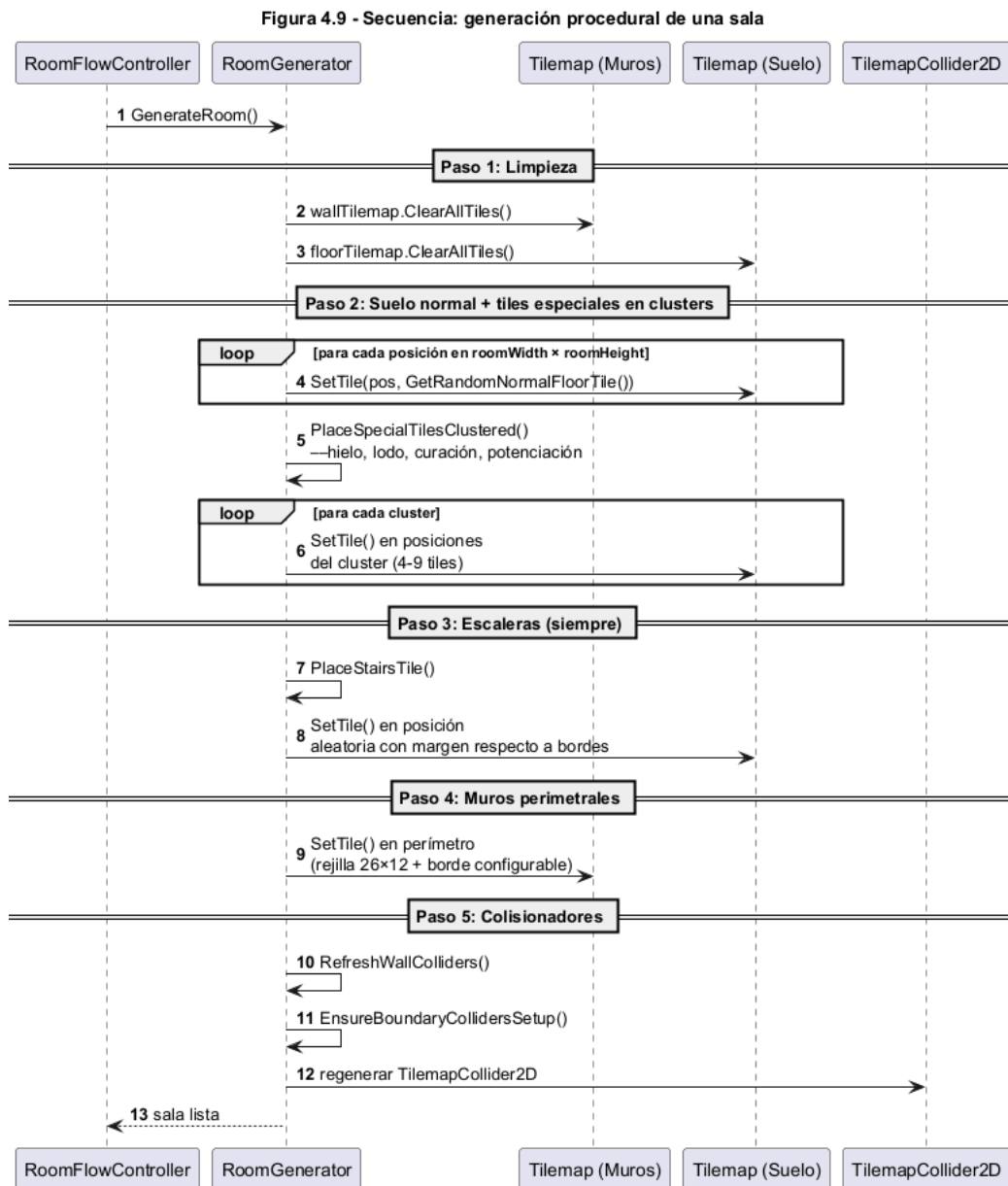


Figura 4.16: Diagrama de secuencia UML de la generación procedural de una sala. *Elaboración propia.*

Paso 3: Colocación de escaleras. Las escaleras se colocan en cada sala generada, en una posición aleatoria dentro de una zona segura definida por `stairsPaddingTiles` (margen configurable respecto a los bordes). El método `PlaceStairsTile()` aplica el margen sobre el rango de la rejilla y selecciona un punto aleatorio con `Random.Range`; si el padding es mayor que el espacio disponible, usa el centro como posición de respaldo. Las escaleras permanecen visibles desde el primer momento, pero el jugador no puede interactuar con ellas hasta que la sala se completa.

Paso 4: Generación de muros perimetrales. Se itera sobre las posiciones del perímetro de la rejilla y se coloca un tile de muro en cada posición mediante `wallTilemap.Set Tile()`, creando un recinto cerrado con colisionadores automáticos. El grosor del borde es configurable mediante `wallBorderThicknessTiles`.

Paso 5: Actualización de colisionadores. Se refresca el `TilemapCollider2D` de los muros mediante `RefreshWallColliders()`, y se garantiza la contención del jugador mediante `EnsureBoundaryCollidersSetup()`, que mantiene `BoxCollider2D` adicionales en los cuatro bordes como respaldo.

4.5.2 VariantTile: tiles con variantes visuales y selección determinista

La clase `VariantTile` extiende `TileBase` de Unity para soportar múltiples sprites por tile, proporcionando variedad visual sin afectar a la jugabilidad. Se seleccionó un sistema de *hash* determinista frente a la alternativa de selección aleatoria pura (`Random.Range`) porque esta última habría producido cambios visuales cada vez que el *tilemap* se redibuja (por ejemplo, al redimensionar la ventana del editor o al ocultar y mostrar la cámara), generando un efecto de parpadeo visual perceptible por el jugador:

```
1 public class VariantTile : TileBase {
2     public Sprite[] variants; // Array de variantes visuales para este tipo de tile
3     public bool deterministic = true;
4     public int seed = 1337;
5     public TileEffect tileEffect; // Efecto asociado (hielo, lodo, curacion, etc.)
6
7     public override void GetTileData(Vector3Int position, ITilemap tilemap,
8         ref TileData tileData) {
9         if (variants != null && variants.Length > 0) {
10             int index;
11             if (deterministic) {
12                 unchecked {
13                     int hash = seed;
14                     hash = hash * 31 + position.x;
15                     hash = hash * 17 + position.y;
16                     if (position.x != 0) hash ^= position.x << 2;
17                     if (position.y != 0) hash ^= position.y << 5;
18                     if (hash < 0) hash = -hash;
19                     index = hash % variants.Length;
20                 }
21             } else {
22                 index = Random.Range(0, variants.Length);
23             }
24             tileData.sprite = variants[index];
25         }
26     }
27 }
28 }
```

Línea por línea:

- `public Sprite[] variants`: array configurado desde el inspector de Unity que contiene las variantes visuales de un mismo tipo de tile.
- `public bool deterministic`: habilita la selección determinista (estable) de sprite por posición. Si se desactiva, el sprite se selecciona de forma pseudoaleatoria, lo que puede producir variaciones visuales entre repintados.

- `public int seed`: semilla adicional que permite variar la distribución sin perder determinismo.
- `unchecked ...`: asegura que la aritmética de enteros permite desbordamientos sin lanzar excepciones, lo que se utiliza como parte de la mezcla del hash.
- `hash = hash * 31 + position.x; hash = hash * 17 + position.y; ... index = hash % variants.Length`: cálculo de un hash simple que mezcla la semilla y las coordenadas del tile para obtener un índice estable dentro del rango de variantes.

4.5.3 TileEffect: efectos de tiles

Los efectos de tiles se definen como *ScriptableObjects*, lo que permite crear y configurar nuevos efectos desde el inspector de Unity sin modificar el código:

```
1 [CreateAssetMenu(menuName = "Tiles/TileEffect")]
2 public class TileEffect : ScriptableObject {
3     public string tileName;
4     public TileEffectType effectType = TileEffectType.None;
5
6     public int healthChange = 0;
7     public float speedMultiplier = 1f;
8     public float effectDuration = 0f;
9
10    public float tickRate = 1f;
11    public bool continuous = false;
12
13    public Color tintColor = Color.white;
14    public GameObject vfxPrefab;
15 }
```

La enumeración `TileEffectType` define los tipos de efecto posibles:

```
1 public enum TileEffectType {
2     None,           // Sin efecto
3     Heal,           // Restaura vida
4     Damage,         // Inflige dano
5     SpeedUp,        // Incrementa velocidad
6     SpeedDown,      // Reduce velocidad
7     Ice,            // Tile de hielo
8     Mud,            // Tile de lodo
9     PowerUp         // Incrementa dano
10 }
```

4.5.4 FloorTileManager: aplicación de efectos

El `FloorTileManager` detecta sobre qué tile se encuentra el jugador en cada fotograma y aplica el efecto correspondiente:

```
1 void Update() {
2     if (floorTilemap == null || playerTransform == null) return;
3
4     Vector3Int tilePos = floorTilemap.WorldToCell(playerTransform.position);
5
6     if (tilePos != currentTilePos) {
7         OnExitTile();
8         currentTilePos = tilePos;
9         OnEnterTile();
10    }
```

```
10 }
11
12 if (currentEffect?.continuous == true) {
13     if (Time.time >= lastEffectTime + currentEffect.tickRate) {
14         ApplyEffect(currentEffect);
15         lastEffectTime = Time.time;
16     }
17 }
18 }
```

La conversión de la posición del jugador en coordenadas de mundo a coordenadas de tile se realiza mediante `WorldToCell`. Cuando el jugador cambia de tile, se ejecutan las funciones de salida del tile anterior y entrada al nuevo tile. Para los efectos con el indicador (*flag*) `continuous` activado, el efecto se aplica periódicamente según la frecuencia definida en `tickRate`.

Seguimiento visual de efectos. Al instanciar los efectos visuales (VFX) de un tile, se añade el componente `VFXFollower` al objeto instanciado para que el efecto visual siga la posición de la entidad afectada (jugador o enemigo). Sin este componente, el VFX quedaría fijo en la posición de instanciación mientras la entidad se desplaza, produciendo una desincronización visual.

Persistencia de efectos de velocidad. Los efectos de modificación de velocidad (hielo, lodo) persisten durante un periodo configurable tras abandonar el tile, evitando que el efecto desaparezca bruscamente:

```
1 private void ApplySpeedModifier(float multiplier) {
2     if (playerController == null) return;
3     playerController.SetSpeedMultiplier(multiplier);
4     hasSpeedModifier = true;
5 }
6
7 private IEnumerator RemoveSpeedAfterDelay(float seconds) {
8     yield return new WaitForSeconds(seconds);
9
10    if (hasSpeedModifier && playerController != null) {
11        playerController.ResetSpeed();
12        hasSpeedModifier = false;
13    }
14 }
```

La separación de responsabilidades es deliberada: `ApplySpeedModifier` solo aplica el multiplicador al controlador del jugador, mientras que la persistencia del efecto tras abandonar el tile se gestiona en el bloque `ApplyEffect()`, donde se lanza la corutina `RemoveSpeedAfterDelay(persistEffectSeconds)` si el tiempo de persistencia es mayor que cero.

Integración con enemigos. Los efectos de velocidad de los tiles también se aplican a los enemigos mediante el componente `EnemyTileEffectReceiver`, que se añade automáticamente a cada enemigo al ser instanciado por el `EnemyManager`. Este componente consulta el tile sobre el que se encuentra el enemigo y aplica el multiplicador de velocidad correspondiente a través de `Enemy.SetTileSpeedMultiplier()`, lo que permite al jugador utilizar estas zonas de forma estratégica.

4.5.5 DamageBoostManager: bonificación de daño

El DamageBoostManager gestiona las bonificaciones de daño otorgadas por los tiles de potenciación. Implementa el patrón *Singleton* y soporta bonificaciones permanentes y temporales:

```
1 public void AddDamageBoostTimed(int amount, float duration) {  
2     if (duration <= 0f) {  
3         AddDamageBoost(amount); // Permanente  
4         return;  
5     }  
6  
7     AddDamageBoost(amount);  
8     StartCoroutine(RemoveDamageBoostAfterDelay(amount, duration));  
9 }  
10  
11 public int GetBoostedDamage(int baseDamage) {  
12     return baseDamage + globalDamageBoost;  
13 }
```

Los proyectiles consultan al DamageBoostManager en su inicialización para calcular su daño efectivo, lo que permite que la bonificación se aplique de forma global y coherente a todas las fuentes de daño del jugador.

4.6 SISTEMA DE GESTIÓN DE SALAS

Evolución iterativa. El sistema de gestión de salas se vio directamente afectado por el rediseño de la generación procedural. Cuando cada sala era una escena independiente, el RoomController no existía como tal: la lógica de escaleras y progresión residía en scripts de cada escena. Al consolidar todo el juego en una única escena, se creó el RoomController como componente central para gestionar la detección de entrada del jugador, el bloqueo de escaleras durante el combate y la detección de sala completada.

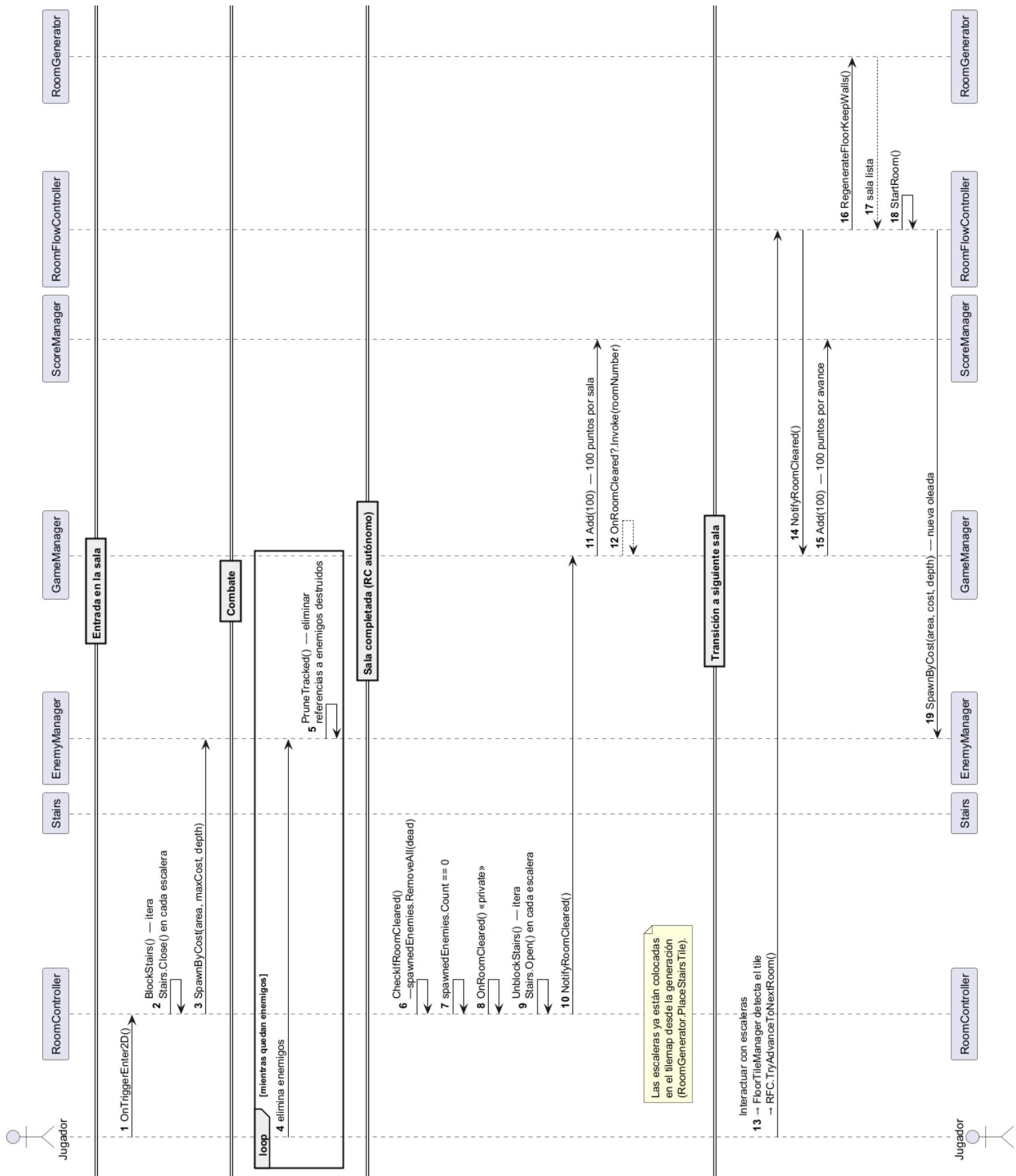
4.6.1 RoomController: control de sala individual

El RoomController gestiona el ciclo de vida de una sala individual: la detección de entrada del jugador, la activación de la aparición de enemigos, el bloqueo de escaleras durante el combate y la detección de sala completada.

```
1 void OnTriggerEnter2D(Collider2D other) {  
2     if (other.CompareTag("Player")) {  
3         playerInside = true;  
4  
5         if (!isCleared) {  
6             CloseAllDoors();  
7         }  
8     }  
9 }
```

Al detectar la entrada del jugador mediante el sistema de *triggers* de Unity, el controlador bloquea las escaleras de la sala y activa la aparición de enemigos. Las escaleras permanecen bloqueadas hasta que todos los enemigos han sido eliminados, momento en el cual se habilitan automáticamente para permitir la progresión.

Figura 4.11 - Secuencia: ciclo de progresión entre salas



4.6.2 Stairs: lógica de escaleras

El componente de escaleras (implementado como clase `Door` por cuestiones de diseño) gestiona el estado visual y funcional de acceso:

```
1 public void Open(bool playEffects = true) {  
2     if (isOpen) return;  
3     isOpen = true;  
4     doorCollider.enabled = false;  
5 }  
6  
7 public void Close(bool playEffects = true) {  
8     isOpen = false;  
9     doorCollider.enabled = true;  
10 }
```

La escalera alterna entre un estado bloqueado y un estado habilitado.

4.7 SISTEMA DE INTERFAZ DE USUARIO

Evolución iterativa. La interfaz de usuario fue el último subsistema en integrarse con el patrón *Observer*. En las primeras iteraciones, cada componente de UI consultaba directamente al `GameManager` en su método `Update()` (lectura síncrona por sondeo), lo que generaba comprobaciones redundantes en cada fotograma y acoplaba la interfaz con la implementación interna del gestor. La adopción del patrón *Observer* (suscripción al evento `OnGameStateChanged`) eliminó esta dependencia: los paneles de pausa, derrota y victoria solo ejecutan lógica cuando reciben una notificación de cambio de estado, no en cada fotograma. El sistema de corazones evolucionó de una implementación estática (número fijo de corazones en la escena) a una generación dinámica basada en `maxHealth`, lo que permitió que las clases de personaje con distinta cantidad de vida máxima se representasen correctamente sin configuración manual.

4.7.1 HeartDisplay: representación de vida

El `HeartDisplay` proporciona una representación visual dinámica de la vida del jugador mediante iconos de corazones. El sistema crea los corazones de forma dinámica en función de la vida máxima del personaje:

```
1 public void InitializeHearts(int maxHealth) {  
2     ClearHearts();  
3     maxHearts = Mathf.Max(0, maxHealth);  
4  
5     for (int i = 0; i < maxHearts; i++) {  
6         CreateHeart(i);  
7     }  
8 }
```

La actualización responde a los cambios en la vida del jugador, alternando los sprites de corazón lleno y vacío:

```
1 public void UpdateHearts(int currentHealth) {  
2     int clampedHealth = Mathf.Clamp(currentHealth, 0, maxHearts);  
3  
4     for (int i = 0; i < heartImages.Count; i++) {
```

4.7. SISTEMA DE INTERFAZ DE USUARIO

```
5      Image heartImage = heartImages[i];  
6      if (heartImage == null) continue;  
7  
8      heartImage.sprite = i < clampedHealth ? heartFull : heartEmpty;  
9      heartImage.enabled = true;  
10     }  
11 }
```

4.7.2 PauseMenu: menú de pausa

El PauseMenu se suscribe al evento `OnGameStateChanged` del `GameManager` y activa o desactiva su panel según el estado actual del juego:

```
1 void Start() {  
2     EnsureRuntimeUIIfMissing();  
3     WireButtonsOnce();  
4  
5     if (pauseMenuPanel != null)  
6         pauseMenuPanel.SetActive(false);  
7  
8     GameManager.OnGameStateChanged += OnGameStateChanged;  
9 }  
10  
11 private void OnGameStateChanged(GameState newState) {  
12     EnsureRuntimeUIIfMissing();  
13     if (pauseMenuPanel == null) return;  
14  
15     bool shouldShow = (newState == GameState.Paused);  
16     pauseMenuPanel.SetActive(shouldShow);  
17 }
```

Este diseño ejemplifica la aplicación del patrón *Observer*: el menú de pausa no consulta activamente el estado del juego, sino que reacciona a la notificación de cambio de estado emitida por el `GameManager`. Las llamadas a `EnsureRuntimeUIIfMissing()` y `WireButtonsOnce()` garantizan que la interfaz de pausa exista y tenga sus botones configurados incluso cuando el `PauseMenu` se crea en tiempo de ejecución sin referencias previas asignadas en el inspector. Esto permite añadir o eliminar elementos de interfaz reactivos sin modificar el código del gestor central.

4.7.3 Pantallas de fin de partida

Las pantallas de derrota (`GameOverScreen`) y victoria (`VictoryScreen`) se activan automáticamente al cambiar el estado del juego a `GameOver` o `Victory`, respectivamente:

```
1 public void Show() {  
2     if (victoryPanel == null) return;  
3     victoryPanel.SetActive(true);  
4  
5     if (scoreText != null && ScoreManager.Instance != null)  
6         scoreText.text = $"Puntuacion Final: {ScoreManager.Instance.CurrentScore}";  
7     if (roomsClearedText != null && GameManager.Instance != null)  
8         roomsClearedText.text = $"Salas limpiadas: {GameManager.Instance.RoomsCleared}";  
9     if (enemiesKilledText != null && GameManager.Instance != null)  
10        enemiesKilledText.text = $"Enemigos eliminados: {GameManager.Instance.EnemiesKilled}";  
11  
12     if (timeText != null) {  
13         float runTime = Time.time - runStartTime;  
14         int minutes = Mathf.FloorToInt(runTime / 60f);  
15     }  
16 }
```

```

14     int seconds = Mathf.FloorToInt(runTime % 60f);
15     timeText.text = $"Tiempo: {minutes:00}:{seconds:00}";
16 }
17 }

```

Ambas pantallas muestran un texto que representa el final de la partida y redirigen al menú principal tras un tiempo.



Figura 4.18: Captura de pantalla del menú de pausa de *Ascension*, mostrando las opciones de reanudar y menú principal. *Elaboración propia.*

4.7.4 ScoreboardMenuUI: historial de puntuaciones

El historial de puntuaciones se genera dinámicamente a partir de los datos almacenados por el `ScoreManager`. En lugar de instanciar un *prefab* por cada entrada, el componente utiliza un único campo `TextMeshProUGUI` (un sistema de renderizado de texto de alta calidad integrado en Unity que soporta formato enriquecido, sombreado y alineación precisa a nivel de píxel) y construye todo el listado mediante un `StringBuilder`, lo que resulta más eficiente en memoria y evita la necesidad de gestionar la destrucción de objetos instanciados:

```

1 public void Refresh() {
2     if (listText == null) return;
3
4     var sb = new StringBuilder();
5     var entries = ScoreManager.Instance != null
6         ? ScoreManager.Instance.GetHistory() : null;
7
8     sb.AppendLine("SCORES");
9     sb.AppendLine("-----");
10
11     if (entries == null || entries.Count == 0) {
12         sb.AppendLine("(sin partidas guardadas)");
13         listText.text = sb.ToString();
14         return;
15     }
16 }

```

```
17  int rank = 1;
18  foreach (var e in entries) {
19      sb.Append(rank).Append(" . ")
20      .Append(e.score).Append(" pts")
21      .Append(" | ").Append(e.result)
22      .Append(" | ").Append(e.playerClass)
23      .Append(" | R ").Append(e.roomsCleared)
24      .Append(" | K ").Append(e.enemiesKilled)
25      .AppendLine();
26      rank++;
27  }
28
29  listText.text = sb.ToString();
30 }
```

4.8 SISTEMAS PREPARADOS PARA EXTENSIÓN

Evolución iterativa. Ambos sistemas descritos en esta sección (jefes con fases y generación procedural de armas) se desarrollaron como prototipos funcionales durante iteraciones intermedias del proyecto. La decisión de no integrarlos en el bucle de juego final se tomó tras evaluar la prioridad de las tareas restantes: el equilibrado de dificultad de los jefes con fases múltiples y la interfaz de recogida de armas generadas requerían un volumen de trabajo que habría comprometido la entrega de los subsistemas prioritarios. Se optó por mantener el código desarrollado como extensión documentada, alineándose con el principio de apertura-cierre: la arquitectura permite su integración futura sin modificar los subsistemas existentes.

La arquitectura del proyecto se ha diseñado teniendo en cuenta la extensibilidad. Se han desarrollado dos sistemas cuya estructura se ha preparado a nivel de código pero que no se encuentran completamente integrados en el bucle de juego al cierre del presente trabajo:

4.8.1 Sistema de jefes con fases múltiples

Se ha desarrollado el componente `BossController` que implementa un sistema de jefes con dos fases diferenciadas basadas en el porcentaje de vida restante. El diseño contempla la transición automática a la segunda fase al alcanzar el 50 % de vida, con cambios en la cadencia de ataque, la velocidad de movimiento y los patrones de disparo. No obstante, al no encontrarse integrado en el bucle de progresión de salas, su funcionamiento en una partida completa no se ha verificado de forma consistente; su integración requiere trabajo adicional en el equilibrado de dificultad y en la definición de patrones de ataque más variados.

4.8.2 Generación procedural de armas con rareza

Se ha desarrollado el componente `WeaponGenerator` que implementa un sistema de generación procedural de armas con cuatro niveles de rareza (Común, Poco Común, Raro, Legendario) y escalado de estadísticas basado en la profundidad de la mazmorra. Al no encontrarse integrado en el flujo de juego (aparición de armas como objetos recogibles en las salas), su funcionamiento en una partida completa no se ha verificado de forma consistente; la integración constituye un desarrollo pendiente para futuras iteraciones del proyecto.

Estos dos sistemas se analizan con mayor detalle en el apartado de líneas de trabajo futuro.

4.9 COMPILACIÓN Y EJECUCIÓN

Evolución iterativa. Las primeras versiones del proyecto se ejecutaban exclusivamente desde el editor de Unity durante el desarrollo. La generación del ejecutable final se realizó una vez estabilizados los subsistemas principales, verificando que el comportamiento del juego compilado fuese idéntico al observado en el editor.

4.9.1 Proceso de compilación

La compilación del proyecto se realiza desde la ventana *Build Profiles* del editor de Unity (File > Build Profiles). La configuración utilizada es la siguiente:

- **Motor:** Unity 6 (versión 6000.2.7f2).
- **Plataforma objetivo:** Windows, arquitectura Intel 64-bit.
- **Backend de scripting:** Mono (compatible con .NET Standard 2.1).
- **Escenas incluidas en el *build*:** se incluyen las tres escenas del proyecto en el orden definido por el flujo de juego:

1. Scenes/MainMenu (índice 0), menú principal.
2. Scenes/ClassSelection (índice 1), pantalla de selección de clase.
3. Scenes/GameScene (índice 2), escena principal de juego.

- **Opciones de compilación desactivadas:** *Copy PDB files*, *Create Visual Studio Solution* y *Development Build*. Esto genera un ejecutable de producción optimizado, sin símbolos de depuración ni herramientas de *profiling*.

El orden de las escenas es relevante: Unity carga automáticamente la escena con índice 0 al iniciar el ejecutable, por lo que MainMenu debe ocupar la primera posición para que el jugador acceda al menú principal al arrancar el juego.

Para generar el ejecutable, se selecciona la plataforma *Windows* en la ventana *Build Profiles*, se verifica que las tres escenas estén incluidas y ordenadas correctamente, y se pulsa *Build*. Unity compila los scripts de C# mediante el compilador Roslyn integrado, empaqueta los activos del proyecto (sprites, tilemaps, prefabs, ScriptableObjects) en archivos binarios optimizados y genera el ejecutable junto con sus dependencias.

4.9.2 Estructura del ejecutable

El proceso de compilación genera la siguiente estructura de archivos en el directorio de salida:

1	Build/	
2	Ascension.exe	Ejecutable principal
3	UnityPlayer.dll	Motor de ejecución de Unity
4	UnityCrashHandler64.exe	Gestor de errores críticos
5	Ascension_Data/	Activos compilados del proyecto
6	level0, level1, level2	Escenas empaquetadas
7	resources.assets	Activos marcados como Resources
8	sharedassets02.assets	Activos específicos de cada escena
9	globalgamemangers	Configuración global del proyecto
10	Managed/	Ensamblados .NET compilados
11	Plugins/x86_64/	Bibliotecas nativas
12	MonoBleedingEdge/	Runtime de Mono (.NET)
13	D3D12/	Bibliotecas de renderizado DirectX 12

Los archivos level0, level1 y level2 corresponden a las tres escenas del proyecto en el orden definido en *Build Profiles*. El directorio Managed/ contiene los ensamblados compilados a partir de los scripts de C# del proyecto (Assembly-CSharp.dll) y las bibliotecas del framework de Unity.

4.9.3 Ejecución del juego

Para ejecutar el juego, el usuario debe:

1. Copiar la carpeta Build/ completa a la ubicación deseada, manteniendo la estructura de directorios intacta.
2. Ejecutar Ascension.exe.

No es necesario instalar Unity ni ningún componente adicional: el ejecutable incluye el *runtime* de Mono y las bibliotecas del motor de Unity necesarias para la ejecución autónoma. El juego se inicia mostrando la pantalla del menú principal, desde la cual el jugador puede acceder a la selección de clase, consultar el historial de puntuaciones o salir del juego.

Requisitos mínimos del sistema:

- Sistema operativo: Windows 10 (64 bits) o posterior.
- Procesador: compatible con x86_64.
- Memoria: 4 GB de RAM.
- Gráficos: compatible con DirectX 11 o DirectX 12.
- Almacenamiento: aproximadamente 100 MB de espacio en disco.

5 CONCLUSIONES Y TRABAJO FUTURO

El presente capítulo recoge las conclusiones del trabajo realizado, evaluando el grado de cumplimiento de los objetivos establecidos, identificando las limitaciones del proyecto y proponiendo líneas de trabajo futuro.

5.1 CUMPLIMIENTO DE OBJETIVOS

A continuación se evalúa el grado de cumplimiento de cada uno de los objetivos específicos definidos en la introducción.

OE-1. Diseñar e implementar una arquitectura modular basada en patrones de diseño. Se considera cumplido. La arquitectura del proyecto aplica patrones de diseño de forma consistente, en particular *Singleton*, *Observer*, *Strategy* y *Template Method*. Adicionalmente, se emplean mecanismos propios de Unity (sistema de componentes) y una máquina de estados para la coordinación del flujo del juego. La modularidad y la separación entre lógica y presentación se verificaron inicialmente de forma manual durante el desarrollo y, posteriormente, mediante el análisis estático automatizado, que confirmó la presencia de los cuatro *Singletons* documentados, las cuatro declaraciones de eventos *Observer* y las cinco clases *ScriptableObject* previstas.

OE-2. Diseñar e implementar un sistema de generación procedural de salas. Se considera cumplido. Se ha desarrollado el componente *RoomGenerator*, que genera salas de 26×12 tiles con muros perimetrales, tiles normales con variantes visuales y tiles especiales distribuidos mediante un algoritmo de expansión por *clusters*. La generación introduce variabilidad en la disposición de elementos entre partidas.

OE-3. Desarrollar una inteligencia artificial diferenciada para al menos tres tipos de enemigo. Se considera cumplido. Se han implementado tres tipos de enemigo con comportamientos distintos: el slime verde (ataque a distancia con mantenimiento de distancia objetivo), el slime rojo (persecución directa) y el slime azul (aproximación por saltos, desplazamientos rápidos o *dash*). Los tres tipos aplican un periodo de gracia tras su aparición y reaccionan a tiles que modifican la velocidad. Asimismo, se ha implementado un sistema de enemigos especiales con atributos escalados que actúan como jefes periódicos. Su diseño e implementación incluyen la formalización de estados y transiciones (FSM) y fragmentos de código del bucle de IA. La comprobación funcional se realizó en *Play Mode* (editor de Unity) observando transiciones de estado, ventanas de gracia y reacción a tiles; adicionalmente, el *stress test* aporta evidencia de estabilidad del subsistema bajo alta carga de instanciación y destrucción de enemigos.

OE-4. Crear un sistema de armas con mecánicas de combate cuerpo a cuerpo y a distancia. Se considera cumplido. El sistema de armas, basado en el patrón *Strategy*, ofrece dos modalidades de combate diferenciadas: *MeleeWeapon* (ataque de barrido con animación procedural sinusoidal) y *RangedWeapon* (disparo de proyectiles dirigidos hacia el cursor). La implementación abarca la clase base *Weapon*, las subclases, los proyectiles, los datos *Weapon Data* y el flujo de daño hasta la puntuación. La verificación se realizó de forma manual en el editor de Unity, comprobando: activación del ataque, generación de proyectiles, colisiones/hit-boxes, *cooldowns* y aplicación de daño. De forma complementaria, los escenarios de estrés ejercitan el uso intensivo de instanciación/destrucción asociada a proyectiles y enemigos.

OE-5. Implementar un sistema de tiles con efectos que modifiquen la jugabilidad. Se considera cumplido. Se han desarrollado cuatro tipos de tiles con efectos: hielo (incrementa velocidad), lodo (reduce velocidad), curación (restaura vida) y potenciación (incrementa daño). Los efectos se configuran mediante *ScriptableObjects* y afectan tanto al jugador como a los enemigos. La distribución se realiza mediante el algoritmo de expansión por *clusters* con probabilidades configurables. Su comprobación se realizó verificando manualmente en el editor de Unity (aplicación/retirada de efectos durante la partida).

OE-6. Implementar un sistema de persistencia de puntuaciones y estadísticas. Se considera cumplido. El *ScoreManager* gestiona la puntuación en tiempo real, el registro de estadísticas por partida y la persistencia del historial mediante *PlayerPrefs* (almacenamiento local clave-valor) y serialización JSON (formato de texto estructurado). El diseño y la implementación abarcan la persistencia y estructura del historial, y se muestra el consumo de estos datos en la interfaz del menú principal. El historial almacena hasta 30 entradas con puntuación, clase, salas completadas, enemigos eliminados, resultado y marca de tiempo. Su comprobación se realizó de forma manual, ejecutando partidas consecutivas y verificando la persistencia de los datos entre ejecuciones del juego.

OE-7. Documentar el proceso de desarrollo y las decisiones técnicas adoptadas. Se considera cumplido. La memoria describe los requisitos, la arquitectura, los módulos implementados, los patrones de diseño aplicados, los resultados obtenidos, las limitaciones y las líneas de trabajo futuro, apoyándose en diagramas y fragmentos de código. La trazabilidad se apoya en la relación entre los objetivos, las historias de usuario y los criterios de aceptación definidos y la valoración del cumplimiento recogida en este capítulo.

5.2 VALORACIÓN DEL TRABAJO REALIZADO

El proyecto *Ascension* ha permitido aplicar de forma integrada los conocimientos adquiridos durante el Grado en Ingeniería Informática en áreas como la ingeniería del software, la programación orientada a objetos, la algoritmia y el diseño de sistemas interactivos.

Desde el punto de vista cuantitativo, a fecha de redacción de esta memoria, el directorio *Assets/Scripts/* contiene **85** scripts C# de proyecto con un total de **12.565** líneas de código. La aportación técnica del trabajo se fundamenta en la implementación e integración de sistemas como la generación procedural de contenido, la inteligencia artificial de los enemigos con tres comportamientos diferenciados y la arquitectura guiada por patrones de diseño.

5.3. LIMITACIONES DEL PROYECTO

La utilización de *ScriptableObjects* como mecanismo de separación entre datos y lógica ha permitido iterar sobre el equilibrado del juego (estadísticas de clases, datos de enemigos, efectos de tiles) sin recompilar el código fuente en cada modificación.

El proceso de desarrollo ha requerido ciclos de prueba funcional y depuración, con especial atención a la coherencia entre subsistemas (generación procedural, combate, IA, interfaz y persistencia) y a la estabilidad del flujo de juego.

5.3 LIMITACIONES DEL PROYECTO

A pesar de los resultados obtenidos, se han identificado las siguientes limitaciones:

1. **Ausencia de sistema de audio.** El proyecto no incluye un sistema de gestión de efectos de sonido ni música. La implementación de un `AudioManager` centralizado con mezcla de canales constituye una mejora necesaria para proporcionar una experiencia completa.
2. **Variedad limitada de contenido.** El juego dispone de tres tipos de enemigo y un conjunto reducido de armas. La generación procedural de armas con sistema de rareza (`WeaponGenerator`) está implementada a nivel de código pero no integrada en el flujo de juego; en consecuencia, su funcionamiento dentro de una partida completa no se ha verificado de forma consistente.
3. **Sistema de jefes no integrado.** El `BossController` implementa un sistema de dos fases con patrones de ataque diferenciados, pero no se encuentra integrado en el flujo de juego de la versión actual. En consecuencia, su funcionamiento dentro de una partida completa no ha podido verificarse de forma consistente.
4. **Ausencia de reutilización de objetos (*object pooling*).** Los proyectiles y efectos visuales se crean y destruyen mediante `Instantiate` y `Destroy`, lo que puede generar presión sobre el recolector de basura en situaciones con alto volumen de proyectiles. La implementación de un sistema de reutilización de objetos (*object pooling*) mejoraría el rendimiento en estos escenarios.
5. **Cobertura de pruebas.** El proyecto no dispone de una suite formal de pruebas unitarias ni de integración con cobertura sistemática de todos los módulos. La implementación de un marco de pruebas más completo mejoraría la detección temprana de regresiones.

5.4 LÍNEAS DE TRABAJO FUTURO

A partir de las limitaciones identificadas y de las oportunidades de extensión detectadas durante el desarrollo, se proponen las siguientes líneas de trabajo futuro:

5.4.1 Integración del sistema de jefes con fases múltiples

El componente `BossController` implementa un sistema de dos fases con transición automática al 50 % de vida. La primera fase presenta ataques moderados con cadencia lenta,

mientras que la segunda fase introduce ráfagas de proyectiles a mayor velocidad. La integración completa de este sistema implica:

- Diseño de patrones de ataque variados para cada fase
- Equilibrado de las estadísticas del jefe en relación con la profundidad de la partida
- Implementación de retroalimentación visual y auditiva para la transición de fase
- Integración con el sistema de puntuación con bonificaciones específicas por derrotar jefes

5.4.2 Integración del sistema de generación procedural de armas

El componente `WeaponGenerator` implementa a nivel de código un sistema de cuatro niveles de rareza (Común con multiplicador $\times 1,0$; Poco Común con $\times 1,3$; Raro con $\times 1,6$; Legendario con $\times 2,0$) y escalado de estadísticas basado en la profundidad (+10 % por nivel). Dado que el sistema no se encuentra integrado en el flujo de juego, su validación se considera pendiente. La integración completa implica:

- Aparición de armas como objetos recogibles en las salas
- Sistema de interfaz para comparar armas y decidir si equipar la nueva
- Retroalimentación visual diferenciada según la rareza del arma
- Equilibrado de las probabilidades de aparición por profundidad

5.4.3 Ampliación del contenido

- Nuevos tipos de enemigo con comportamientos diferenciados
- Nuevos tipos de tiles con efectos adicionales (veneno, teletransporte)
- Nuevos tipos de sala (salas de tesoro, salas de tienda, salas trampa)
- Sistemas de progresión permanente entre partidas (*meta-progression*)

5.4.4 Mejoras técnicas

- Implementación de *object pooling* para proyectiles y efectos visuales
- Sistema de gestión de audio centralizado
- Creación de pruebas unitarias y de integración con cobertura sistemática
- Optimización de rendimiento mediante perfilado con *Unity Profiler*
- Sistema de guardado y carga del progreso de partida

5.5 REFLEXIÓN FINAL

El desarrollo del videojuego *Ascension* ha constituido un ejercicio integrador que ha permitido abordar, en un marco práctico, retos técnicos propios del desarrollo de software con múltiples subsistemas: diseño arquitectónico, implementación de algoritmos, aplicación de patrones de diseño y gestión de la calidad del código.

El subgénero *roguelike* constituye un contexto adecuado para la demostración de competencias de ingeniería del software, ya que exige la integración de múltiples subsistemas (generación procedural, inteligencia artificial, gestión de estados e interacción con el entorno) en un sistema cohesionado y funcional.

La arquitectura modular adoptada, fundamentada en patrones de diseño de la ingeniería del software y en mecanismos propios de Unity (*ScriptableObjects* y sistema de componentes), permite extender el sistema mediante la incorporación de nuevos subsistemas. En particular, se han implementado estructuras para un sistema de jefes con fases y para la generación procedural de armas, cuya integración en el flujo de juego se identifica como trabajo futuro.

BIBLIOGRAFÍA

- [1] Newzoo, "Newzoo global games market report 2020 | light version," 2020, versión gratuita del informe del mercado global de videojuegos; incluye estimaciones de ingresos globales (2020: 159.3B USD). [Online]. Available: <https://newzoo.com/insights/trend-reports/newzoo-global-games-market-report-2020-light-version/>
- [2] J. Ryan, "The game awards 2020: Every result, winner, world premiere, trailers and more," 2020, lista de ganadores y resultados de The Game Awards 2020 (incluye el premio a Best Indie Game). [Online]. Available: <https://www.cnet.com/news/the-game-awards-2020-every-result-winner-world-premiere-trailers-and-more/>
- [3] The FreeBSD Project, "Rogue(6): exploring the dungeons of doom," 2026, manual page del juego Rogue; incluye autores y referencia al documento "A Guide to the Dungeons of Doom". [Online]. Available: <https://man.freebsd.org/cgi/man.cgi?query=rogue&sektion=6&manpath=FreeBSD+15.0-RELEASE+and+Ports>
- [4] International Roguelike Development Conference, "Berlin interpretation," 2008, criterios propuestos para la definición técnica del subgénero roguelike.
- [5] Valve Corporation, "Slay the spire (steam store page)," 2026, página de Steam; describe el juego como un deckbuilder que fusiona juegos de cartas y roguelikes. [Online]. Available: https://store.steampowered.com/app/646570/Slay_the_Spire/
- [6] N. Shaker, J. Togelius, and M. J. Nelson, *Procedural Content Generation in Games*. Springer, 2016, texto de referencia sobre técnicas de generación procedural de contenido en videojuegos.
- [7] A. M. Smith and M. Mateas, "Answer set programming for procedural content generation: A design space approach," in *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, 2011, pp. 187–200, generación de niveles de plataformas 2D mediante satisfacción de restricciones.
- [8] V. Volz, J. Schrum, J. Liu, S. M. Lucas, A. Smith, and S. Risi, "Evolving mario levels in the latent space of a deep convolutional generative adversarial network," in *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO)*, 2018, pp. 221–228, generación de niveles de videojuegos mediante redes neuronales generativas adversariales.
- [9] K. Perlin, "An image synthesizer," *ACM SIGGRAPH Computer Graphics*, vol. 19, no. 3, pp. 287–296, 1985, función de ruido para generación de texturas procedurales.

-
- [10] M. Gumin, “Wavefunctioncollapse,” <https://github.com/mxgmn/WaveFunctionCollapse>, 2016, implementación de referencia del algoritmo Wave Function Collapse.
- [11] I. Karth and A. M. Smith, “Wavefunctioncollapse is constraint solving in the wild,” in *Proceedings of the 12th International Conference on the Foundations of Digital Games (FDG)*, 2017, análisis formal de WFC como propagación de restricciones.
- [12] I. Millington and J. Funge, *Artificial Intelligence for Games*, 2nd ed. Morgan Kaufmann, 2009, texto de referencia sobre técnicas de inteligencia artificial aplicadas al desarrollo de videojuegos.
- [13] J. Orkin, “Applying goal-oriented action planning to games,” in *AI Game Programming Wisdom 2*. Charles River Media, 2003, introducción de GOAP en el videojuego F.E.A.R.
- [14] A. Juliani, V.-P. Berges, E. Teng, A. Cohen, J. Harper, C. Elion, C. Goy, Y. Gao, H. Hunter, K. Lange-Elkins *et al.*, “Unity: A general platform for intelligent agents,” *arXiv preprint arXiv:1809.02627*, 2018, framework Unity ML-Agents para aprendizaje por refuerzo en videojuegos.
- [15] Unity Technologies, “Unity for mobile game development,” 2026, página oficial: afirma que 70 %+ de los 1.000 juegos móviles top fueron hechos con Unity (Q4 2022; fuente Apptopia). [Online]. Available: <https://unity.com/solutions/mobile-game-development>
- [16] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994, referencia fundacional sobre patrones de diseño de software orientado a objetos.
- [17] U. Technologies, *Unity Manual 2023.3 LTS*. Unity Documentation, 2024, current industry standard for 2025 cross-platform deployment.
- [18] Microsoft, “Reflection in .net,” 2024, documentación oficial sobre reflexión en .NET para inspección de tipos, miembros y metadatos en tiempo de ejecución. [Online]. Available: <https://learn.microsoft.com/en-us/dotnet/framework/reflection-and-codedom/reflection>
- [19] R. Nystrom, *Game Programming Patterns*. Genever Benning, 2014, referencia sobre patrones de diseño aplicados específicamente al desarrollo de videojuegos. [Online]. Available: <https://gameprogrammingpatterns.com/>
- [20] J. Togelius, G. N. Yannakakis, K. O. Stanley, and C. Browne, “Search-based procedural content generation: A taxonomy and survey,” *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 3, no. 3, pp. 172–186, 2011, taxonomía y revisión sistemática de técnicas de generación procedural basadas en búsqueda.

A ARTEFACTOS SOFTWARE

Este apéndice recoge los principales artefactos software generados durante el desarrollo del Trabajo de Fin de Grado. Su inclusión complementa la memoria principal con una vista técnica de los entregables reales del proyecto.

Además, se proporciona un enlace al repositorio de GitHub donde se encuentra el código fuente completo del videojuego: <https://github.com/nsh255/TFE.git>

A.1 ESTRUCTURA DEL PROYECTO

El proyecto se ha desarrollado en Unity con una organización modular orientada a responsabilidades. A nivel práctico, los artefactos más relevantes del repositorio son:

- **Ascension/Ascension/Assets/Scripts**: lógica principal del juego en C#, organizada por subsistemas (Core, Player, Enemies, Weapons, UI, Tiles y LevelGeneration).
- **Ascension/Ascension/Assets/Prefabs**: objetos reutilizables del juego (jugador, enemigos, armas, escaleras y elementos de sala).
- **Ascension/Ascension/Assets/Scenes**: escenas del flujo completo (menú principal, selección de class y escena principal de juego).
- **Ascension/Ascension/LATEX/Diagrams**: fuentes PlantUML de los diagramas técnicos utilizados en la memoria.
- **Ascension/Ascension/LATEX**: memoria final, bibliografía y figuras integradas para compilación del documento.

Esta estructura ha permitido mantener bajo acoplamiento entre módulos y facilitar la iteración de mecánicas.

A.2 ARQUITECTURA DEL SISTEMA

La arquitectura del videojuego se basa en un enfoque modular con patrones *Singleton*, *Observer*, *Strategy* y *Template Method*. Entre los subsistemas implementados destacan:

- Gestión de estado global de partida (GameManager).

- Progresión de salas y coordinación de oleadas (RoomFlowController y EnemyManager).
- Generación procedural de salas y tiles con efectos (RoomGenerator y FloorTileManager).
- Sistema de jugador, combate y armas (cuerpo a cuerpo y a distancia).
- Interfaz de usuario y persistencia de puntuaciones.

Con el objetivo de aportar una visión estructural del sistema, en el siguiente apartado se incluyen los diagramas UML utilizados durante el diseño.

A.3 DIAGRAMAS DEL SISTEMA

La Figura A.1 muestra el diagrama de clases del sistema, en el que se representan las principales entidades del videojuego y sus relaciones. Este diagrama permite comprender la organización interna del código y la separación de responsabilidades entre los distintos componentes.

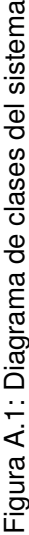


Figura A.1: Diagrama de clases del sistema

A.3.1 Diagrama de clases por secciones

Las Figuras A.2–A.4 dividen el diagrama de clases en secciones para mejorar la legibilidad.

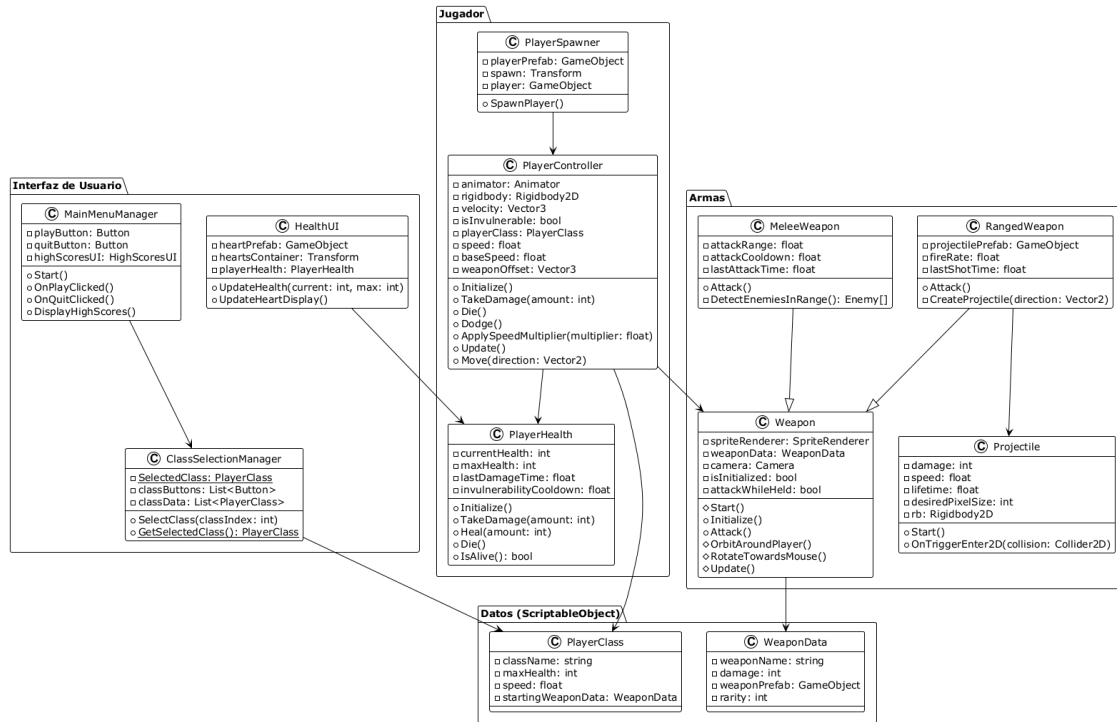


Figura A.2: Diagrama de clases del sistema (sección 1: interfaz de usuario, jugador y armas)

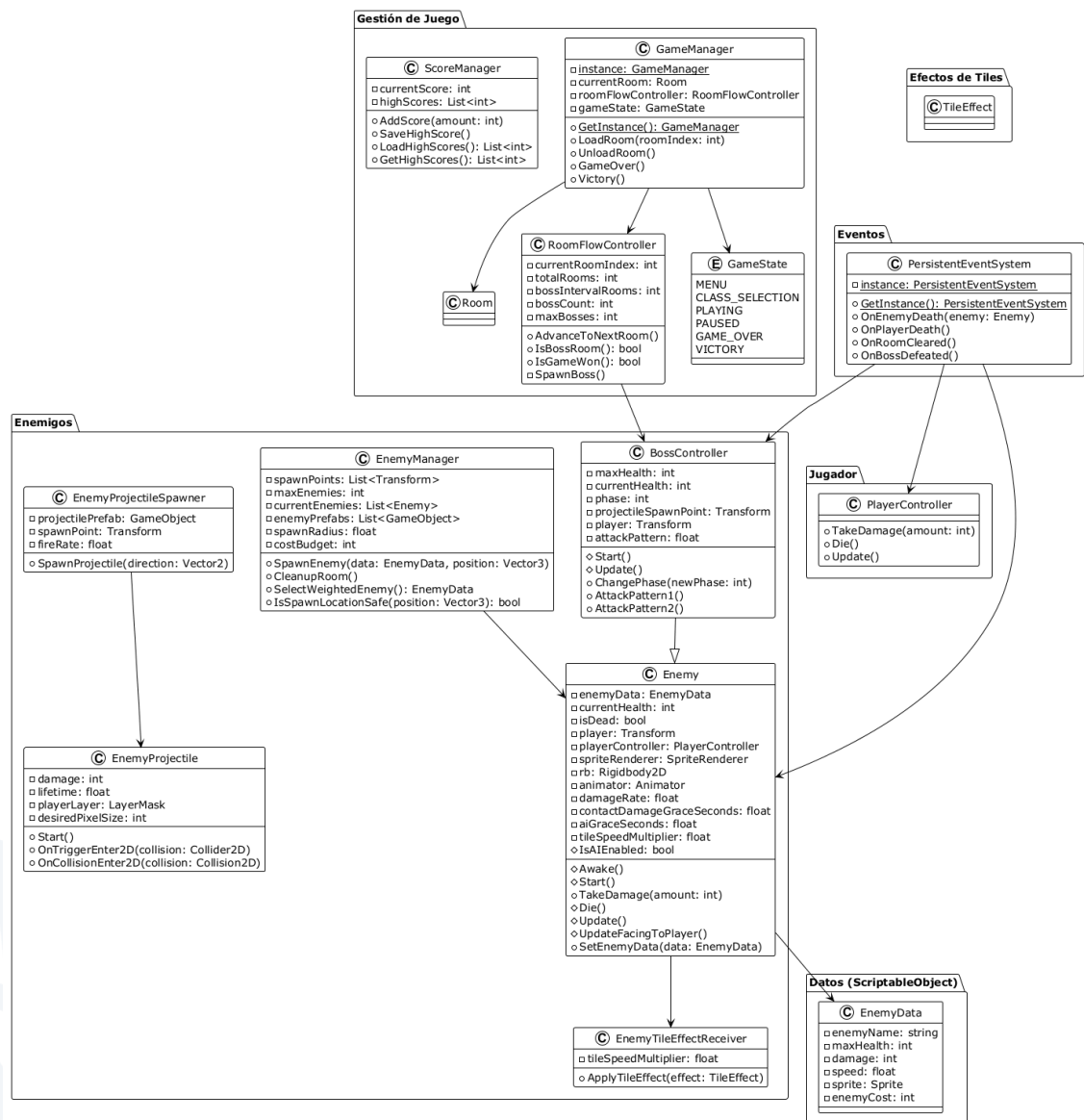


Figura A.3: Diagrama de clases del sistema (sección 2: gestión de juego, eventos y enemigos)

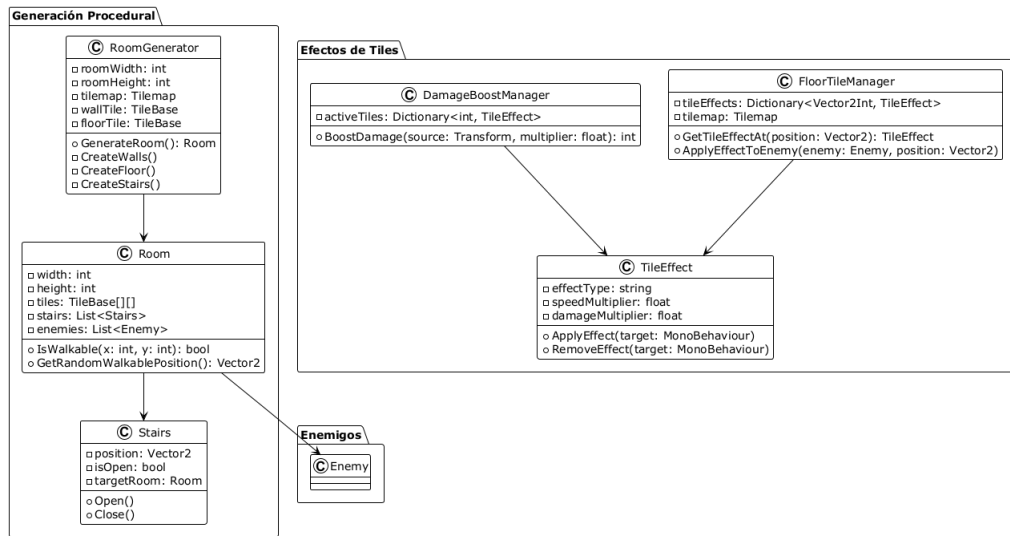


Figura A.4: Diagrama de clases del sistema (sección 3: generación procedural y efectos de tiles)

Por otro lado, la Figura A.5 presenta el diagrama de casos de uso, que describe las interacciones principales entre el jugador y el sistema, proporcionando una visión funcional del comportamiento esperado del videojuego.



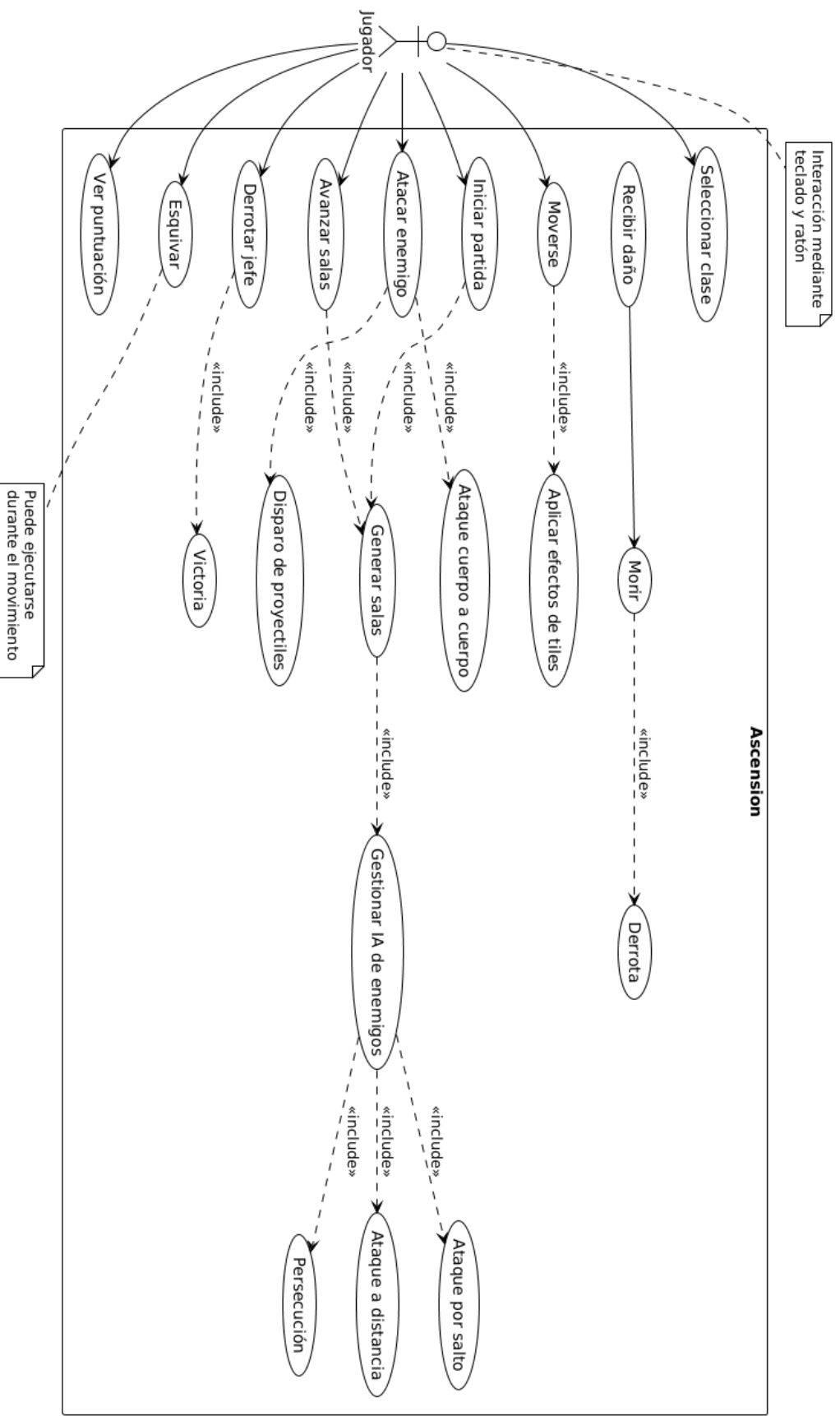


Figura A.5: Diagrama de casos de uso del sistema

Este Trabajo de Fin de Grado presenta Ascension, un videojuego 2D roguelike desarrollado en Unity. El sistema implementa generación procedural, IA enemiga, gestión de estados y un sistema de puntuación. La arquitectura utiliza ScriptableObjects y patrones de diseño (Singleton, Observer, Strategy, Template Method) para asegurar el desacoplamiento y la mantenibilidad del código. Los resultados confirman una solución robusta y extensible, con mejoras futuras previstas en el diseño de jefes finales y el sistema de audio.

Abstract. This Final Degree Project presents Ascension, a 2D roguelike game developed in Unity. The system features procedural generation, enemy AI, state management, and score tracking. The architecture employs ScriptableObjects and design patterns (Singleton, Observer, Strategy, Template Method) to ensure modularity and maintainability. Results demonstrate a robust and extensible architecture, with future work focusing on boss integration and the audio systems.

